

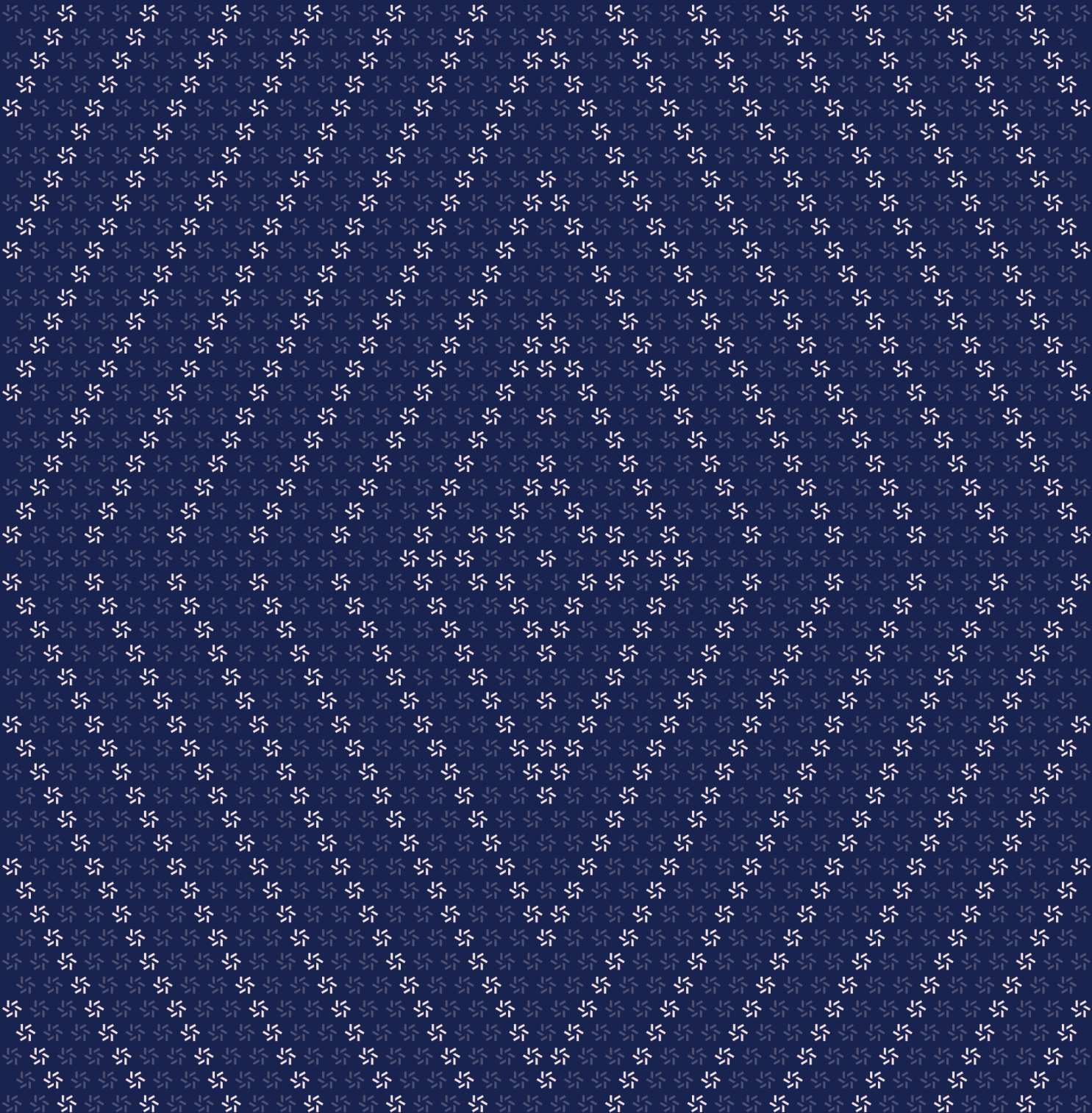


Prepared for
Will Beeson
Noah Foltz
Uniform Labs

Prepared by
Evan Hwang
Jaeu Kim
Oliver Murray
Zellic

July 22, 2026

Multiliquid EVM Smart Contract Security Assessment



Contents

About Zellic	5
1. Overview	6
1.1. Executive Summary	6
1.2. Goals of the Assessment	6
1.3. Non-goals and Limitations	6
1.4. Results	6
2. Introduction	8
2.1. About Multiliquid EVM	8
2.2. Methodology	8
2.3. Scope	10
2.4. Project Overview	10
2.5. Project Timeline	11
3. Detailed Findings	12
3.1. Daily-rate correction is not retroactive; an early accruer locks in an erroneous rate and holds redeemable value above the corrected backing	12
3.2. Unbounded withheld-credit merge can grief Treasury accrual and autosweep paths	14
3.3. ULTRAAdapter and JTRSYAdapter accept a zero price with no validation	16
3.4. Stablecoin redemption applies the last posted daily rate over an unbounded fall-back window	17
3.5. Upgrade leaves all swaps disabled until setMaxSwapLegs is configured	19
3.6. Unauthenticated first-writer proxy initializer enables delegate takeover	20

3.7.	Adapter ownership transfer leaves the LayerZero endpoint delegate stale	22
4.	Threat Model	24
4.1.	Module: BalanceSheetStablecoinDelegate.sol	24
4.2.	Module: BraleStablecoinDelegate.sol	32
4.3.	Module: ChainlinkPriceAdapter.sol	40
4.4.	Module: ChroniclePriceAdapter.sol	45
4.5.	Module: DollarPeggedAdapter.sol	49
4.6.	Module: GenericMintBurnStablecoinDelegate.sol	49
4.7.	Module: IssuerAdminUpgradeable.sol	52
4.8.	Module: JTRSYAdapter.sol	54
4.9.	Module: JTRSYWhitelistAdapter.sol	56
4.10.	Module: JTRSYWhitelistAdapterV2.sol	57
4.11.	Module: MintBurnStablecoinDelegate.sol	59
4.12.	Module: MultiliquidSwap.sol	68
4.13.	Module: RWADelegate.sol	154
4.14.	Module: StablecoinDelegateBase.sol	162
4.15.	Module: ThirdPartySetterPriceAdapter.sol	187
4.16.	Module: Treasury.sol	191
4.17.	Module: TreasuryDelegate.sol	211
4.18.	Module: TreasuryOFTAdapter.sol	273
4.19.	Module: ULTRAAdapter.sol	285
4.20.	Module: ULTRADelegate.sol	286
4.21.	Module: ULTRAWhitelistAdapter.sol	294

4.22.	Module: USTBAdapter.sol	296
4.23.	Module: USTBWhitelistAdapter.sol	298
4.24.	Module: UniformLabsDelegate.sol	300
4.25.	Module: VBILLWhitelistAdapter.sol	302
4.26.	Module: WTGXXWhitelistAdapter.sol	303
4.27.	Module: WithheldCreditsDeque.sol	305

5.	Assessment Results	313
5.1.	Disclaimer	313

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and follow @zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Uniform Labs from June 30th to July 20th, 2026. During this engagement, Zellic reviewed Multiliquid EVM's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does the swap function use the correct price?
 - Does the swap function use the correct decimal conversion?
 - Does the swap function use the correct exchange-rate calculation?
 - Does the yield-calculation function calculate yield correctly?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

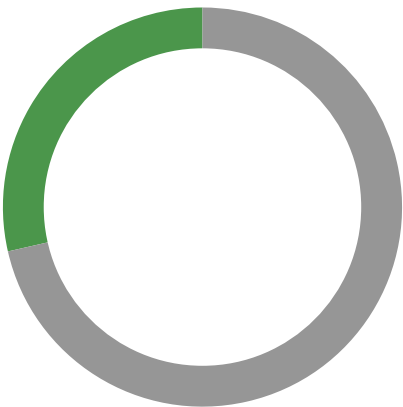
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Multiliquid EVM contracts, we discovered seven findings. No critical issues were found. Two findings were of low impact and the remaining findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	2
Informational	5



2. Introduction

2.1. About Multiliquid EVM

Uniform Labs contributed the following description of Multiliquid EVM:

Multiliquid operates as neutral market infrastructure, facilitating fixed-price swaps based on Net Asset Value between tokenized assets and stablecoins. The protocol sources liquidity directly from integrated stablecoin issuers and balance sheet providers, eliminating traditional DeFi inefficiencies.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Multiliquid EVM Contracts

Type	Solidity
Platform	EVM-compatible
Target	Multiliquid-EVM
Repository	https://github.com/uniformlabs/Multiliquid-EVM ↗
Version	1fcc714e735ded15881f7fc133e7c4cdcb2320b6
Programs	src/**/*.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 4.3 person-weeks. The assessment was conducted by three consultants over the course of three calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Evan Hwang
Engineer
evan@zellic.io ↗

Jaeeu Kim
Engineer
jaeeu@zellic.io ↗

Oliver Murray
Engineer
oliver@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 30, 2026 Kick-off call

June 30, 2026 Start of primary review period

July 20, 2026 End of primary review period

3. Detailed Findings

3.1. Daily-rate correction is not retroactive; an early accruer locks in an erroneous rate and holds redeemable value above the corrected backing

Target	TreasuryDelegate		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

TreasuryDelegate compounds each finished day exactly once and advances lastAccrualDay[user]. A day that has already been accrued is never reprocessed. The correctDailyRate overwrites a past day's stored APY but does not rewind any user's accrual cursor or yieldMultiplier.

```
// src/prod/ProtocolStablecoinDelegates/TreasuryDelegate.sol#L457-L462
uint256 oldRate = dailyRates[dayNumber];
dailyRates[dayNumber] = netRate;
dailyCompoundingRates[dayNumber] = _convertApyToDailyRate(netRate);
ratePosted[dayNumber] = true;
// lastRatePostingDay is NOT updated - this is a correction, not an advancement
```

Because accrueInterest, batchAccrueInterest, and accrueInterestPaged are permissionless, a user who observes an anomalously high posted rate can accrue before the operator corrects it and permanently lock that multiplier. The only on-chain repair is manual setUserYieldMultiplier, whose NatSpec notes it can increase redeemable value beyond current backing.

Impact

After a rate correction, users who accrued early can redeem more Treasury than the corrected backing supports. The surplus is not clawed back on chain and requires per-address admin repair.

Recommendations

Reconcile user multipliers when correctDailyRate runs, or block accrual for a day until corrections are finalized.

Remediation

This issue has been acknowledged by Uniform Labs.

3.2. Unbounded withheld-credit merge can grief Treasury accrual and autosweep paths

Target	TreasuryDelegate, MultiliquidSwap		
Category	Protocol Risks	Severity	Low
Likelihood	Low	Impact	Low

Description

TreasuryDelegate appends one withheldCredits deque record per fee or deposit with no coalescing. After maturity, accrueInterest* calls _mergeWithheldCredits, which drains every eligible record in a single unbounded while loop; accrueInterestPaged(maxDays) caps only the daily compounding loop, not the merge.

Protocol fees credit the shared multiliquidVault; LP fees credit the shared stablecoinCustodyAddress. And accrueInterest is permissionless. If a shared queue is large enough, merge can exceed the gas limit, revert atomically, and leave the deque unchanged.

```
// src/prod/ProtocolStablecoinDelegates/TreasuryDelegate.sol#L946-L954
bool mainCreditsCaughtUp = (userCredits == 0) || (finalLastAccrualDay
    >= targetDay);
if (totalWithheldCredits[user] > 0 && mainCreditsCaughtUp) {
    _mergeWithheldCredits(user);
}
```

When autoLiquidityStablecoinID is set, prefund and autosweep paths call accrueInterest on the custody address inside another user's swap, so a large custody queue can revert unrelated legs in the same transaction.

Impact

An attacker who spends swap volume and fees to bloat shared vault or custody queues can stall yield bookkeeping merge and, with autoliquidity enabled, revert unrelated users' prefund or autosweep legs. Normal USDC <-> Treasury mint/redeem only accrues params.user; there is no fund-theft path.

Recommendations

Coalesce compatible withheld credits on insert, and add a paged merge (mergeWithheldCreditsPaged(user, maxRecords)) separate from daily accrual paging.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [a7713199](#).

3.3. ULTRAAdapter and JTRSYAdapter accept a zero price with no validation

Target	ULTRAAdapter, JTRSYAdapter, MultiliquidSwap		
Category	Code Maturity	Severity	Informational
Likelihood	Low	Impact	Informational

Description

USTBAdapter rejects a nonpositive price, but ULTRAAdapter and JTRSYAdapter return their upstream NAV values unchecked. MultiliquidSwap divides by the RWA USD value in route conversions with no zero guard.

```
// src/prod/PriceAdapters/JTRSYAdapter.sol#L22-L24
function getPrice() external view returns (uint256) {
    return asyncVault.pricePerShare() * 1e12;
}
```

If the upstream value is zero, ULTRAAdapter and JTRSYAdapter return 0 from `getPrice()` without reverting, while USTBAdapter reverts `InvalidPrice` under the same condition. Swaps into the RWA then revert on division by zero, or an exact-in leg with no min-out can quote zero output. A privileged operator pushing zero into the upstream update function is the realistic trigger; unprivileged users cannot force a zero price.

Impact

Conditional on an upstream zero, the affected RWA pair becomes unavailable or mispriced. The impact is bounded to that pair and is not attacker-profitable under normal slippage settings.

Recommendations

Reject `price <= 0` in ULTRAAdapter and JTRSYAdapter, matching USTBAdapter.

As defense in depth, have MultiliquidSwap reject a zero RWA USD value after `getPrice()`.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [ac5298c3](#).

3.4. Stablecoin redemption applies the last posted daily rate over an unbounded fallback window

Target	TreasuryDelegate		
Category	Business Logic	Severity	Informational
Likelihood	Low	Impact	Informational

Description

Treasury→stablecoin redemptions use continuous partial-day yield. When the current day's rate is not posted, the delegate falls back to the last posted daily rate and projects it linearly over elapsed seconds, capped only by `maxFallbackRateSeconds`.

```
// src/prod/ProtocolStablecoinDelegates/TreasuryDelegate.sol#L850-L853
function _capFallbackRateSeconds(uint256 secondsElapsed)
    internal view returns (uint256) {
    uint256 maxSeconds = maxFallbackRateSeconds;
    return maxSeconds != 0 && secondsElapsed > maxSeconds ? maxSeconds :
        secondsElapsed;
}
```

Since `initialize` does not set the cap, the default is unbounded (0 disables the cap). The linear projection during posting gaps is documented intended behavior; the residual issue is that deployment scripts never configure the spec's expected bound (e.g., seven days).

Impact

During a prolonged rate-posting outage, stablecoin redeemers can extract yield projected from the stale last-posted rate beyond what main-credit accrual or RWA exit paths would pay. Value already paid during the gap cannot be recovered if later days are backfilled at a lower rate.

Recommendations

Initialize `maxFallbackRateSeconds` to a sane default in `initialize`, and reject 0 in `setMaxFallbackRateSeconds`.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [5835e308](#).

3.5. Upgrade leaves all swaps disabled until setMaxSwapLegs is configured

Target	MultiliquidSwap		
Category	Code Maturity	Severity	Informational
Likelihood	Low	Impact	Informational

Description

MultiliquidSwap stores `_maxSwapLegs` in a new proxy slot that defaults to zero on upgrade, and swap rejects any nonempty batch when `_maxSwapLegs` is zero.

```
// src/prod/MultiliquidSwap.sol#L452-L453
uint256 inputCount = inputs.length;
if (inputCount == 0 || inputCount > _maxSwapLegs) revert InvalidLegCount();
```

The V3 upgrade scripts call `upgradeToAndCall(impl, "")` with empty calldata and do not require `setMaxSwapLegs` before `unpause`. This default is intentional (swaps disabled until configured) but is omitted from the committed runbook.

Impact

If operators `unpause` after upgrade without calling `setMaxSwapLegs(N)`, every public swap reverts until an `OPERATOR_ROLE` transaction sets a positive limit. No user funds are at risk while paused.

Recommendations

Add `setMaxSwapLegs(N)` to the postupgrade checklist, and optionally bundle it in upgrade calldata.

Remediation

This issue has been acknowledged by Uniform Labs.

3.6. Unauthenticated first-writer proxy initializer enables delegate takeover

Target	BraleStablecoinDelegate		
Category	Code Maturity	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The `initialize` function is a public initializer that accepts caller-selected admin, issuer-admin, swap-engine, stablecoin ID, and token addresses. It performs no authentication against a deployer, factory, or expected bootstrap authority before the base initializer grants all roles.

```
// src/prod/ProtocolStablecoinDelegates/BraleStablecoinDelegate.sol
function initialize(
    address multiliquidAdmin,
    address initialIssuerAdmin,
    address _multiliquidSwap,
    bytes32 _stablecoinID,
    address _stablecoinAddress
) public initializer {
    // grants DEFAULT_ADMIN_ROLE, PAUSE_ROLE, issuer admin, and UUPS upgrade
    // authority to caller-supplied arguments, with no authentication
    __stablecoinDelegateBase_init(/* ... */);
}
```

As a result, a front-runner can initialize the proxy with attacker-controlled identities, gaining UUPS upgrade authority and issuer administration while binding the swap-execution role to an attacker-controlled caller.

Impact

An attacker who wins the initialization race can take over an uninitialized Brale delegate proxy, permanently excluding the intended administrators and controlling its upgrades and issuer configuration.

Recommendations

Following proxy deployment best practices, the implementation upgrade and the `initialize` call should be executed within the same transaction.

Remediation

This issue has been acknowledged by Uniform Labs.

3.7. Adapter ownership transfer leaves the LayerZero endpoint delegate stale

Target	TreasuryOFTAdapter		
Category	Code Maturity	Severity	Informational
Likelihood	Low	Impact	Informational

Description

The constructor assigns `_delegate` both adapter ownership and the independent LayerZero endpoint-delegate privilege, but inherited ownership rotation updates only the former.

```
// src/prod/TreasuryOFTAdapter.sol
constructor(address _token, address _lzEndpoint, address _delegate)
    OFTAdapter(_token, _lzEndpoint, _delegate) // sets
    EndpointV2.delegates[adapter] = _delegate
    Ownable(_delegate) // sets Ownable owner = _delegate
{ }
```

After `transferOwnership(newOwner)`, the previous owner remains authorized in `EndpointV2.delegates[address(adapter)]` until the new owner separately calls `setDelegate`. That stale principal can change the adapter's receive library and DVN configuration, including replacing the verification set with an attacker-controlled DVN. It can then verify a forged packet claiming the already configured remote peer, causing `_credit` to invoke `bridgeMint` for an attacker-selected recipient and amount.

Impact

If ownership is rotated without updating the endpoint delegate, the previous owner retains LayerZero configuration authority that is not reflected in `owner()`. A malicious or compromised ex-delegate could reconfigure receive verification and, in the worst case, approve forged inbound packets that trigger `bridgeMint` and mint unbacked Treasury. This is a trusted-party and deployment-runbook risk, not a permissionless exploit. There is no mainnet impact until a bridge adapter is deployed and `BRIDGE_ROLE` is granted.

Recommendations

The `setDelegate` function should be called during ownership transfer and renunciation to ensure the LayerZero endpoint delegate is updated before the `Ownable` authority is changed.

Remediation

This issue has been acknowledged by Uniform Labs, and a fix was implemented in commit [3e64416c](#).

4. Threat Model

This section contains threat models for selected functions in the contracts. Each threat model identifies externally controlled inputs and the code paths an attacker can influence.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

4.1. Module: BalanceSheetStablecoinDelegate.sol

Function: constructor ()

The `constructor` function disables initializers on the implementation contract so the logic contract cannot be initialized directly, following the UUPS upgradable pattern.

Branches and code coverage

Intended branches

- Calls `_disableInitializers()` to permanently lock initialization on the implementation.

☒ Test coverage

Negative behavior

- Cannot initialize the implementation directly after deployment.

☒ Negative test

Function: deployStablecoin(DelegateParams params)

The `deployStablecoin` function issues stablecoin from issuer custody to the receiver after MultiliquidSwap routes an RWA-for-stablecoin swap, pulling backing RWA into balance sheet custody and routing protocol fees from stablecoin custody to the vault.

Inputs

- `params`
 - **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations; end users cannot call this delegate directly.
 - **Constraints:** Enforced by `onlyRole(MULTILIQUID_SWAP_CONTRACT)`, `whenNotPaused`, `nonReentrant`, `onlyWhitelistedRWA(params.assetInAddress)`, blacklist checks on user and receiver, and `custodyAddressesSet`; MultiliquidSwap must ensure user has approved the RWA transfer and stablecoin custody has approved outflows.

- **Impact:** Moves RWA into balanceSheetCustodyAddress, transfers protocol fee stablecoin from custody to vault, and transfers assetOutAmount stablecoin from custody to receiver.

Branches and code coverage

Intended branches

- Transfers assetInAmount of assetInAddress RWA from user to balanceSheetCustodyAddress.
☒ Test coverage
- Transfers protocolFeeAmt stablecoin from stablecoinCustodyAddress to vault when the fee is positive.
☒ Test coverage
- Transfers assetOutAmount stablecoin from stablecoinCustodyAddress to receiver.
☒ Test coverage
- Returns true on success.
☒ Test coverage

Negative behavior

- Reverts if the caller lacks MULTILIQUID_SWAP_CONTRACT.
☒ Negative test
- Reverts when paused or on reentrancy.
☒ Negative test
- Reverts if assetInAddress is not a whitelisted RWA.
☒ Negative test
- Reverts if user or receiver is blacklisted or custody addresses are unset.
☒ Negative test
- Reverts if RWA or stablecoin transfers fail (insufficient allowance, balance, or malicious token behavior).
☐ Negative test

Function call analysis

- SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user, this.balanceSheetCustodyAddress, params.assetInAmount)

- **What is controllable?** RWA token contract and user approval — amount and addresses from MultiliquidSwap params.
- **If the return value is controllable, how is it used and how can it go wrong?** Deposits backing collateral — failed transfer reverts the issuance leg.
- **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts atomically; nonReentrant blocks callback reentry.
- `SafeERC20.safeTransferFrom(IERC20(this.stablecoinAddress), this.stablecoinCustodyAddress, params.vault, params.protocolFeeAmt)`
 - **What is controllable?** Stablecoin custody balance and prior approval to this delegate; fee amount from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Moves prefunded fee stablecoin to the vault; insufficient custody balance reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back the RWA deposit in the same transaction.
- `SafeERC20.safeTransferFrom(IERC20(this.stablecoinAddress), this.stablecoinCustodyAddress, params.receiver, params.assetOutAmount)`
 - **What is controllable?** Custody inventory and approval — output amount from MultiliquidSwap pricing.
 - **If the return value is controllable, how is it used and how can it go wrong?** Pays stablecoin from issuer reserves; underfunded custody causes revert rather than overissuance.
 - **What happens if it reverts, reenters or does other unusual control flow?** Malicious ERC-20 behavior causes atomic failure — cannot reenter due to nonReentrant.

Function: `exchangeRWAs(DelegateParams params)`

The `exchangeRWAs` function swaps one RWA for another through issuer custody without stablecoin touching the user's wallet, collecting incoming RWA, paying protocol fees from stablecoin custody, and delivering the output RWA.

Inputs

- `params`
 - **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations; end users cannot call this delegate directly.
 - **Constraints:** Enforced by `onlyRole(MULTILQUID_SWAP_CONTRACT)`, `whenNotPaused`, `nonReentrant`, whitelist checks on both RWA legs, blacklist checks on user and receiver, and `custodyAddressesSet`; MultiliquidSwap must ensure the user has approved the input RWA transfer and custody has approved output RWA and stablecoin outflows.

- **Impact:** Moves input RWA to custody, transfers protocol-fee stablecoin from custody to vault (LP fee remains in custody), and delivers output RWA to receiver.

Branches and code coverage

Intended branches

- Transfers `assetInAmount` of `assetInAddress` from user to `balanceSheetCustodyAddress`.
☒ Test coverage
- Transfers `protocolFeeAmt` stablecoin from `stablecoinCustodyAddress` to vault when the protocol fee is positive.
☒ Test coverage
- Leaves LP/redemption fee stablecoin in custody (already netted in `MultiliquidSwap` pricing).
☒ Test coverage
- Transfers `assetOutAmount` of `assetOutAddress` from `balanceSheetCustodyAddress` to receiver.
☒ Test coverage
- Returns `true` on success.
☒ Test coverage

Negative behavior

- Reverts if the caller lacks `MULTILIQUEID_SWAP_CONTRACT`.
☒ Negative test
- Reverts when paused or on reentrancy.
☒ Negative test
- Reverts if either RWA is not whitelisted.
☒ Negative test
- Reverts if user or receiver is blacklisted or custody addresses are unset.
☒ Negative test
- Reverts if RWA or stablecoin transfers fail.
☐ Negative test

Function call analysis

- SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user, this.balanceSheetCustodyAddress, params.assetInAmount)
 - **What is controllable?** Input RWA token and user approval — amounts from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Deposits sold RWA into custody; a failed transfer reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts atomically; nonReentrant blocks callback reentry.
- SafeERC20.safeTransferFrom(IERC20(this.stablecoinAddress), this.stablecoinCustodyAddress, params.vault, params.protocolFeeAmt)
 - **What is controllable?** Stablecoin custody balance and approval — protocol fee from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Pays protocol fee from prefunded custody; insufficient balance reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back the input RWA transfer.
- SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.balanceSheetCustodyAddress, params.receiver, params.assetOutAmount)
 - **What is controllable?** Output RWA token and custody approval to this delegate.
 - **If the return value is controllable, how is it used and how can it go wrong?** Delivers purchased RWA; insufficient custody inventory reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Malicious RWA revert/callback cannot reenter.

Function: exchangeStablecoins(DelegateParams params)

The exchangeStablecoins function swaps one stablecoin for another through issuer custody by pulling the input stablecoin from the user into custody, paying protocol fees from custody, and transferring the output stablecoin from custody to the receiver.

Inputs

- params
 - **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations; end users cannot call this delegate directly.
 - **Constraints:** Enforced by onlyRole(MULTILQUID_SWAP_CONTRACT), whenNotPaused, nonReentrant, blacklist checks, custodyAddressesSet, _checkIssuerStablecoinInExchange, and _onlyWhitelistedStablecoin

on the nonissuer leg; MultiliquidSwap must ensure user has approved the input stablecoin transfer and custody has approved fee and output outflows.

- **Impact:** Moves stablecoins between user, custody, vault, and receiver; acceptance and redemption fees remain in custody per MultiliquidSwap pricing.

Branches and code coverage

Intended branches

- Transfers assetInAmount of assetInAddress from user to stablecoinCustodyAddress.
☒ Test coverage
- Transfers protocolFeeAmt of issuer stablecoin from stablecoinCustodyAddress to vault when the fee is positive.
☒ Test coverage
- Leaves acceptance and redemption fees in custody (already netted in MultiliquidSwap pricing).
☒ Test coverage
- Transfers assetOutAmount of assetOutAddress from stablecoinCustodyAddress to receiver.
☒ Test coverage
- Returns true on success.
☒ Test coverage

Negative behavior

- Reverts if the caller lacks MULTILIQUID_SWAP_CONTRACT.
☒ Negative test
- Reverts when paused or on reentrancy.
☒ Negative test
- Reverts if neither leg is the issuer stablecoin (InvalidStablecoinExchange).
☒ Negative test
- Reverts if the external stablecoin leg is not whitelisted.
☒ Negative test
- Reverts if user or receiver is blacklisted or custody addresses are unset.
☒ Negative test
- Reverts if stablecoin transfers fail (insufficient balance, allowance, or malicious token behavior).

❑ Negative test

Function call analysis

- SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user, this.stablecoinCustodyAddress, params.assetInAmount)
 - **What is controllable?** Input stablecoin token and user approval — amount from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Deposits incoming stablecoin into issuer custody — failed transfer reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts atomically; nonReentrant blocks callback reentry.
- SafeERC20.safeTransferFrom(IERC20(this.stablecoinAddress), this.stablecoinCustodyAddress, params.vault, params.protocolFeeAmt)
 - **What is controllable?** Issuer stablecoin custody balance and approval — protocol fee from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Pays protocol fee from custody reserves — insufficient balance reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back the input deposit in the same transaction.
- SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.stablecoinCustodyAddress, params.receiver, params.assetOutAmount)
 - **What is controllable?** Output stablecoin in custody and approval to this delegate; amount from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Delivers whitelisted stablecoin to the user; underfunded custody reverts rather than overpaying.
 - **What happens if it reverts, reenters or does other unusual control flow?** Malicious ERC-20 revert/callback cannot bypass nonReentrant.

Function: receiveStablecoin(DelegateParams params)

The receiveStablecoin function redeems stablecoin for RWA by collecting protocol fees to the vault, moving the remaining stablecoin into issuer custody and releasing RWA from balance sheet custody to the receiver.

Inputs

- params

- **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations — end users cannot call this delegate directly.
- **Constraints:** Enforced by `onlyRole(MULTILIQUEID_SWAP_CONTRACT)`, `whenNotPaused`, `nonReentrant`, `onlyWhitelistedRWA(params.assetOutAddress)`, blacklist checks on user and receiver, and `custodyAddressesSet`; MultiliquidSwap must ensure user has approved stablecoin transfers and custody has approved RWA outflow.
- **Impact:** Routes protocol fees to vault, deposits redeemed stablecoin (including LP fee) into `stablecoinCustodyAddress`, and delivers `assetOutAmount` of RWA to receiver.

Branches and code coverage

Intended branches

- Transfers `protocolFeeAmt` stablecoin from user to vault when the fee is positive.
☒ Test coverage
- Transfers `assetInAmount - protocolFeeAmt` stablecoin from user to `stablecoinCustodyAddress` (includes LP/redemption fee in the total).
☒ Test coverage
- Transfers `assetOutAmount` of `assetOutAddress` from `balanceSheetCustodyAddress` to receiver.
☒ Test coverage
- Returns `true` on success.
☒ Test coverage

Negative behavior

- Reverts if the caller lacks `MULTILIQUEID_SWAP_CONTRACT`.
☒ Negative test
- Reverts when paused or on reentrancy.
☒ Negative test
- Reverts if `assetOutAddress` is not a whitelisted RWA.
☒ Negative test
- Reverts if user or receiver is blacklisted or custody addresses are unset.
☒ Negative test
- Reverts if stablecoin or RWA transfers fail.
☐ Negative test

Function call analysis

- `SafeERC20.safeTransferFrom(IERC20(this.stablecoinAddress), params.user, params.vault, params.protocolFeeAmt)`
 - **What is controllable?** Stablecoin token and user approval — protocol fee from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Moves fee stablecoin to vault — failed transfer reverts redemption.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap; nonReentrant blocks callback reentry.
- `SafeERC20.safeTransferFrom(IERC20(this.stablecoinAddress), params.user, this.stablecoinCustodyAddress, params.assetInAmount - params.protocolFeeAmt)`
 - **What is controllable?** User balance and allowance for the net redemption amount including LP fee.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns redeemed stablecoin to issuer custody; insufficient funds or fee-on-transfer behavior reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Malicious ERC-20 callbacks cannot reenter; whole leg fails atomically.
- `SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.balanceSheetCustodyAddress, params.receiver, params.assetOutAmount)`
 - **What is controllable?** RWA token and custody approval — output from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Delivers backing RWA — insufficient custody inventory reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back stablecoin legs in the same transaction.

4.2. Module: BraleStablecoinDelegate.sol

Function: constructor ()

The constructor function disables initializers on the implementation contract so the logic contract cannot be initialized directly, following the UUPS upgradable pattern.

Branches and code coverage

Intended branches

- Calls `_disableInitializers()` to permanently lock initialization on the implementation.

☒ Test coverage

Negative behavior

- Cannot initialize the implementation directly after deployment.

☒ Negative test

Function: exchangeStablecoins(DelegateParams params)

The exchangeStablecoins function swaps one stablecoin for another through issuer custody, burning or pulling the input stablecoin, minting fees and output when the issuer token is involved, and transferring external stablecoins via custody otherwise.

Inputs

- params
 - **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations — end users cannot call this delegate directly.
 - **Constraints:** Enforced by onlyRole(MULTILQUID_SWAP_CONTRACT), whenNotPaused, nonReentrant, blacklist checks, custodyAddressesSet, _checkIssuerStablecoinInExchange, and _onlyWhitelistedStablecoin on the nonissuer leg; MultiliquidSwap must ensure user has approved relevant token transfers.
 - **Impact:** Moves stablecoins between user, custody, vault, and receiver — mints or burns issuer stablecoin and protocol/LP fee balances.

Branches and code coverage

Intended branches

- When assetInAddress is the issuer stablecoin, transfers it from user to this contract and burns assetInAmount.

☒ Test coverage

- When assetInAddress is external, transfers assetInAmount from user to stablecoinCustodyAddress.

☒ Test coverage

- Mints protocolFeeAmt issuer stablecoin to vault and lpFeeAmt to stablecoinCustodyAddress when fees are positive.

☒ Test coverage

- When assetOutAddress is the issuer stablecoin, mints assetOutAmount to receiver.

☒ Test coverage

- When `assetOutAddress` is external, transfers `assetOutAmount` from `stablecoinCustodyAddress` to receiver.

☒ Test coverage

- Returns `true` on success.

☒ Test coverage

Negative behavior

- Reverts if the caller lacks `MULTILIQUID_SWAP_CONTRACT`.

☒ Negative test

- Reverts when paused or on reentrancy.

☒ Negative test

- Reverts if neither leg is the issuer stablecoin (`InvalidStablecoinExchange`).

☒ Negative test

- Reverts if the external stablecoin leg is not whitelisted.

☒ Negative test

- Reverts if user or receiver is blacklisted or custody addresses are unset.

☒ Negative test

- Reverts if token transfers, mints, or burns fail.

☐ Negative test

Function call analysis

- `SafeERC20.safeTransferFrom(stablecoin, params.user, address(this), params.assetInAmount)`

- **What is controllable?** User approval and Brale stablecoin behavior for the issuer-token input leg.
- **If the return value is controllable, how is it used and how can it go wrong?** Pulls issuer stablecoin before burn — failed transfer aborts the exchange.
- **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back; `nonReentrant` blocks callback reentry.

- `stablecoin.burn(params.assetInAmount)`

- **What is controllable?** Delegate must hold burn authority on the Brale token for balances on this contract.
- **If the return value is controllable, how is it used and how can it go wrong?** Destroys incoming issuer supply — unauthorized burn reverts.
- **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts after any prior transfers in the transaction.

- `SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user, this.stablecoinCustodyAddress, params.assetInAmount)`
 - **What is controllable?** External stablecoin contract and user allowance for the nonissuer input leg.
 - **If the return value is controllable, how is it used and how can it go wrong?** Deposits backing stablecoin into issuer custody — undertransfer reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Malicious ERC-20 revert/callback cannot reenter; whole swap fails atomically.
- `stablecoin.mint(params.vault, params.protocolFeeAmt)`
 - **What is controllable?** Delegate minter role on issuer stablecoin — fee amount from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Inflates vault balance by protocol fee — failed mint reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents partial fee collection.
- `stablecoin.mint(this.stablecoinCustodyAddress, params.lpFeeAmt)`
 - **What is controllable?** Same minter authority; LP fee amount from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Credits issuer custody with acceptance/redemption fees — excessive mint without matching inflow is a deployment/configuration risk outside this hook.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the exchange.
- `stablecoin.mint(params.receiver, params.assetOutAmount)`
 - **What is controllable?** Minter role and output amount chosen by MultiliquidSwap pricing.
 - **If the return value is controllable, how is it used and how can it go wrong?** Delivers issuer stablecoin to the user; mint failure reverts the swap.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back prior legs in the same transaction.
- `SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.stablecoinCustodyAddress, params.receiver, params.assetOutAmount)`
 - **What is controllable?** External stablecoin in custody and prior approval to this delegate.
 - **If the return value is controllable, how is it used and how can it go wrong?** Pays out whitelisted external stablecoin from custody; insufficient custody balance reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts — token hooks cannot bypass `nonReentrant`.

Function: initialize(address multiliquidAdmin, address initialIssuerAdmin, address _multiliquidSwap, byte[32] _stablecoinID, address _stablecoinAddress)

The initialize function performs one-time proxy setup by delegating to __stablecoinDelegateBase_init, which validates addresses, initializes upgradable modules, and grants protocol roles.

Inputs

- multiliquidAdmin
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero and distinct from initialIssuerAdmin and _multiliquidSwap.
 - **Impact:** Receives DEFAULT_ADMIN_ROLE and PAUSE_ROLE over this delegate.
- initialIssuerAdmin
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero and distinct from multiliquidAdmin and _multiliquidSwap.
 - **Impact:** Becomes the issuer admin who can manage RWA/stablecoin whitelists, fees, and custody addresses.
- _multiliquidSwap
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero and distinct from both admin addresses.
 - **Impact:** Stored as multiliquidSwap and granted MULTILIQID_SWAP_CONTRACT, making it the sole caller of swap hooks.
- _stablecoinID
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero (bytes32(0) reverts).
 - **Impact:** Stored as the protocol identifier for this stablecoin integration.
- _stablecoinAddress
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero.
 - **Impact:** Stored as the Brale ERC-20 token whose mint / burn this delegate must be authorized to call.

Branches and code coverage

Intended branches

- Initializes access control, pausable, reentrancy guard, UUPS, and issuer admin modules via `__stablecoinDelegateBase_init`.

☑ Test coverage

- Stores `multiliquidSwap`, `stablecoinID`, and `stablecoinAddress`.

☑ Test coverage

- Grants `DEFAULT_ADMIN_ROLE` and `PAUSE_ROLE` to `multiliquidAdmin` and `MULTILIQUEID_SWAP_CONTRACT` to `_multiliquidSwap`.

☑ Test coverage

Negative behavior

- Reverts if called when already initialized (initializer modifier).

☑ Negative test

- Reverts on zero or colliding admin/swap addresses (`InvalidInitializationParams`, `InvalidIssuerAdmin`, `InvalidMultiliquidSwap`).

☑ Negative test

Function call analysis

- No external calls — internal `__stablecoinDelegateBase_init` only (OpenZeppelin initializer chain).

Function: `receiveStablecoin(DelegateParams params)`

The `receiveStablecoin` function redeems stablecoin for RWA by collecting fees, pulling stablecoin from the user, burning the net amount (via transfer-then-burn because Brale lacks `burnFrom`), and releasing RWA from balance sheet custody to the receiver.

Inputs

- `params`
 - **Control:** Fully specified by `MultiliquidSwap` after pricing, slippage, and fee calculations — end users cannot call this delegate directly.
 - **Constraints:** Enforced by `onlyRole(MULTILIQUEID_SWAP_CONTRACT)`, `whenNotPaused`, `nonReentrant`, `onlyWhitelistedRWA(params.assetOutAddress)`, blacklist checks on user and receiver, and `custodyAddressesSet`; `MultiliquidSwap` must ensure user has approved stablecoin transfers and custody has approved RWA outflow.

- **Impact:** Moves protocol and LP fees, burns redeemed stablecoin, and delivers `assetOutAmount` of RWA to `params.receiver`.

Branches and code coverage

Intended branches

- Transfers `protocolFeeAmt` stablecoin from user to vault when the fee is positive.
 - ☒ Test coverage
- Transfers `lpFeeAmt` stablecoin from user to `stablecoinCustodyAddress` when the fee is positive.
 - ☒ Test coverage
- Transfers `assetInAmount - protocolFeeAmt - lpFeeAmt` stablecoin to this contract and burns it when the net amount is positive.
 - ☒ Test coverage
- Transfers `assetOutAmount` of `assetOutAddress` from `balanceSheetCustodyAddress` to receiver.
 - ☒ Test coverage
- Returns `true` on success.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller lacks `MULTILIQUID_SWAP_CONTRACT`.
 - ☒ Negative test
- Reverts when paused or on reentrancy.
 - ☒ Negative test
- Reverts if `assetOutAddress` is not a whitelisted RWA.
 - ☒ Negative test
- Reverts if user or receiver is blacklisted.
 - ☒ Negative test
- Reverts if custody addresses are unset.
 - ☒ Negative test
- Reverts if stablecoin or RWA transfers/burn fail (insufficient balance, allowance, or malicious token behavior).
 - ☐ Negative test

Function call analysis

- `SafeERC20.safeTransferFrom(stablecoin, params.user, params.vault, params.protocolFeeAmt)`
 - **What is controllable?** The stablecoin token contract and whether `params.user` has approved the delegate; fee amount and recipients come from MultiliquidSwap params.
 - **If the return value is controllable, how is it used and how can it go wrong?** SafeERC20 enforces a successful transfer — a false return or balance mismatch reverts the entire redemption.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the swap; a fee-on-transfer or callback token could break accounting assumed by MultiliquidSwap but `nonReentrant` blocks reentry into this hook.
- `SafeERC20.safeTransferFrom(stablecoin, params.user, this.stablecoinCustodyAddress, params.lpFeeAmt)`
 - **What is controllable?** Same as the protocol-fee transfer — user approval and stablecoin implementation govern pull success.
 - **If the return value is controllable, how is it used and how can it go wrong?** Credits LP/redemption fees to issuer custody — undertransfer or failed return reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back all prior legs; malicious ERC-20 callbacks cannot reenter due to `nonReentrant`.
- `SafeERC20.safeTransferFrom(stablecoin, params.user, address(this), amountToBurn)`
 - **What is controllable?** User balance and allowance for the net burn amount derived from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Pulls redeemable stablecoin to this contract before burn — insufficient funds or deflationary behavior causes revert.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents burn — token hooks cannot reenter this function.
- `stablecoin.burn(amountToBurn)`
 - **What is controllable?** Brale token enforces that only authorized minters/burners (this delegate) can destroy balances held here.
 - **If the return value is controllable, how is it used and how can it go wrong?** Permanently removes redeemed supply — if delegate lacks burn permission, the call reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts redemption after any prior transfers in the same transaction.
- `SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.balanceSheetCustodyAddress, params.receiver, params.assetOutAmount)`

- **What is controllable?** RWA token contract and prior issuer approval from custody to this delegate — output asset and amount come from params.
- **If the return value is controllable, how is it used and how can it go wrong?** Delivers backing RWA to the user — failed transfer or underdelivery reverts the swap.
- **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back stablecoin legs; malicious RWA can revert or callback but cannot bypass nonReentrant.

4.3. Module: ChainlinkPriceAdapter.sol

Function: constructor(address _priceFeed, uint256 _maxAge)

The constructor function initializes the Chainlink feed, staleness bound, WAD scaling multiplier, and deployer admin/operator roles.

Inputs

- `_priceFeed`
 - **Control:** Set by the deployer.
 - **Constraints:** The address must not be zero. The selected feed must implement `decimals` and return no more than 18 decimals.
 - **Impact:** The immutable oracle and WAD scaling source.
- `_maxAge`
 - **Control:** Set by the deployer.
 - **Constraints:** Must not be zero.
 - **Impact:** The initial maximum permitted oracle age.

Branches and code coverage

Intended branches

- Initializes scaling, `MAX_AGE`, and deployer roles with a valid feed and nonzero age.

☒ Test coverage

Negative behavior

- Reverts if the feed address is zero.

☒ Negative test

- Reverts if `_maxAge` is zero.

☒ Negative test

- Reverts if the decimals call returns a value above 18.
 - ☒ Negative test
- Reverts if the decimals call reverts or returns malformed data.
 - ☐ Negative test

Function call analysis

- `PRICE_FEED.decimals()`
 - **What is controllable?** The feed selected at deployment controls the `uint8` result.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned decimal count sets the scaling exponent to 18 minus that value. Values above 18 are rejected.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or malformed return propagates. Reentrancy cannot reach usable runtime code while construction is in progress.

Function: `getPrice()`

The `getPrice` function reads Chainlink round data, validates price and round freshness, and scales the answer to WAD.

Branches and code coverage

Intended branches

- Returns a positive, complete, and fresh answer scaled by `DECIMAL_MULTIPLIER`.
 - ☒ Test coverage
- Disables staleness rejection when `MAX_AGE` is the maximum `uint256` value.
 - ☒ Test coverage

Negative behavior

- Reverts if the answer is zero or negative.
 - ☒ Negative test
- Reverts if `updatedAt` is zero, `answeredInRound` is less than `roundId`, or the timestamp is in the future.
 - ☒ Negative test
- Reverts if the answer is older than `MAX_AGE`.

- ☒ Negative test
- Reverts if scaling overflows.
- ☒ Negative test

Function call analysis

- `PRICE_FEED.latestRoundData()`
 - **What is controllable?** The configured feed controls all returned round fields.
 - **If the return value is controllable, how is it used and how can it go wrong?** Scales answer into the result. Uses `roundId`, `updatedAt`, and `answeredInRound` for validation. Ignores `startedAt`.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or malformed return propagates.

Function: `grantRole(bytes32 role, address account)`

The `grantRole` function grants `role` to `account` if the caller has the admin role for `role`.

Inputs

- `role`
 - **Control:** Set by role admin.
 - **Constraints:** The caller must hold the admin role associated with `role`.
 - **Impact:** The role to grant.
- `account`
 - **Control:** Set by role admin.
 - **Constraints:** Arbitrary.
 - **Impact:** The account to grant the role to.

Branches and code coverage

Intended branches

- Grants and enumerates a new member.

☐ Test coverage

Negative behavior

- Reverts if the caller does not have the admin role for `role`.

☐ Negative test

Function: `renounceRole(bytes32 role, address callerConfirmation)`

The `renounceRole` function revokes `role` from `msg.sender` if `callerConfirmation` equals `msg.sender`.

Inputs

- `role`
 - **Control:** Arbitrary.
 - **Constraints:** Arbitrary.
 - **Impact:** The role to renounce.
- `callerConfirmation`
 - **Control:** Arbitrary.
 - **Constraints:** Must equal `msg.sender`.
 - **Impact:** The address that must match `msg.sender`.

Branches and code coverage

Intended branches

- Removes `role` and its enumeration entry after matching confirmation.

☐ Test coverage

Negative behavior

- Reverts if `callerConfirmation` does not equal `msg.sender`.

☐ Negative test

Function: `revokeRole(bytes32 role, address account)`

The `revokeRole` function revokes `role` from `account` if the caller has the admin role for `role`.

Inputs

- `role`
 - **Control:** Set by role admin.
 - **Constraints:** The caller must hold the admin role associated with `role`.

- **Impact:** The role to revoke.
- account
 - **Control:** Set by role admin.
 - **Constraints:** Arbitrary.
 - **Impact:** The account to revoke the role from.

Branches and code coverage

Intended branches

- Revokes `role` and removes its enumeration entry.
- ☐ Test coverage

Negative behavior

- Reverts if the caller does not have the admin role for `role`.
- ☐ Negative test

Function: `setMaxAge(uint256 _maxAge)`

The `setMaxAge` function updates the Chainlink staleness threshold for future price reads.

Inputs

- `_maxAge`
 - **Control:** Set by operator.
 - **Constraints:** Must not be zero and caller must hold `OPERATOR_ROLE`.
 - **Impact:** The maximum permitted feed age.

Branches and code coverage

Intended branches

- Updates `MAX_AGE` for a nonzero value.
- ☒ Test coverage

Negative behavior

- Reverts if the caller does not have `OPERATOR_ROLE`.
- ☒ Negative test
- Reverts if `_maxAge` is zero.

☒ Negative test

4.4. Module: ChroniclePriceAdapter.sol

Function: constructor(address _oracle, uint256 _maxAge)

The constructor function initializes the Chronicle oracle, staleness bound, and deployer admin/operator roles.

Inputs

- `_oracle`
 - **Control:** Set by the deployer.
 - **Constraints:** Must not be zero.
 - **Impact:** The immutable Chronicle oracle.
- `_maxAge`
 - **Control:** Set by the deployer.
 - **Constraints:** Must not be zero.
 - **Impact:** The initial maximum permitted oracle age.

Branches and code coverage

Intended branches

- Initializes the oracle, MAX_AGE, and deployer roles with valid nonzero inputs.

☒ Test coverage

Negative behavior

- Reverts if the oracle address is zero.

☒ Negative test

- Reverts if `_maxAge` is zero.

☒ Negative test

Function: getPrice()

The `getPrice` function reads Chronicle's validity, WAD price, and timestamp and requires a valid, fresh price that is not zero.

Branches and code coverage

Intended branches

- Returns a valid, nonzero, and fresh value unchanged.
 - ☒ Test coverage
- Disables staleness rejection when MAX_AGE is the maximum uint256 value.
 - ☒ Test coverage

Negative behavior

- Reverts if the oracle flag is false or the value is zero.
 - ☒ Negative test
- Reverts if the timestamp is in the future.
 - ☒ Negative test
- Reverts if the timestamp is older than MAX_AGE.
 - ☒ Negative test
- Reverts if the tryReadWithAge call reverts or returns malformed data.
 - ☐ Negative test

Function call analysis

- ORACLE.tryReadWithAge()
 - **What is controllable?** The configured oracle controls validity, value, and timestamp.
 - **If the return value is controllable, how is it used and how can it go wrong?** The value is returned directly after validity and freshness checks. A false validity flag or a zero value is rejected.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or malformed return propagates.

Function: grantRole(bytes32 role, address account)

The grantRole function grants role to account if the caller has the admin role for role.

Inputs

- role
 - **Control:** Set by the role admin.

- **Constraints:** The caller must hold the admin role associated with `role`.
 - **Impact:** The role to grant.
- `account`
 - **Control:** Set by the role admin.
 - **Constraints:** Arbitrary.
 - **Impact:** The account to grant the role to.

Branches and code coverage

Intended branches

- Grants and enumerates a new member.
- ☐ Test coverage

Negative behavior

- Reverts if the caller does not have the admin role for `role`.
- ☐ Negative test

Function: `renounceRole(bytes32 role, address callerConfirmation)`

The `renounceRole` function revokes `role` from `msg.sender` if `callerConfirmation` equals `msg.sender`.

Inputs

- `role`
 - **Control:** Arbitrary.
 - **Constraints:** Arbitrary.
 - **Impact:** The role to renounce.
- `callerConfirmation`
 - **Control:** Arbitrary.
 - **Constraints:** Must equal `msg.sender`.
 - **Impact:** The address that must match `msg.sender`.

Branches and code coverage

Intended branches

- Removes `role` and its enumeration entry after matching confirmation.

☐ Test coverage

Negative behavior

- Reverts if `callerConfirmation` does not equal `msg.sender`.

☐ Negative test

Function: `revokeRole(bytes32 role, address account)`

The `revokeRole` function revokes `role` from `account` if the caller has the admin role for `role`.

Inputs

- `role`
 - **Control:** Set by role admin.
 - **Constraints:** The caller must hold the admin role associated with `role`.
 - **Impact:** The role to revoke.
- `account`
 - **Control:** Set by role admin.
 - **Constraints:** Arbitrary.
 - **Impact:** The account to revoke the role from.

Branches and code coverage

Intended branches

- Revokes `role` and removes its enumeration entry.

☐ Test coverage

Negative behavior

- Reverts if the caller does not have the admin role for `role`.

☐ Negative test

Function: `setMaxAge(uint256 _maxAge)`

The `setMaxAge` function updates the Chronicle staleness threshold for future price reads.

Inputs

- `_maxAge`
 - **Control:** Set by operator.
 - **Constraints:** Must not be zero and caller must hold `OPERATOR_ROLE`.
 - **Impact:** The maximum permitted oracle age.

Branches and code coverage

Intended branches

- Updates `MAX_AGE` for a nonzero value.

☒ Test coverage

Negative behavior

- Reverts if the caller does not have `OPERATOR_ROLE`.

☒ Negative test

- Reverts if `_maxAge` is zero.

☒ Negative test

4.5. Module: DollarPeggedAdapter.sol

Function: `getPrice()`

The `getPrice` function returns the fixed one-dollar WAD price of `1e18`.

Branches and code coverage

Intended branches

- Returns `1e18`.

☒ Test coverage

Negative behavior

- Does not revert because the function returns a constant price.

☐ Negative test

4.6. Module: GenericMintBurnStablecoinDelegate.sol

Function: constructor ()

The constructor function disables initializers on the implementation contract so the logic contract cannot be initialized directly, following the UUPS upgradable pattern.

Branches and code coverage

Intended branches

- Calls `_disableInitializers()` to permanently lock initialization on the implementation.

☒ Test coverage

Negative behavior

- Cannot initialize the implementation directly after deployment.

☒ Negative test

Function: initialize(address multiliquidAdmin, address initialIssuerAdmin, address _multiliquidSwap, byte[32] _stablecoinID, address _stablecoinAddress)

The initialize function performs one-time proxy setup by delegating to `__stablecoinDelegateBase_init`, which validates addresses, initializes upgradable modules, and grants protocol roles.

Inputs

- `multiliquidAdmin`
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero and distinct from `initialIssuerAdmin` and `_multiliquidSwap`.
 - **Impact:** Receives `DEFAULT_ADMIN_ROLE` and `PAUSE_ROLE` over this delegate.
- `initialIssuerAdmin`
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero and distinct from `multiliquidAdmin` and `_multiliquidSwap`.
 - **Impact:** Becomes the issuer admin who can manage RWA/stablecoin whitelists, fees, and custody addresses.
- `_multiliquidSwap`
 - **Control:** Chosen by the proxy deployer at initialization time.

- **Constraints:** Must be nonzero and distinct from both admin addresses.
 - **Impact:** Stored as `multiliquidSwap` and granted `MULTILIQID_SWAP_CONTRACT`, making it the sole caller of swap hooks.
- `_stablecoinID`
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero (`bytes32(0)` reverts).
 - **Impact:** Stored as the protocol identifier for this stablecoin integration.
- `_stablecoinAddress`
 - **Control:** Chosen by the proxy deployer at initialization time.
 - **Constraints:** Must be nonzero.
 - **Impact:** Stored as the ERC-20 used for mint, burn, and transfer operations in swap hooks.

Branches and code coverage

Intended branches

- Initializes access control, pausable, reentrancy guard, UUPS, and issuer admin modules via `__stablecoinDelegateBase_init`.
 - ☒ Test coverage
- Stores `multiliquidSwap`, `stablecoinID`, and `stablecoinAddress`.
 - ☒ Test coverage
- Grants `DEFAULT_ADMIN_ROLE` and `PAUSE_ROLE` to `multiliquidAdmin` and `MULTILIQID_SWAP_CONTRACT` to `_multiliquidSwap`.
 - ☒ Test coverage

Negative behavior

- Reverts if called when already initialized (initializer modifier).
 - ☒ Negative test
- Reverts on zero or colliding admin/swap addresses (`InvalidInitializationParams`, `InvalidIssuerAdmin`, `InvalidMultiliquidSwap`).
 - ☒ Negative test

Function call analysis

- No external calls — internal `__stablecoinDelegateBase_init` only (OpenZeppelin initializer chain).

4.7. Module: IssuerAdminUpgradeable.sol

Function: `__IssuerAdmin_init(address initialIssuerAdmin)`

The `__IssuerAdmin_init` function seeds the issuer-admin mapping during delegate initialization and emits `IssuerAdminAdded`.

Inputs

- `initialIssuerAdmin`
 - **Control:** Deployer via the proxy initializer.
 - **Constraints:** Must be nonzero.
 - **Impact:** Becomes the first issuer admin who can add or remove other issuer admins.

Branches and code coverage

Intended branches

- Marks a nonzero initial issuer admin and emits `IssuerAdminAdded`.
 - ☒ Test coverage
- Runs only during the initializer path via `onlyInitializing`.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidIssuerAdmin` when `initialIssuerAdmin` is zero.
 - ☒ Negative test
- Reverts if called outside an initializer context.
 - ☐ Negative test

Function: `_onlyIssuerAdmin()`

The `_onlyIssuerAdmin` internal guard requires `msg.sender` to be marked as an issuer admin in storage.

Branches and code coverage

Intended branches

- Allows execution when `issuerAdmins[msg.sender]` is true.

- ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.

- ☒ Negative test

Function: `addIssuerAdmin(address issuerAdmin)`

The `addIssuerAdmin` function lets an existing issuer admin grant issuer-admin status to another address.

Inputs

- `issuerAdmin`
 - **Control:** Existing issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Must be nonzero and not already an issuer admin.
 - **Impact:** Gains issuer-admin privileges including custody configuration and issuer-only delegate actions.

Branches and code coverage

Intended branches

- Adds a new issuer admin and emits `IssuerAdminAdded`.
 - ☒ Test coverage
- Returns without state change when the address is already an issuer admin.
 - ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts with `InvalidIssuerAdmin` when `issuerAdmin` is zero.
 - ☒ Negative test

Function: `removeIssuerAdmin(address issuerAdmin)`

The `removeIssuerAdmin` function lets an issuer admin revoke issuer-admin status from another address but not from themselves.

Inputs

- `issuerAdmin`
 - **Control:** Existing issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Must be nonzero, not the caller, and currently an issuer admin.
 - **Impact:** Loses issuer-admin privileges and can no longer call issuer-only delegate functions.

Branches and code coverage

Intended branches

- Clears issuer-admin status and emits `IssuerAdminRemoved`.
 - ☒ Test coverage
- Returns without state change when the address is not an issuer admin.
 - ☒ Test coverage

Negative behavior

- Reverts with `CannotRemoveSelf` when the caller tries to remove themselves.
 - ☒ Negative test
- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts with `InvalidIssuerAdmin` when `issuerAdmin` is zero.
 - ☒ Negative test

4.8. Module: JTRSYAdapter.sol

Function: `constructor(address _asyncVault)`

The `constructor` function stores the immutable vault used for JTRSY price calls.

Inputs

- `_asyncVault`
 - **Control:** Set by the deployer.
 - **Constraints:** Must not be zero.
 - **Impact:** The immutable external price source.

Branches and code coverage

Intended branches

- Stores a nonzero vault address.

☒ Test coverage

Negative behavior

- Reverts if the vault address is zero.

☒ Negative test

Function: getPrice()

The getPrice function reads the vault price per share with six decimals and scales it to WAD units with 18 decimals.

Branches and code coverage

Intended branches

- Returns the vault price multiplied by 1e12.

☒ Test coverage

Negative behavior

- Reverts if multiplication overflows.

☒ Negative test

- Reverts if the vault price call reverts.

☐ Negative test

Function call analysis

- `asyncVault.pricePerShare()`

- **What is controllable?** The configured vault controls the returned share price.
- **If the return value is controllable, how is it used and how can it go wrong?** Multiplies the returned value by 1e12. Reverts if multiplication overflows. Returns zero for zero.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

4.9. Module: JTRSYWhitelistAdapter.sol

Function: constructor(address _whitelist, address _jtrsyContract)

The constructor function stores the immutable JTRSY membership contract and token key.

Inputs

- `_whitelist`
 - **Control:** Set by the deployer.
 - **Constraints:** Must not be zero.
 - **Impact:** The immutable JTRSY membership contract.
- `_jtrsyContract`
 - **Control:** Set by the deployer.
 - **Constraints:** Must not be zero.
 - **Impact:** The token key passed to `isMember`.

Branches and code coverage

Intended branches

- Deploys with nonzero dependency addresses.
- ☒ Test coverage

Negative behavior

- Reverts if any required dependency address is zero.
- ☒ Negative test

Function: isWhitelisted(address from, address to, uint256 amount)

The `isWhitelisted` function checks recipient membership for the configured JTRSY token and ignores sender and amount.

Inputs

- `from`
 - **Control:** Arbitrary.
 - **Constraints:** Arbitrary.
 - **Impact:** Ignored.

- to
 - **Control:** Arbitrary.
 - **Constraints:** Arbitrary.
 - **Impact:** The recipient passed to isMember.
- amount
 - **Control:** Arbitrary.
 - **Constraints:** Arbitrary.
 - **Impact:** Ignored.

Branches and code coverage

Intended branches

- Returns the membership isValid flag for the configured token and recipient.
- ☒ Test coverage

Negative behavior

- Returns false if the recipient is not a JTRSY member.
- ☒ Negative test
- Reverts if the whitelist dependency reverts.
- ☐ Negative test

Function call analysis

- `WHITELIST.isMember(JTRSY_CONTRACT, to)`
 - **What is controllable?** to is controlled by the caller, JTRSY_CONTRACT is set at deployment, and the whitelist controls the returned tuple.
 - **If the return value is controllable, how is it used and how can it go wrong?** Only the isValid boolean is used. The validity timestamp is ignored.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

4.10. Module: JTRSYWhitelistAdapterV2.sol

Function: constructor(address _jtrsyContract)

The constructor function stores the immutable JTRSY transfer restriction contract.

Inputs

- `_jtrsyContract`
 - **Control:** Set by the deployer.
 - **Constraints:** Must not be zero.
 - **Impact:** The immutable JTRSY transfer restriction contract.

Branches and code coverage

Intended branches

- Deploys with nonzero dependency addresses.
- ☒ Test coverage

Negative behavior

- Reverts if any required dependency address is zero.
- ☒ Negative test

Function: `isWhitelisted(address from, address to, uint256 amount)`

The `isWhitelisted` function calls `checkTransferRestriction` with the transfer context and returns its boolean result.

Inputs

- `from`
 - **Control:** Arbitrary.
 - **Constraints:** Arbitrary.
 - **Impact:** The token sender passed to `checkTransferRestriction`.
- `to`
 - **Control:** Arbitrary.
 - **Constraints:** Arbitrary.
 - **Impact:** The token recipient passed to `checkTransferRestriction`.
- `amount`
 - **Control:** Arbitrary.
 - **Constraints:** Arbitrary.
 - **Impact:** The transfer amount passed to `checkTransferRestriction`.

Branches and code coverage

Intended branches

- Returns the external restriction result unchanged.
- ☒ Test coverage

Negative behavior

- Reverts if the JTRSY contract call reverts.
- ☐ Negative test

Function call analysis

- `JTRSY_CONTRACT.checkTransferRestriction(from, to, amount)`
 - **What is controllable?** `from`, `to`, and `amount` are controlled by the caller. The configured contract controls the boolean return.
 - **If the return value is controllable, how is it used and how can it go wrong?** The boolean is returned directly, so a malicious or misconfigured dependency can approve or reject any transfer.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

4.11. Module: MintBurnStablecoinDelegate.sol

Function: constructor ()

The `constructor` function disables initializers on the implementation contract so the logic contract cannot be initialized directly, following the UUPS upgradable pattern.

Branches and code coverage

Intended branches

- Calls `_disableInitializers()` to permanently lock initialization on the implementation.
- ☒ Test coverage

Negative behavior

- Cannot initialize the implementation directly after deployment.
- ☒ Negative test

Function: deployStablecoin(DelegateParams params)

The deployStablecoin function mints stablecoin to the receiver after MultiliquidSwap routes an RWA-for-stablecoin swap, pulling backing RWA into balance sheet custody and minting protocol fees to the vault.

Inputs

- params
 - **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations; end users cannot call this delegate directly.
 - **Constraints:** Enforced by onlyRole(MULTILIQID_SWAP_CONTRACT), whenNotPaused, nonReentrant, onlyWhitelistedRWA(params.assetInAddress), blacklist checks on user and receiver, and custodyAddressesSet; MultiliquidSwap must ensure user has approved the RWA transfer.
 - **Impact:** Moves RWA into balanceSheetCustodyAddress, mints protocol fee stablecoin to vault, and mints assetOutAmount stablecoin to receiver.

Branches and code coverage

Intended branches

- Transfers assetInAmount of assetInAddress RWA from user to balanceSheetCustodyAddress.
 - ☒ Test coverage
- Mints protocolFeeAmt stablecoin to vault when the fee is positive.
 - ☒ Test coverage
- Mints assetOutAmount stablecoin to receiver when the output is positive.
 - ☒ Test coverage
- Returns true on success.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller lacks MULTILIQID_SWAP_CONTRACT.
 - ☒ Negative test
- Reverts when paused or on reentrancy.
 - ☒ Negative test
- Reverts if assetInAddress is not a whitelisted RWA.

- ☒ Negative test
 - Reverts if user or receiver is blacklisted or custody addresses are unset.
- ☒ Negative test
 - Reverts if RWA transfer or stablecoin mint fails (insufficient allowance/balance or missing minter role).
- ☐ Negative test

Function call analysis

- SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user, this.balanceSheetCustodyAddress, params.assetInAmount)
 - **What is controllable?** RWA token contract and user approval — amount and addresses come from MultiliquidSwap params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Deposits backing collateral — failed or partial transfer reverts the mint leg.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap; malicious ERC-20 callbacks cannot reenter due to nonReentrant.
- stablecoin.mint(params.vault, params.protocolFeeAmt)
 - **What is controllable?** Delegate must hold minter role on the stablecoin — fee amount from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Creates protocol fee supply in the vault; unauthorized or failing mint reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back the RWA transfer in the same transaction.
- stablecoin.mint(params.receiver, params.assetOutAmount)
 - **What is controllable?** Minter role and output amount from MultiliquidSwap pricing.
 - **If the return value is controllable, how is it used and how can it go wrong?** Delivers newly minted stablecoin to the user — excessive mint without matching RWA is a pricing/custody risk outside this hook.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the entire deployment leg atomically.

Function: exchangeRWAs(DelegateParams params)

The exchangeRWAs function swaps one RWA for another through issuer custody without stablecoin touching the user's wallet, collecting incoming RWA, minting fee stablecoin, and paying out the output RWA.

Inputs

- `params`
 - **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations — end users cannot call this delegate directly.
 - **Constraints:** Enforced by `onlyRole(MULTILIQID_SWAP_CONTRACT)`, `whenNotPaused`, `nonReentrant`, whitelist checks on both RWA legs, blacklist checks on user and receiver, and `custodyAddressesSet`; MultiliquidSwap must ensure user has approved the input RWA transfer and custody has approved the output RWA transfer.
 - **Impact:** Moves input RWA to custody, mints protocol and LP fee stablecoin, and delivers output RWA to receiver.

Branches and code coverage

Intended branches

- Transfers `assetInAmount` of `assetInAddress` from user to `balanceSheetCustodyAddress`.
 - ☒ Test coverage
- Mints `protocolFeeAmt` stablecoin to `vault` when the protocol fee is positive.
 - ☒ Test coverage
- Mints `lpFeeAmt` stablecoin to `stablecoinCustodyAddress` when the LP fee is positive.
 - ☒ Test coverage
- Transfers `assetOutAmount` of `assetOutAddress` from `balanceSheetCustodyAddress` to receiver.
 - ☒ Test coverage
- Returns `true` on success.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller lacks `MULTILIQID_SWAP_CONTRACT`.
 - ☒ Negative test
- Reverts when paused or on reentrancy.
 - ☒ Negative test
- Reverts if either RWA is not whitelisted.
 - ☒ Negative test
- Reverts if user or receiver is blacklisted or custody addresses are unset.

- ☒ Negative test
- Reverts if RWA transfers or stablecoin mints fail.
- ☐ Negative test

Function call analysis

- SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user, this.balanceSheetCustodyAddress, params.assetInAmount)
 - **What is controllable?** Input RWA token and user approval — amounts from MultiliquidSwap params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Deposits sold RWA into custody — failed transfer reverts the exchange.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts atomically; nonReentrant blocks callback reentry.
- stablecoin.mint(params.vault, params.protocolFeeAmt)
 - **What is controllable?** Delegate minter role and protocol fee amount from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Creates fee supply in the vault — unauthorized mint reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back the input RWA transfer.
- stablecoin.mint(this.stablecoinCustodyAddress, params.lpFeeAmt)
 - **What is controllable?** Minter role and LP fee from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Credits issuer custody with redemption fee stablecoin — mint failure reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the entire RWA exchange leg.
- SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.balanceSheetCustodyAddress, params.receiver, params.assetOutAmount)
 - **What is controllable?** Output RWA token and custody approval to this delegate.
 - **If the return value is controllable, how is it used and how can it go wrong?** Delivers purchased RWA; insufficient custody inventory reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Malicious RWA revert/callback cannot reenter; whole swap fails.

Function: exchangeStablecoins(DelegateParams params)

The exchangeStablecoins function swaps one stablecoin for another through issuer custody, burning or pulling the input stablecoin, minting fees and output when the issuer token is involved,

and transferring external stablecoins from custody otherwise.

Inputs

- `params`
 - **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations — end users cannot call this delegate directly.
 - **Constraints:** Enforced by `onlyRole(MULTILIQUEID_SWAP_CONTRACT)`, `whenNotPaused`, `nonReentrant`, `blacklist` checks, `custodyAddressesSet`, `_checkIssuerStablecoinInExchange`, and `_onlyWhitelistedStablecoin` on the nonissuer leg; MultiliquidSwap must ensure user has approved relevant token transfers.
 - **Impact:** Moves stablecoins between user, custody, vault, and receiver; mints or burns issuer stablecoin and protocol/LP fee balances.

Branches and code coverage

Intended branches

- When `assetInAddress` is the issuer stablecoin, burns `assetInAmount` from user via `burnFrom`.
 - ☒ Test coverage
- When `assetInAddress` is external, transfers `assetInAmount` from user to `stablecoinCustodyAddress`.
 - ☒ Test coverage
- Mints `protocolFeeAmt` issuer stablecoin to vault and `lpFeeAmt` to `stablecoinCustodyAddress` when fees are positive.
 - ☒ Test coverage
- When `assetOutAddress` is the issuer stablecoin, mints `assetOutAmount` to receiver.
 - ☒ Test coverage
- When `assetOutAddress` is external, transfers `assetOutAmount` from `stablecoinCustodyAddress` to receiver.
 - ☒ Test coverage
- Returns `true` on success.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller lacks `MULTILIQUEID_SWAP_CONTRACT`.
 - ☒ Negative test

- Reverts when paused or on reentrancy.
 - ☒ Negative test
- Reverts if neither leg is the issuer stablecoin (InvalidStablecoinExchange).
 - ☒ Negative test
- Reverts if the external stablecoin leg is not whitelisted.
 - ☒ Negative test
- Reverts if user or receiver is blacklisted or custody addresses are unset.
 - ☒ Negative test
- Reverts if token transfers, burnFrom, or mints fail.
 - ☐ Negative test

Function call analysis

- `stablecoin.burnFrom(params.user, params.assetInAmount)`
 - **What is controllable?** Delegate burn permission and user balance for the issuer-token input leg.
 - **If the return value is controllable, how is it used and how can it go wrong?** Destroys incoming issuer supply — insufficient balance or allowance reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the exchange atomically.
- `SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user, this.stablecoinCustodyAddress, params.assetInAmount)`
 - **What is controllable?** External stablecoin contract and user approval for the nonissuer input leg.
 - **If the return value is controllable, how is it used and how can it go wrong?** Deposits backing stablecoin into issuer custody; failed transfer reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Malicious ERC-20 callbacks cannot reenter due to nonReentrant.
- `stablecoin.mint(params.vault, params.protocolFeeAmt)`
 - **What is controllable?** Delegate minter role and protocol fee from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Credits vault with fee supply — unauthorized mint reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents partial fee collection.
- `stablecoin.mint(this.stablecoinCustodyAddress, params.lpFeeAmt)`
 - **What is controllable?** Minter role and LP fee amount from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Credits issuer custody with acceptance/redemption fees.

- **What happens if it reverts, reenters or does other unusual control flow?**
Revert rolls back prior legs in the transaction.
- `stablecoin.mint(params.receiver, params.assetOutAmount)`
 - **What is controllable?** Minter role and priced output amount from MultiliquidSwap.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Delivers issuer stablecoin to the user — mint failure reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Revert aborts the entire stablecoin exchange.
- `SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.stablecoinCustodyAddress, params.receiver, params.assetOutAmount)`
 - **What is controllable?** External stablecoin in custody and approval to this delegate.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Pays whitelisted external stablecoin from custody; insufficient balance reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Token revert/callback cannot bypass `nonReentrant`.

Function: `receiveStablecoin(DelegateParams params)`

The `receiveStablecoin` function redeems stablecoin for RWA by collecting protocol and LP fees from the user, burning the net stablecoin amount via `burnFrom`, and releasing RWA from balance sheet custody to the receiver.

Inputs

- `params`
 - **Control:** Fully specified by MultiliquidSwap after pricing, slippage, and fee calculations; end users cannot call this delegate directly.
 - **Constraints:** Enforced by `onlyRole(MULTILIQUID_SWAP_CONTRACT)`, `whenNotPaused`, `nonReentrant`, `onlyWhitelistedRWA(params.assetOutAddress)`, blacklist checks on user and receiver, and `custodyAddressesSet`; MultiliquidSwap must ensure user has approved stablecoin transfers and custody has approved RWA outflow.
 - **Impact:** Collects fees to `vault` and `stablecoinCustodyAddress`, burns redeemed stablecoin, and delivers `assetOutAmount` of RWA to receiver.

Branches and code coverage

Intended branches

- Transfers protocolFeeAmt stablecoin from user to vault when the fee is positive.
 - ☒ Test coverage
- Transfers lpFeeAmt stablecoin from user to stablecoinCustodyAddress when the fee is positive.
 - ☒ Test coverage
- Burns assetInAmount - protocolFeeAmt - lpFeeAmt from user via burnFrom when the net amount is positive.
 - ☒ Test coverage
- Transfers assetOutAmount of assetOutAddress from balanceSheetCustodyAddress to receiver.
 - ☒ Test coverage
- Returns true on success.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller lacks MULTILIQUID_SWAP_CONTRACT.
 - ☒ Negative test
- Reverts when paused or on reentrancy.
 - ☒ Negative test
- Reverts if assetOutAddress is not a whitelisted RWA.
 - ☒ Negative test
- Reverts if user or receiver is blacklisted or custody addresses are unset.
 - ☒ Negative test
- Reverts if stablecoin transfers, burnFrom, or RWA transfer fail.
 - ☐ Negative test

Function call analysis

- SafeERC20.safeTransferFrom(stablecoin, params.user, params.vault, params.protocolFeeAmt)
 - **What is controllable?** Stablecoin token and user approval — fee routing from params.
 - **If the return value is controllable, how is it used and how can it go wrong?** Moves protocol fee to vault — failed transfer reverts redemption.

- **What happens if it reverts, reenters or does other unusual control flow?**
Revert aborts the swap — nonReentrant blocks callback reentry.
- SafeERC20.safeTransferFrom(stablecoin, params.user, this.stablecoinCustodyAddress, params.lpFeeAmt)
 - **What is controllable?** User allowance and stablecoin implementation for LP/redemption fees.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Credits issuer custody — undertransfer reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Malicious ERC-20 behavior causes atomic failure.
- stablecoin.burnFrom(params.user, amountToBurn)
 - **What is controllable?** Delegate burn permission on the stablecoin and user balance for the net redeem amount.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Destroys redeemed supply directly from the user — insufficient balance or missing allowance reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Revert rolls back fee transfers in the same transaction.
- SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.balanceSheetCustodyAddress, params.receiver, params.assetOutAmount)
 - **What is controllable?** RWA token and custody approval — output asset from params.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Pays out backing RWA; insufficient custody balance reverts.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Revert aborts after stablecoin legs; token hooks cannot bypass nonReentrant.

4.12. Module: MultiliquidSwap.sol

Function: DOMAIN_SEPARATOR ()

The DOMAIN_SEPARATOR view returns the EIP-712 domain separator used to hash swap batch approval permits.

Branches and code coverage

Intended branches

- Returns _domainSeparator () for off-chain permit signing.

☒ Test coverage

Negative behavior

- No revert paths; callable by any address.

☐ Negative test

Function: `_applyLegExactIn(FeeLeg leg, uint256 currentRouteWad, FeeTotals fees)`

The `_applyLegExactIn` function applies one forward fee leg to a route-unit amount and accumulates spread and explicit LP fees.

Inputs

- `leg`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Fee leg with rate, spread take, and rounding flags.
- `currentRouteWad`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Current route-unit WAD value.
- `fees`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Mutable fee accumulator for reporting.

Branches and code coverage

Intended branches

- Subtracts leg and spread fees from route value and records explicit LP fees when flagged.

☒ Test coverage

Negative behavior

- No revert paths — pure function.

☐ Negative test

Function: `_applyLegExactOut(FeeLeg leg, uint256 currentRouteWad, FeeTotals fees)`

The `_applyLegExactOut` function inverts one fee leg to find the gross route-unit amount needed before fees.

Inputs

- `leg`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Fee leg with rate, spread take, and rounding flags.
- `currentRouteWad`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Target net route-unit WAD after fees.
- `fees`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Mutable fee accumulator for reporting.

Branches and code coverage

Intended branches

- Solves gross route value with `divWadUp` and applies a one-wei rounding buffer when needed.
- ☒ Test coverage

Negative behavior

- No revert paths — pure function.
- ☐ Negative test

Function: `_applyYieldExactIn(PricingInputs inputs, uint256 inputAssetWad)`

The `_applyYieldExactIn` function converts a raw exact-in amount to yield-adjusted value for yield-bearing input stablecoins.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Pricing inputs containing yield mode and delegate address.
- `inputAssetWad`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Raw input amount in WAD units.

Branches and code coverage

Intended branches

- Returns the input unchanged when yield mode is none — otherwise calls `getYieldAmount`.

☒ Test coverage

Negative behavior

- Delegate view failures propagate as reverts.

☒ Negative test

Function call analysis

- `IYieldBearingDelegate(inputs.yieldDelegate)`
`.getYieldAmount(inputs.user, inputAssetWad, continuous)`
- **What is controllable?** `inputs.user` and the quoted input amount are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The yield-adjusted WAD amount feeds exact-in pricing; an inflated return understates required raw input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap `nonReentrant` guard.

Function: `_applyYieldExactOut(PricingInputs inputs, uint256 inputAssetWad, FeeTotals fees)`

The `_applyYieldExactOut` function converts required yield-adjusted value back to raw input and optionally rescales fee debits.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Pricing inputs containing yield mode and delegate address.
- `inputAssetWad`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Yield-adjusted input requirement in WAD units.
- `fees`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Mutable fee accumulator that may be rescaled to raw units.

Branches and code coverage

Intended branches

- Calls `getAmountForTargetValue` for yield-bearing inputs and optionally scales fee debits back to raw units.

☒ Test coverage

Negative behavior

- Delegate view failures propagate as reverts.

☒ Negative test

Function call analysis

- `IYieldBearingDelegate(inputs.yieldDelegate)`
`.getAmountForTargetValue(inputs.user, inputAssetWad, continuous)`
- **What is controllable?** `inputs.user` and the target yield-adjusted value are caller-controlled; the yield delegate is operator-configured.

- **If the return value is controllable, how is it used and how can it go wrong?**
The raw input WAD is derived for exact-out routes; a deflated return overstates required input.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap nonReentrant guard.

Function: `_buildRWAToRWAInputs(byte[32] stablecoinID, byte[32] rwaInID, byte[32] rwaOutID, uint256 rwaInAmount, uint256 rwaOutAmount)`

The `_buildRWAToRWAInputs` function assembles two-leg PricingInputs for an RWA-to-RWA route.

Inputs

- `stablecoinID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Delegate stablecoin ID owning the fee schedule.
- `rwaInID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Input RWA ID.
- `rwaOutID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output RWA ID.
- `rwaInAmount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-in input amount or exact-out max input bound.
- `rwaOutAmount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-out target or exact-in minimum output bound.

Branches and code coverage

Intended branches

- Loads both RWA adapter prices, route-unit USD value, discount/redemption fees, and issuer protocol fee into PricingInputs.

☒ Test coverage

Negative behavior

- Reverts when price adapter calls fail.

☒ Negative test

Function call analysis

- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.

Function: `_buildRWAToStablecoinInputs(byte[32] stablecoinID, byte[32] rwaID, uint256 rwaAmount, uint256 stablecoinAmount)`

The `_buildRWAToStablecoinInputs` function assembles PricingInputs for a one-leg RWA-to-stablecoin route.

Inputs

- `stablecoinID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output stablecoin ID.
- `rwaID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Input RWA ID.
- `rwaAmount`

- **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-in input amount or exact-out max input bound.
- stablecoinAmount
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-out target or exact-in minimum output bound.

Branches and code coverage

Intended branches

- Loads RWA adapter price, admin stablecoin USD value, discount rate, and issuer protocol fee into PricingInputs.
- ☒ Test coverage

Negative behavior

- Reverts when price adapter calls fail.
- ☒ Negative test

Function call analysis

- IPriceAdapter(priceAdapter).getPrice()
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.

Function: `_buildStablecoinToRWAInputs(byte[32] stablecoinID, byte[32] rwaID, uint256 stablecoinAmount, uint256 rwaAmount, address user)`

The `_buildStablecoinToRWAInputs` function assembles PricingInputs for a one-leg stablecoin-to-RWA route.

Inputs

- stablecoinID

- **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Input stablecoin ID.
- rwaID
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output RWA ID.
- stablecoinAmount
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-in input amount or exact-out max input bound.
- rwaAmount
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-out target or exact-in minimum output bound.
- user
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Account used for yield-bearing input pricing.

Branches and code coverage

Intended branches

- Loads admin USD value, RWA adapter price, redemption fee, and issuer protocol fee into PricingInputs.
- ☒ Test coverage

Negative behavior

- Reverts when price adapter calls fail.
- ☒ Negative test

Function call analysis

- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.

- **If the return value is controllable, how is it used and how can it go wrong?**
The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.

Function: `_buildStablecoinToStablecoinInputs(bool useDelegateForStablecoinOut, byte[32] stablecoinInID, uint256 stablecoinInAmount, byte[32] stablecoinOutID, uint256 stablecoinOutAmount, address user)`

The `_buildStablecoinToStablecoinInputs` function assembles `PricingInputs` for a one-leg stablecoin-to-stablecoin route.

Inputs

- `useDelegateForStablecoinOut`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Selects whether the output delegate owns the fee schedule.
- `stablecoinInID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Input stablecoin ID.
- `stablecoinInAmount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-in input amount or exact-out max input bound.
- `stablecoinOutID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output stablecoin ID.
- `stablecoinOutAmount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-out target or exact-in minimum output bound.
- `user`
 - **Control:** Calling internal swap path.

- **Constraints:** Must satisfy internal guard preconditions.
- **Impact:** Account used for yield-bearing input pricing.

Branches and code coverage

Intended branches

- Selects acceptance/redemption fees from the delegate stablecoin and packs yield mode flags for input-side yield bearing.

☒ Test coverage

Negative behavior

- No local revert paths beyond downstream quote failures.

☒ Negative test

Function: `_calculateExactIn(PricingInputs inputs)`

The `_calculateExactIn` function quotes an exact-input swap by applying yield adjustment, route conversion, issuer fee, and fee legs before output conversion.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Packed pricing inputs for one route.

Branches and code coverage

Intended branches

- Applies yield, route-unit conversion, issuer fee, each fee leg, and final output conversion with conservative rounding.

☒ Test coverage

- Reverts when route-unit-input fee debits would exceed `tokenInAmt`.

☒ Test coverage

Negative behavior

- Reverts with `InsufficientStablecoinInput` when explicit fee debits exceed the input amount.

☑ Negative test

Function call analysis

- `IYieldBearingDelegate(inputs.yieldDelegate)`
`.getYieldAmount(inputs.user, inputAssetWad, continuous)`
- **What is controllable?** `inputs.user` and the quoted input amount are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The yield-adjusted WAD amount feeds exact-in pricing; an inflated return understates required raw input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap `nonReentrant` guard.

Function: `_calculateExactOut(PricingInputs inputs)`

The `_calculateExactOut` function quotes an exact-output swap by reversing output conversion, fee legs, issuer fee, and input conversion.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Packed pricing inputs for one route.

Branches and code coverage

Intended branches

- Grosses up route value through reversed fee legs and may floor input to cover explicit fee debits.

☑ Test coverage

Negative behavior

- No explicit local reverts — downstream slippage checks enforce feasibility.

☑ Negative test

Function call analysis

- `IYieldBearingDelegate(inputs.yieldDelegate)`
- `.getAmountForTargetValue(inputs.user, inputAssetWad, continuous)`
- **What is controllable?** `inputs.user` and the target yield-adjusted value are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The raw input WAD is derived for exact-out routes; a deflated return overstates required input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap `nonReentrant` guard.

Function: `_calculateSpreadProtocolFeeRate(byte[32] stablecoinID, uint256 discountOrAcceptanceFee, uint256 redemptionFee)`

The `_calculateSpreadProtocolFeeRate` function computes the protocol take on issuer spread between configured leg fees.

Inputs

- `stablecoinID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stablecoin whose exemption status is checked.
- `discountOrAcceptanceFee`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Incoming discount or acceptance fee rate.
- `redemptionFee`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Outgoing redemption fee rate.

Branches and code coverage

Intended branches

- Returns zero when protocol-fee exempt, otherwise returns spread take rate floored at one basis point.

☒ Test coverage

Negative behavior

- No revert paths — view function.

☐ Negative test

Function: `_calculateSwapQuote(address user, SwapInputs inputs, uint256 routeFamily)`

The `_calculateSwapQuote` function builds `PricingInputs` for one route family and returns exact-in or exact-out resolved amounts.

Inputs

- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Account used for yield-bearing quote math.
- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Swap leg parameters and slippage hints.
- `routeFamily`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Resolved route family 0–3.

Branches and code coverage

Intended branches

- Builds route-specific pricing inputs and dispatches to `_calculateExactIn` or `_calculateExactOut`.

☒ Test coverage

Negative behavior

- Reverts when pricing input construction or fee math fails downstream.

☐ Negative test

Function call analysis

- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.
- `IYieldBearingDelegate(inputs.yieldDelegate)`
 - `.getYieldAmount(inputs.user, inputAssetWad, continuous)`
 - **What is controllable?** `inputs.user` and the quoted input amount are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The yield-adjusted WAD amount feeds exact-in pricing; an inflated return understates required raw input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap `nonReentrant` guard.
- `IYieldBearingDelegate(inputs.yieldDelegate)`
 - `.getAmountForTargetValue(inputs.user, inputAssetWad, continuous)`
 - **What is controllable?** `inputs.user` and the target yield-adjusted value are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The raw input WAD is derived for exact-out routes; a deflated return overstates required input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap `nonReentrant` guard.

Function: `_checkCanPause()`

The `_checkCanPause` function allows pausing when the caller has `OPERATOR_ROLE`, otherwise requires `EXTERNAL_PAUSER_ROLE`.

Branches and code coverage

Intended branches

- Permits `OPERATOR_ROLE` unconditionally, otherwise enforces `EXTERNAL_PAUSER_ROLE`.

☒ Test coverage

Negative behavior

- Reverts with OpenZeppelin access-control error when neither role is held.

☒ Negative test

Function: `_checkRoutePriceGuardrails(SwapInputs inputs, uint256 routeFamily)`

The `_checkRoutePriceGuardrails` function applies enabled stablecoin oracle guardrails to every stablecoin touched by a route family.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Swap leg whose stablecoin IDs determine which guardrails run.
- `routeFamily`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Resolved route family 0–3.

Branches and code coverage

Intended branches

- Checks input, output, or delegate stablecoin guardrails depending on route family.

☒ Test coverage

Negative behavior

- Reverts when any invoked `_checkStablecoinPriceGuardrail` fails.

☒ Negative test

Function call analysis

- `IPriceAdapter(priceAdapter).getPrice()`

- **What is controllable?** The configured price adapter controls the returned WAD price.
- **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.

Function: `_checkRwaAcceptance(byte[32] rwaID, RWAInfo rwa)`

The `_checkRwaAcceptance` function requires an accepted RWA with nonzero asset metadata and a configured price adapter.

Inputs

- `rwaID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** RWA identifier used to look up the price adapter.
- `rwa`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stored `RWAInfo` for the asset.

Branches and code coverage

Intended branches

- Allows execution when the RWA is accepted, has asset metadata, and has a price adapter.

☒ Test coverage

Negative behavior

- Reverts with `RWANotAccepted` when acceptance or metadata is missing.

☒ Negative test

- Reverts with `PriceAdapterNotSet` when no adapter is configured.

☒ Negative test

Function: `_checkRwaAndDeployStablecoin(RWAInfo rwa, address user, address receiver, address stablecoinDelegate, address stablecoinAddress, uint256 rwaAmount, uint256 stablecoinAmount, uint256 feeAmt)`

The `_checkRwaAndDeployStablecoin` function validates outbound RWA risk and settles an RWA-to-stablecoin leg through the stablecoin delegate.

Inputs

- `rwa`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Input RWA registry entry.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Payer of input RWA.
- `receiver`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Recipient of output stablecoin.
- `stablecoinDelegate`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Delegate executing custody transfers.
- `stablecoinAddress`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output stablecoin ERC-20 address.
- `rwaAmount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Resolved RWA input amount.
- `stablecoinAmount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Resolved stablecoin output amount.
- `feeAmt`

- **Control:** Calling internal swap path.
- **Constraints:** Must satisfy internal guard preconditions.
- **Impact:** Protocol fee routed to multiliquidVault.

Branches and code coverage

Intended branches

- Runs `_checkRwaOut` and calls `deployStablecoin` with protocol fee amount.

☒ Test coverage

Negative behavior

- Reverts with `RWAValidityCheckFailed` or `FailedToDeployStablecoin` when checks or delegate settlement fail.

☒ Negative test

Function call analysis

- `IRWADelegate(rwa.delegate).checkRWAOut(user, amount)`
 - **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValidityCheckFailed` and blocks outbound RWA risk.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(stablecoinDelegate)`
 - `.deployStablecoin(DelegateParams(...))`
- **What is controllable?** Swap caller supplies user, receiver, and amounts; delegate and asset addresses are protocol-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployStablecoin`; the delegate pulls RWA and mints/transfers stablecoin.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.

Function: `_checkRwaAndReceiveStablecoin(RWAInfo rwa, address user, address receiver, address stablecoinDelegate, address stablecoinAddress, uint256 rwaAmount, uint256 stablecoinAmount, uint256 feeAmt, uint256 lpFeeAmt)`

The `_checkRwaAndReceiveStablecoin` function validates inbound RWA risk and settles a stablecoin-to-RWA leg through the stablecoin delegate.

Inputs

- `rwa`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output RWA registry entry.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Payer of input stablecoin.
- `receiver`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Recipient of output RWA.
- `stablecoinDelegate`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Delegate executing custody transfers.
- `stablecoinAddress`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Input stablecoin ERC-20 address.
- `rwaAmount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Resolved RWA output amount.
- `stablecoinAmount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Resolved stablecoin input amount.

- feeAmt
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Protocol fee routed to multiliquidVault.
- lpFeeAmt
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Explicit LP/redemption fee retained by the issuer.

Branches and code coverage

Intended branches

- Runs `_checkRwaIn` and calls `receiveStablecoin` with protocol and LP fee amounts.
- ☒ Test coverage

Negative behavior

- Reverts with `RWAValidityCheckFailed` or `FailedToDeployRWA` when checks or delegate settlement fail.
- ☒ Negative test

Function call analysis

- `IRWADelegate(rwa.delegate).checkRWAIn(user, amount)`
 - **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValidityCheckFailed` and blocks inbound RWA risk.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(stablecoinDelegate)`
 - `.receiveStablecoin(DelegateParams(...))`
- **What is controllable?** Swap caller supplies user, receiver, and amounts; delegate and asset addresses are protocol-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployRWA`; the delegate performs ERC-20 transfers and RWA delivery.

- **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but nonReentrant guards the swap entry point.

Function: `_checkRwaIn(RWAInfo rwa, address user, uint256 amount)`

The `_checkRwaIn` function runs optional inbound RWA risk checks before stablecoin is deployed to a receiver.

Inputs

- `rwa`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** RWA registry entry including delegate metadata.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Account receiving inbound RWA exposure.
- `amount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Quoted RWA amount.

Branches and code coverage

Intended branches

- No-ops when `additionalRiskControls` is false, otherwise requires `checkRwaIn` to succeed.

☒ Test coverage

Negative behavior

- Reverts with `RWAValidityCheckFailed` when delegate risk checks fail.

☒ Negative test

Function call analysis

- `IRWADelegate(rwa.delegate).checkRwaIn(user, amount)`

- **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
- **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValidityCheckFailed` and blocks inbound RWA risk.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.

Function: `_checkRwaOut(RWAInfo rwa, address user, uint256 amount)`

The `_checkRwaOut` function runs optional outbound RWA risk checks before RWA leaves the user.

Inputs

- `rwa`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** RWA registry entry including delegate metadata.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Account sending outbound RWA.
- `amount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Quoted RWA amount.

Branches and code coverage

Intended branches

- No-ops when `additionalRiskControls` is false, otherwise requires `checkRWAOut` to succeed.

☒ Test coverage

Negative behavior

- Reverts with `RWAValidityCheckFailed` when delegate risk checks fail.

☒ Negative test

Function call analysis

- `IRWADelegate(rwa.delegate).checkRWAOut(user, amount)`
 - **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValidityCheckFailed` and blocks outbound RWA risk.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.

Function: `_checkStablecoinAcceptance(byte[32] stablecoinID)`

The `_checkStablecoinAcceptance` function requires a fully delegated stablecoin to be accepted for RWA-capable routes.

Inputs

- `stablecoinID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stablecoin identifier to validate.

Branches and code coverage

Intended branches

- Reuses swap acceptance checks and additionally requires a nonzero delegate address.
- ☒ Test coverage

Negative behavior

- Reverts with `StablecoinNotAccepted` when delegate metadata is missing.
- ☒ Negative test

Function: `_checkStablecoinAcceptanceForSwap(byte[32] stablecoinID)`

The `_checkStablecoinAcceptanceForSwap` function requires a stablecoin to be accepted for swap-only routes without requiring a delegate.

Inputs

- stablecoinID
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stablecoin identifier to validate.

Branches and code coverage

Intended branches

- Allows execution when acceptance, asset metadata, and admin USD value are configured.

☒ Test coverage

Negative behavior

- Reverts with StablecoinNotAccepted or USDValueNotSet when configuration is incomplete.

☒ Negative test

Function: `_checkStablecoinDelegateID(byte[32] actual, byte[32] expected)`

The `_checkStablecoinDelegateID` function requires the submitted stablecoin delegate ID to match the route's expected stablecoin ID.

Inputs

- actual
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Submitted `stablecoinDelegateID`.
- expected
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stablecoin ID implied by the route family.

Branches and code coverage

Intended branches

- Allows execution when `actual == expected`.
- ☒ Test coverage

Negative behavior

- Reverts with `InvalidStablecoinDelegateID` on mismatch.
- ☒ Negative test

Function: `_checkStablecoinPriceGuardrail(byte[32] stablecoinID)`

The `_checkStablecoinPriceGuardrail` function rejects swaps when an enabled stablecoin oracle price deviates beyond the configured band from the admin peg.

Inputs

- `stablecoinID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stablecoin whose guardrail config is consulted.

Branches and code coverage

Intended branches

- No-ops when guardrails are disabled.
- ☒ Test coverage
- Compares `IPriceAdapter.getPrice()` against `stablecoinUSDValue` within `stablecoinGuardrailBands`.
- ☒ Test coverage

Negative behavior

- Reverts with `adapter`, `USD value`, `band`, or `StablecoinPriceOutsideBand` errors when enabled config is invalid or deviation is too large.
- ☒ Negative test

Function call analysis

- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.

Function: `_checkUserWhitelistedForRWA(byte[32] rwaID, address rwaCustodyAddress, address user, uint256 amount)`

The `_checkUserWhitelistedForRWA` function enforces optional RWA recipient whitelist checks before outbound RWA settlement.

Inputs

- `rwaID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** RWA identifier used to look up the whitelist adapter.
- `rwaCustodyAddress`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Custody address originating the RWA transfer.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Intended RWA receiver.
- `amount`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Quoted RWA output amount.

Branches and code coverage

Intended branches

- No-ops when no adapter is configured, otherwise requires `isWhitelisted` to return `true`.

☑ Test coverage

Negative behavior

- Reverts with `UserNotWhitelisted` when adapter rejects the receiver.

☑ Negative test

Function call analysis

- `IWhitelistAdapter(adapter).isWhitelisted(rwaCustodyAddress, user, amount)`
 - **What is controllable?** The adapter address is operator-configured; user and amount come from the swap leg.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `UserNotWhitelisted`; a permissive adapter allows any receiver.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; adapter hooks could reenter, but `nonReentrant` guards the swap entry point.

Function: `_checkWad(uint256 value)`

The `_checkWad` function enforces that a fee rate is strictly less than 100% (1 WAD).

Inputs

- `value`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must be $< \text{WAD}$.
 - **Impact:** Fee or rate value to validate.

Branches and code coverage

Intended branches

- Allows values in $[0, \text{WAD})$.

☑ Test coverage

Negative behavior

- Reverts with `InvalidRate` when `value >= WAD`.

☒ Negative test

Function: `_domainSeparator()`

The `_domainSeparator` function returns the contract-specific EIP-712 domain separator, including chain ID and verifying contract address.

Branches and code coverage

Intended branches

- Hashes `name`, `version`, `block.chainid`, and `address(this)` into the domain separator.

☒ Test coverage

Negative behavior

- No revert paths — view function.

☐ Negative test

Function: `_executeAutoSweepIfAuthorized(SwapInputs inputs, byte[32] autoLiquidityStablecoinID, address lpVault, uint256 balanceBefore)`

The `_executeAutoSweepIfAuthorized` function converts an LP vault's postleg stablecoin balance delta into an exact-in stablecoin-to-yield-stablecoin leg when the vault has granted input allowance.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Original requested leg whose input stablecoin may have landed in the LP vault.
- `autoLiquidityStablecoinID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must be accepted and pass guardrails when executed.
 - **Impact:** Configured yield stablecoin ID for sweeps.
- `lpVault`

- **Control:** Calling internal swap path.
 - **Constraints:** Must not be zero or blacklisted.
 - **Impact:** Stablecoin custody address for the delegate.
- `balanceBefore`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** LP vault token balance snapshot taken before the requested leg.

Branches and code coverage

Intended branches

- Returns early when balance did not increase or LP allowance is zero.
 - ☒ Test coverage
- Quotes and executes a derived exact-in sweep leg atomically with the parent swap.
 - ☒ Test coverage

Negative behavior

- Reverts with `InsufficientStablecoinOutput` when the quoted sweep output is zero.
 - ☒ Negative test
- Reverts when guardrails or nested `_executeSingleLeg` settlement fails.
 - ☐ Negative test

Function call analysis

- `IERC20Metadata(stablecoinInfo[assetInID].assetAddress)`
 - `.balanceOf(lpVault)`
- **What is controllable?** The LP vault address comes from the stablecoin delegate; the token address is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The balance delta sizes optional autosweep legs; a manipulated balance could skew sweep input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts sweep derivation; fee-on-transfer tokens may report unexpected balances.
- `IYieldBearingDelegate(stablecoin.delegate).accrueInterest(user)`
 - **What is controllable?** user is the swap account whose yield state is synchronized before pricing.

- **If the return value is controllable, how is it used and how can it go wrong?**
No return value is consumed; accrual updates delegate state used by subsequent yield math.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the entire swap leg; malicious delegates could grief settlement.
- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.

Function: `_executeRequestedLegWithAutoLiquidity(SwapInputs inputs, address user, address receiver, address operator, bool usePermit)`

The `_executeRequestedLegWithAutoLiquidity` function executes one user-requested leg and may insert LP prefund and postleg sweep stablecoin-to-stablecoin legs when autoliquidity is configured.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Defines route family, assets, delegate ID, and slippage bounds for the leg.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Economic account for the requested leg.
- `receiver`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output recipient for the requested leg.
- `operator`
 - **Control:** Calling internal swap path.

- **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Swap submitter or address(this) for derived LP legs.
- usePermit
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Whether batch-level EIP-712 authorization already validated delegated spending.

Branches and code coverage

Intended branches

- Validates the leg, resolves route family, and optionally prefunds output custody for prefunded route IDs.
 - ☒ Test coverage
- Measures LP vault balance delta around the requested leg when sweep is enabled.
 - ☒ Test coverage
- Executes the requested leg and then an authorized autosweep when allowance and balance delta permit.
 - ☒ Test coverage

Negative behavior

- Reverts with InsufficientSwapAllowance when a prefunded route is requested but no autoliquidity stablecoin is configured.
 - ☒ Negative test
- Reverts when guardrails, quoting, prefund execution, requested-leg settlement, or sweep settlement fails.
 - ☐ Negative test

Function call analysis

- 'IStablecoinDelegate(stablecoinInfo[stablecoinDelegateID].delegate).stablecoinCustodyAddress()'
- **What is controllable?** The stablecoin delegate address is operator-configured at acceptance time.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned LP vault address gates prefund, sweep, and autoliquidity legs; zero or blacklisted vaults revert.

- **What happens if it reverts, reenters or does other unusual control flow?** A revert or invalid custody address aborts autoliquidity; delegate view calls cannot reenter.
- `IYieldBearingDelegate(stablecoin.delegate).accrueInterest(user)`
 - **What is controllable?** The user is the swap account whose yield state is synchronized before pricing.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed; accrual updates delegate state used by subsequent yield math.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the entire swap leg; malicious delegates could grief settlement.
- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.
- `IERC20Metadata(stablecoinInfo[assetInID].assetAddress)`
- `.balanceOf(lpVault)`
- **What is controllable?** The LP vault address comes from the stablecoin delegate; the token address is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The balance delta sizes optional autosweep legs; a manipulated balance could skew sweep input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts sweep derivation; fee-on-transfer tokens may report unexpected balances.

Function: `_executeSingleLeg(SwapInputs inputs, address user, address receiver, address operator, bool usePermit, uint256 routeFamily)`

The `_executeSingleLeg` function dispatches one validated leg to the correct route handler and emits a Swap event with resolved amounts and fees.

Inputs

- `inputs`

- **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Validated swap leg parameters.
- user
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Economic account for the leg.
- receiver
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output recipient for the leg.
- operator
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Swap submitter for allowance and permit logic.
- usePermit
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Whether delegated spending was authorized by batch permit.
- routeFamily
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Resolved route family 0–3 from route ID validation.

Branches and code coverage

Intended branches

- Dispatches to stable-to-RWA, RWA-to-stable, RWA-to-RWA, or stable-to-stable handlers based on routeFamily.
 - ☒ Test coverage
- Synchronizes yield-bearing input state before stable-to-stable legs when the input delegate executes.
 - ☒ Test coverage
- Emits Swap with resolved token amounts and fee breakdown.
 - ☒ Test coverage

Negative behavior

- Reverts when the selected route handler reverts.

☒ Negative test

Function call analysis

- `IYieldBearingDelegate(stablecoin.delegate).accrueInterest(user)`
 - **What is controllable?** The user is the swap account whose yield state is synchronized before pricing.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed; accrual updates delegate state used by subsequent yield math.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the entire swap leg; malicious delegates could grief settlement.

Function: `_flagByte(uint256 flags, uint256 shift)`

The `_flagByte` function reads one packed byte field from `PricingInputs.flags`.

Inputs

- `flags`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Packed quote metadata word.
- `shift`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Bit shift locating the desired eight-bit field.

Branches and code coverage

Intended branches

- Returns `uint8(flags >> shift)`.

☒ Test coverage

Negative behavior

- No revert paths — pure function.

☐ Negative test

Function: `_fromWad(uint256 value, uint8 decimals)`

The `_fromWad` function scales a WAD amount down to a target decimal precision with truncation.

Inputs

- `value`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** WAD-scaled amount.
- `decimals`
 - **Control:** Calling internal swap path.
 - **Constraints:** Values ≥ 18 pass through unchanged.
 - **Impact:** Target token decimals.

Branches and code coverage

Intended branches

- Divides by $10^{(18-\text{decimals})}$ when decimals are below 18.

☒ Test coverage

Negative behavior

- No revert paths — pure function.

☐ Negative test

Function: `_fromWadUp(uint256 value, uint8 decimals)`

The `_fromWadUp` function scales a WAD amount down to target decimals with upward rounding.

Inputs

- `value`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** WAD-scaled amount.

- decimals
 - **Control:** Calling internal swap path.
 - **Constraints:** Values ≥ 18 pass through unchanged.
 - **Impact:** Target token decimals.

Branches and code coverage

Intended branches

- Rounds up when converting to fewer decimals.
- ☒ Test coverage

Negative behavior

- No revert paths — pure function.
- ☐ Negative test

Function: `_getIssuerPaidProtocolFeeRate(byte[32] stablecoinID)`

The `_getIssuerPaidProtocolFeeRate` function returns the issuer-paid explicit protocol fee rate, respecting protocol-fee exemption.

Inputs

- `stablecoinID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stablecoin whose protocol fee rate is queried.

Branches and code coverage

Intended branches

- Returns zero when `protocolFeeExempt[stablecoinID]` is true, otherwise returns configured rate.
- ☒ Test coverage

Negative behavior

- No revert paths — view function.
- ☐ Negative test

Function: `_hashSwapInputsArray(SwapInputs[] inputs)`

The `_hashSwapInputsArray` function Keccak-hashes each swap leg and packs the array hash for EIP-712 batch permits.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact leg array to hash.

Branches and code coverage

Intended branches

- Returns `keccak256(abi.encodePacked(inputHashes))` over per-leg struct hashes.
- ☒ Test coverage

Negative behavior

- No revert paths — pure function.
- ☐ Negative test

Function: `_hashTypedData(byte[32] structHash)`

The `_hashTypedData` function computes the EIP-712 V0 digest (0x1901 prefix) for permit verification.

Inputs

- `structHash`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Struct hash to combine with the domain separator.

Branches and code coverage

Intended branches

- Returns `keccak256(abi.encodePacked('\x19\x01', domainSeparator,`

```
structHash)).
```

☒ Test coverage**Negative behavior**

- No revert paths — view function.

☐ Negative test**Function: `_isEip7702DelegatedAccount(address user)`**

The `_isEip7702DelegatedAccount` function detects EIP-7702 delegated EOAs by their 23-byte delegation code prefix.

Inputs

- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Signing account whose runtime code is inspected.

Branches and code coverage**Intended branches**

- Returns true when `user.code` matches the EIP-7702 delegation prefix bytes.

☒ Test coverage**Negative behavior**

- Returns false for EOAs and nondelegated contracts.

☒ Negative test**Function call analysis**

- `user.code` (external code read)
 - **What is controllable?** The signing user account controls its own bytecode via EIP-7702 delegation.
 - **If the return value is controllable, how is it used and how can it go wrong?** Delegation prefix bytes gate whether ECDSA is attempted before ERC-1271 validation.

- **What happens if it reverts, reenters or does other unusual control flow?**
Reading code is a view operation with no state change; empty code selects the EOA ECDSA path.

Function: `_isValidApprovalSignature(address user, byte[32] digest, bytes signature)`

The `_isValidApprovalSignature` function selects ECDSA or ERC-1271 validation based on the signing account's code layout.

Inputs

- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Signing account.
- `digest`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** EIP-712 digest to validate.
- `signature`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Signature bytes supplied by the permit.

Branches and code coverage

Intended branches

- Uses ECDSA for EOAs, ECDSA-first for EIP-7702 delegated EOAs, and ERC-1271 for other contracts.

☒ Test coverage

Negative behavior

- Returns false rather than reverting when signature format or validation fails.

☒ Negative test

Function call analysis

- `user.staticcall(abi.encodeWithSelector(ERC1271_MAGIC_VALUE, digest, signature))`
 - **What is controllable?** user is the signing account; digest and signature come from the submitted permit.
 - **If the return value is controllable, how is it used and how can it go wrong?** A returned magic value authorizes the batch permit; a malformed response is treated as invalid.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or short return fails signature validation; contract wallets control their own response.
- `user.code` (external code read)
 - **What is controllable?** The signing user account controls its own bytecode via EIP-7702 delegation.
 - **If the return value is controllable, how is it used and how can it go wrong?** Delegation prefix bytes gate whether ECDSA is attempted before ERC-1271 validation.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reading code is a view operation with no state change; empty code selects the EOA ECDSA path.

Function: `_isValidERC1271ApprovalSignature(address user, byte[32] digest, bytes signature)`

The `_isValidERC1271ApprovalSignature` function validates a contract wallet signature via ERC-1271 `isValidSignature`.

Inputs

- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Contract wallet expected to validate the digest.
- `digest`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** EIP-712 digest passed to ERC-1271.
- `signature`
 - **Control:** Calling internal swap path.

- **Constraints:** Must satisfy internal guard preconditions.
- **Impact:** Opaque signature bytes forwarded to the wallet.

Branches and code coverage

Intended branches

- Accepts the signature when a staticcall returns the ERC-1271 magic value.

☒ Test coverage

Negative behavior

- Returns false on failed staticcall, short return data, or non-magic response.

☒ Negative test

Function call analysis

- `user.staticcall(abi.encodeWithSelector(ERC1271_MAGIC_VALUE, digest, signature))`
 - **What is controllable?** user is the signing account; digest and signature come from the submitted permit.
 - **If the return value is controllable, how is it used and how can it go wrong?** A returned magic value authorizes the batch permit; a malformed response is treated as invalid.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or short return fails signature validation; contract wallets control their own response.

Function: `_isValidEcdsaApprovalSignature(address user, byte[32] digest, bytes signature)`

The `_isValidEcdsaApprovalSignature` function validates 65-byte or 64-byte EIP-2098 ECDSA signatures against user.

Inputs

- user
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Expected signer address.
- digest

- **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Message digest to recover against.
- signature
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** 65-byte or 64-byte ECDSA signature.

Branches and code coverage

Intended branches

- Recovers the signer with `ECDSA.tryRecover` and compares it to `user`.
- ☒ Test coverage

Negative behavior

- Returns false for unsupported signature lengths or recovery errors.
- ☒ Negative test

Function: `_notBlacklisted(address user, address sender, address receiver)`

The `_notBlacklisted` function rejects swap batch identities present in the global blacklist.

Inputs

- user
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Economic account for the batch.
- sender
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Swap operator for the batch.
- receiver
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Batch output recipient.

Branches and code coverage

Intended branches

- Allows execution when none of the three addresses is blacklisted.

☒ Test coverage

Negative behavior

- Reverts with `UserBlacklisted` when any address is blacklisted.

☒ Negative test

Function: `_onlyStablecoinDelegate(byte[32] stablecoinID)`

The `_onlyStablecoinDelegate` function requires `msg.sender` to be the registered delegate for `stablecoinID`.

Inputs

- `stablecoinID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stablecoin domain whose delegate mapping is checked.

Branches and code coverage

Intended branches

- Allows execution when `stablecoinInfo[stablecoinID].delegate == msg.sender`.

☒ Test coverage

Negative behavior

- Reverts with `NotStablecoinDelegate` on mismatch.

☒ Negative test

Function: `_packSwapFlags(uint256 routeUnitSide, uint256 yieldMode, uint256 legCount, uint8 assetInDecimals, uint8 assetOutDecimals, uint8 feeStablecoinDecimals, uint256 quoteFlags)`

The `_packSwapFlags` function packs route metadata into one flags word for the quote engine.

Inputs

- routeUnitSide
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Two-bit route-unit-side enum.
- yieldMode
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Two-bit yield-mode enum.
- legCount
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Two-bit fee-leg count.
- assetInDecimals
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Input asset decimals byte.
- assetOutDecimals
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Output asset decimals byte.
- feeStablecoinDecimals
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Fee denomination decimals byte.
- quoteFlags
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact-out quote flag bits.

Branches and code coverage

Intended branches

- OR-combines all fields into a single flags word.

☒ Test coverage

Negative behavior

- No revert paths — pure function.
- Negative test

Function: `_spendSwapAllowance(SwapInputs inputs, uint256 actualAmountIn, address user, address receiver, address operator, bool usePermit)`

The `_spendSwapAllowance` function enforces delegated swap authorization by decrementing standing input-asset allowance when the operator is not the user.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Swap leg whose `assetInID` scopes allowance spending.
- `actualAmountIn`
 - **Control:** Calling internal swap path.
 - **Constraints:** May exceed `calldata` max on exact-out routes.
 - **Impact:** Resolved input amount after quoting.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Economic account whose allowance is charged.
- `receiver`
 - **Control:** Calling internal swap path.
 - **Constraints:** Delegated output-to-third-party routes require a permit.
 - **Impact:** Must equal `user` for allowance spending.
- `operator`
 - **Control:** Calling internal swap path.
 - **Constraints:** Skips spending when equal to `user`.
 - **Impact:** Swap submitter.
- `usePermit`
 - **Control:** Calling internal swap path.
 - **Constraints:** Permit bypasses allowance spending.
 - **Impact:** Batch permit authorization flag.

Branches and code coverage

Intended branches

- No-ops for direct user swaps, permit-authorized batches, or infinite allowance.
 - ☒ Test coverage
- Decrements swapInputAllowances and emits SwapInputAllowanceSpent otherwise.
 - ☒ Test coverage

Negative behavior

- Reverts with InsufficientSwapAllowance when delegated spending is required but allowance is insufficient or receiver != user.
 - ☒ Negative test

Function: `_stablecoinDelegateCustodyAddress(byte[32] stablecoinDelegateID)`

The `_stablecoinDelegateCustodyAddress` function returns the LP vault address from the registered stablecoin delegate and rejects invalid custody targets.

Inputs

- `stablecoinDelegateID`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must map to an accepted stablecoin with a delegate.
 - **Impact:** Stablecoin delegate domain for auto-liquidity legs.

Branches and code coverage

Intended branches

- Returns `IStablecoinDelegate.stablecoinCustodyAddress()` when the vault is nonzero, not this contract, and not blacklisted.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidVault` when custody is zero, points to this contract, or is blacklisted.
 - ☒ Negative test

Function call analysis

- `IStablecoinDelegate(stablecoinInfo[stablecoinDelegateID].delegate)`
`.stablecoinCustodyAddress()`
- **What is controllable?** The stablecoin delegate address is operator-configured at acceptance time.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned LP vault address gates prefund, sweep, and autoliquidity legs; zero or blacklisted vaults revert.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or invalid custody address aborts autoliquidity; delegate view calls cannot reenter.

Function: `_swapIntoRWA(SwapInputs inputs, address user, address receiver, address operator, bool usePermit)`

The `_swapIntoRWA` function quotes a stablecoin-to-RWA leg, enforces slippage and whitelist checks, and settles through the stablecoin delegate's `receiveStablecoin` hook.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stable-to-RWA route parameters and slippage bounds.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Payer of input stablecoin.
- `receiver`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Recipient of output RWA.
- `operator`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Delegated swap submitter when applicable.
- `usePermit`

- **Control:** Calling internal swap path.
- **Constraints:** Must satisfy internal guard preconditions.
- **Impact:** Whether delegated spending uses a batch permit instead of allowance.

Branches and code coverage

Intended branches

- Quotes exact-in or exact-out and enforces slippage bounds.
 - ☒ Test coverage
- Spends delegated input allowance when required.
 - ☒ Test coverage
- Checks RWA recipient whitelist and calls `receiveStablecoin` on the stablecoin delegate.
 - ☒ Test coverage

Negative behavior

- Reverts with `InsufficientStablecoinInput` or `InsufficientRWAOutput` on slippage failure.
 - ☒ Negative test
- Reverts with `UserNotWhitelisted`, `RWAValidityCheckFailed`, or `FailedToDeployRWA` when downstream checks fail.
 - ☒ Negative test

Function call analysis

- `IStablecoinDelegate(stablecoinDelegate).balanceSheetCustodyAddress()`
 - **What is controllable?** The stablecoin delegate is route-configured via `stablecoinDelegateID`.
 - **If the return value is controllable, how is it used and how can it go wrong?** The custody address is passed to the RWA whitelist adapter as the transfer origin.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the swap leg before delegate settlement.
- `IWhitelistAdapter(adapter).isWhitelisted(rwaCustodyAddress, user, amount)`
 - **What is controllable?** The adapter address is operator-configured; user and amount come from the swap leg.
 - **If the return value is controllable, how is it used and how can it go wrong?**

A false return reverts with `UserNotWhitelisted`; a permissive adapter allows any receiver.

- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; adapter hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IRWADelegate(rwa.delegate).checkRWAIn(user, amount)`
 - **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValidityCheckFailed` and blocks inbound RWA risk.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(stablecoinDelegate)`
 - `.receiveStablecoin(DelegateParams(...))`
- **What is controllable?** Swap caller supplies user, receiver, and amounts; delegate and asset addresses are protocol-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployRWA`; the delegate performs ERC-20 transfers and RWA delivery.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.

Function: `_swapIntoStablecoin(SwapInputs inputs, address user, address receiver, address operator, bool usePermit)`

The `_swapIntoStablecoin` function quotes an RWA-to-stablecoin leg, runs optional RWA outbound risk checks, and settles through the stablecoin delegate's `deployStablecoin` hook.

Inputs

- inputs
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** RWA-to-stable route parameters and slippage bounds.
- user
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.

- **Impact:** Payer of input RWA.
- receiver
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Recipient of output stablecoin.
- operator
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Delegated swap submitter when applicable.
- usePermit
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Whether delegated spending uses a batch permit instead of allowance.

Branches and code coverage

Intended branches

- Quotes exact-in or exact-out and enforces slippage bounds.
 - ☒ Test coverage
- Spends delegated input allowance when required.
 - ☒ Test coverage
- Calls deployStablecoin after optional checkRWAOut.
 - ☒ Test coverage

Negative behavior

- Reverts with InsufficientRWAInput or InsufficientStablecoinOutput on slippage failure.
 - ☒ Negative test
- Reverts with RWAValidityCheckFailed or FailedToDeployStablecoin when delegate settlement fails.
 - ☒ Negative test

Function call analysis

- IRWADelegate(rwa.delegate).checkRWAOut(user, amount)

- **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
- **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValidityCheckFailed` and blocks outbound RWA risk.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(stablecoinDelegate)`
`.deployStablecoin(DelegateParams(...))`
- **What is controllable?** Swap caller supplies user, receiver, and amounts; delegate and asset addresses are protocol-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployStablecoin`; the delegate pulls RWA and mints/transfers stablecoin.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.

Function: `_swapRWAToRWA(SwapInputs inputs, address user, address receiver, address operator, bool usePermit)`

The `_swapRWAToRWA` function quotes an RWA-to-RWA leg, runs optional RWA risk hooks, and settles atomically through the stablecoin delegate's `exchangeRWAs` hook.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** RWA-to-RWA route parameters and slippage bounds.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Payer of input RWA.
- `receiver`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Recipient of output RWA.

- operator
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Delegated swap submitter when applicable.
- usePermit
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Whether delegated spending uses a batch permit instead of allowance.

Branches and code coverage

Intended branches

- Quotes exact-in or exact-out and enforces slippage bounds.
 - ☒ Test coverage
- Checks output RWA whitelist and optional checkRWAIn / checkRWAOut hooks.
 - ☒ Test coverage
- Calls exchangeRWAs on the route's stablecoin delegate.
 - ☒ Test coverage

Negative behavior

- Reverts with InsufficientRWAInput or InsufficientRWAOutput on slippage failure.
 - ☒ Negative test
- Reverts with RWAValidityCheckFailed or FailedToDeployRWA when delegate settlement fails.
 - ☒ Negative test

Function call analysis

- `IStablecoinDelegate(stablecoinDelegate).balanceSheetCustodyAddress()`
 - **What is controllable?** The stablecoin delegate is route-configured via `stablecoinDelegateID`.
 - **If the return value is controllable, how is it used and how can it go wrong?** The custody address is passed to the RWA whitelist adapter as the transfer origin.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the swap leg before delegate settlement.

- `IWhitelistAdapter(adapter).isWhitelisted(rwaCustodyAddress, user, amount)`
 - **What is controllable?** The adapter address is operator-configured; user and amount come from the swap leg.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `UserNotWhitelisted`; a permissive adapter allows any receiver.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; adapter hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IRWADelegate(rwa.delegate).checkRWAIn(user, amount)`
 - **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValidityCheckFailed` and blocks inbound RWA risk.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IRWADelegate(rwa.delegate).checkRWAOut(user, amount)`
 - **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValidityCheckFailed` and blocks outbound RWA risk.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(stablecoinDelegate)`
 - `.exchangeRWAs(DelegateParams(...))`
 - **What is controllable?** Swap caller supplies user, receiver, and amounts; RWA addresses come from accepted asset registry.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployRWA`; the delegate atomically exchanges both RWA legs.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.

Function: `_swapStablecoinToStablecoin(SwapInputs inputs, address user, address receiver, address operator, bool usePermit)`

The `_swapStablecoinToStablecoin` function quotes a stablecoin-to-stablecoin leg and settles through the selected delegate's `exchangeStablecoins` hook.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Stable-to-stable route parameters and slippage bounds.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Payer of input stablecoin.
- `receiver`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Recipient of output stablecoin.
- `operator`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Delegated swap submitter when applicable.
- `usePermit`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Whether delegated spending uses a batch permit instead of allowance.

Branches and code coverage

Intended branches

- Quotes exact-in or exact-out and enforces slippage bounds.
 - ☒ Test coverage
- Selects the delegate on the input or output stablecoin based on `stablecoinDelegateID`.
 - ☒ Test coverage

- Calls `exchangeStablecoins` with protocol and LP fee amounts.

☑ Test coverage

Negative behavior

- Reverts with `InsufficientStablecoinInput` or `InsufficientStablecoinOutput` on slippage failure.

☑ Negative test

- Reverts with `FailedToDeployStablecoin` when delegate settlement fails.

☑ Negative test

Function call analysis

- `IStablecoinDelegate(delegateStablecoin)`
`.exchangeStablecoins(DelegateParams(...))`
- **What is controllable?** Swap caller supplies user, receiver, and amounts; stablecoin addresses come from accepted asset registry.
 - **If the return value is controllable, how is it used and how can it go wrong?**
A false return reverts with `FailedToDeployStablecoin`; the delegate atomically exchanges both stablecoin legs.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.

Function: `_syncYieldBearingStablecoinForPricing(StablecoinInfo stablecoin, address user)`

The `_syncYieldBearingStablecoinForPricing` function accrues yield on a yield-bearing stablecoin delegate before quoting or settling a leg.

Inputs

- `stablecoin`
 - **Control:** Calling internal swap path.
 - **Constraints:** No-op when `yieldBearing` is false.
 - **Impact:** Accepted stablecoin registry entry.
- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.

- **Impact:** Account whose yield position should be synchronized.

Branches and code coverage

Intended branches

- Calls `accrueInterest(user)` only when `stablecoin.yieldBearing` is true.

☒ Test coverage

Negative behavior

- No local revert paths — delegate accrual failures propagate.

☒ Negative test

Function call analysis

- `IYieldBearingDelegate(stablecoin.delegate).accrueInterest(user)`
 - **What is controllable?** `user` is the swap account whose yield state is synchronized before pricing.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed; accrual updates delegate state used by subsequent yield math.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the entire swap leg; malicious delegates could grief settlement.

Function: `_toWad(uint256 value, uint8 decimals)`

The `_toWad` function scales a native-token amount up to 18-decimal WAD precision.

Inputs

- `value`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Native-token amount.
- `decimals`
 - **Control:** Calling internal swap path.
 - **Constraints:** Values ≥ 18 pass through unchanged.
 - **Impact:** Current token decimals.

Branches and code coverage

Intended branches

- Multiplies by 10^{18} (18-decimals) when decimals are below 18.
- ☒ Test coverage

Negative behavior

- No revert paths — pure function.
- ☐ Negative test

Function: `_validateRouteConfig(SwapInputs inputs, uint256 routeId)`

The `_validateRouteConfig` function checks asset acceptance, delegate ID alignment, and route-family-specific constraints.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Swap leg with route, asset, and delegate IDs.
- `routeId`
 - **Control:** Calling internal swap path.
 - **Constraints:** Prefunded aliases resolve to base families 1 or 3.
 - **Impact:** Submitted route ID.

Branches and code coverage

Intended branches

- Resolves route family and enforces acceptance, delegate alignment, and distinct asset addresses per family.
- ☒ Test coverage

Negative behavior

- Reverts with `InvalidStablecoinDelegateID`, `RWANotAccepted`, `StablecoinNotAccepted`, `IdenticalAssets`, or `IdenticalStablecoins`.
- ☒ Negative test

Function: `_validateSwapEnvelope(SwapInputs inputs, uint256 routeId)`

The `_validateSwapEnvelope` function enforces supported route IDs and rejects identical input/output asset IDs.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Swap leg whose asset IDs are compared.
- `routeId`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must be `<= PREFUNDED_STABLE_TO_STABLE_EXACT_OUT`.
 - **Impact:** Route ID taken from the leg.

Branches and code coverage**Intended branches**

- Allows distinct `assetInID` and `assetOutID` for supported route IDs.
 - ☒ Test coverage

Negative behavior

- Reverts with `UnsupportedRouteId` for unknown route IDs.
 - ☒ Negative test
- Reverts with `IdenticalAssets` when input and output asset IDs match.
 - ☒ Negative test

Function: `_validateSwapLeg(SwapInputs inputs)`

The `_validateSwapLeg` function validates one leg's envelope, route configuration, and price guardrails before settlement.

Inputs

- `inputs`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.

- **Impact:** Swap leg to validate.

Branches and code coverage

Intended branches

- Calls `_validateSwapEnvelope`, `_validateRouteConfig`, and `_checkRoutePriceGuardrails`.
 - ☒ Test coverage
- Returns the resolved route family for downstream handlers.
 - ☒ Test coverage

Negative behavior

- Reverts when any nested validation or guardrail check fails.
 - ☐ Negative test

Function call analysis

- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.

Function: `_verifyApprovalSignature(address user, byte[32] structHash, bytes signature)`

The `_verifyApprovalSignature` function hashes typed data and requires a valid ECDSA or ERC-1271 approval signature from user.

Inputs

- `user`
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Signing account for the permit.

- structHash
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** EIP-712 struct hash to wrap in the domain separator.
- signature
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** ECDSA or ERC-1271 signature bytes.

Branches and code coverage

Intended branches

- Builds the EIP-712 digest and delegates to `_isValidApprovalSignature`.
- ☒ Test coverage

Negative behavior

- Reverts with `InvalidSignature` when validation fails.
- ☒ Negative test

Function: `_verifySwapBatchPermit(address user, address receiver, SwapInputs[] inputs, ApprovalPermit permit)`

The `_verifySwapBatchPermit` function verifies one EIP-712 batch permit for the exact submitted leg array and increments the user nonce on success.

Inputs

- user
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Economic account that signed the permit.
- receiver
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Batch output recipient bound into the permit digest.
- inputs
 - **Control:** Calling internal swap path.

- **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Exact leg array hashed into the permit.
- permit
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Deadline and signature material for EIP-712 verification.

Branches and code coverage

Intended branches

- Rejects expired permits, verifies the typed-data signature, and increments `nonces[user]`.

☒ Test coverage

Negative behavior

- Reverts with `PermitExpired` when `block.timestamp > permit.deadline`.

☒ Negative test

- Reverts with `InvalidSignature` when ECDSA/ERC-1271 validation fails.

☒ Negative test

Function: `_writeFeeOutputs(PricingInputs inputs, FeeTotals fees, SwapResolved outputs)`

The `_writeFeeOutputs` function writes accumulated route-unit fees into the resolved output using fee-stablecoin decimals.

Inputs

- inputs
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Pricing inputs containing packed fee-stablecoin decimals.
- fees
 - **Control:** Calling internal swap path.
 - **Constraints:** Must satisfy internal guard preconditions.
 - **Impact:** Accumulated issuer, spread, and explicit LP fees in route-unit WAD.
- outputs

- **Control:** Calling internal swap path.
- **Constraints:** Must satisfy internal guard preconditions.
- **Impact:** Mutable resolved output struct to populate.

Branches and code coverage

Intended branches

- Converts issuer, spread, and LP fees from WAD to native fee-token units with upward rounding.

☒ Test coverage

Negative behavior

- No revert paths — pure function.

☐ Negative test

Function: `adjustSwapInputAllowance(address operator, byte[32] assetInID, uint256 amount, bool increase)`

The `adjustSwapInputAllowance` function lets a user increase or decrease standing delegated input-asset approval for a specific operator.

Inputs

- `operator`
 - **Control:** Caller-chosen delegated swap submitter.
 - **Constraints:** Must be nonzero.
 - **Impact:** Address allowed to spend the caller's input-asset allowance.
- `assetInID`
 - **Control:** Caller-chosen approved input asset ID.
 - **Constraints:** Arbitrary accepted asset key.
 - **Impact:** Scopes allowance to one input asset instead of one route pair.
- `amount`
 - **Control:** Caller-chosen delta magnitude.
 - **Constraints:** Must not exceed current allowance when decreasing.
 - **Impact:** Amount added or removed from the standing approval.
- `increase`
 - **Control:** Caller chooses add versus subtract.

- **Constraints:** Boolean flag only.
- **Impact:** `true` increases allowance; `false` decreases it.

Branches and code coverage

Intended branches

- Updates `swapInputAllowances[user][operator][assetInID]` and emits `SwapInputAllowanceSet`.
☒ Test coverage
- Supports infinite approval via `type(uint256).max`, which is not decremented by swaps.
☒ Test coverage

Negative behavior

- Reverts with `InvalidSwapAddress` when operator is zero.
☒ Negative test
- Reverts with `InsufficientSwapAllowance` when decreasing by more than the current allowance.
☒ Negative test

Function: `configureStablecoinGuardrail(byte[32] stablecoinID, address priceAdapter, uint256 band, bool enabled)`

The `configureStablecoinGuardrail` function enables or disables stablecoin oracle guardrails and stores adapter and band configuration.

Inputs

- `stablecoinID`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must satisfy function-specific validation.
 - **Impact:** Selects the stablecoin whose guardrail config is changed.
- `priceAdapter`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must be nonzero when `enabled` is `true`.
 - **Impact:** Stored when enabling guardrails.
- `band`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.

- **Constraints:** Must be nonzero and $< \text{WAD}$ when enabled.
 - **Impact:** Maximum allowed WAD deviation from the admin peg.
- enabled
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** When false, stored config is deleted.
 - **Impact:** Turns guardrail enforcement on or off.

Branches and code coverage

Intended branches

- On enable, stores adapter, band, and enabled flag after validating admin USD value is set.
 - ☒ Test coverage
- On disable, deletes adapter, band, and enabled state and emits a cleared guardrail event.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidRate` when enabled and band is zero or $\geq \text{WAD}$.
 - ☒ Negative test
- Reverts with `StablecoinPriceAdapterNotSet` when enabled and adapter is zero.
 - ☒ Negative test
- Reverts with `USDValueNotSet` when enabled and admin USD value is zero.
 - ☒ Negative test

Function: constructor ()

The constructor function disables initializers on the implementation contract so the logic contract cannot be initialized outside the proxy deployment path.

Branches and code coverage

Intended branches

- Calls `_disableInitializers()` so only the proxy initializer can run `initialize`.
 - ☒ Test coverage

Negative behavior

- No revert paths; the constructor always succeeds.

☐ Negative test

Function: `initialize(address admin, address operator, address vault)`

The `initialize` function wires upgradable modules, grants admin/operator/blacklister roles, and stores the protocol fee vault address.

Inputs

- `admin`
 - **Control:** Deployer via the proxy initializer.
 - **Constraints:** Must be nonzero.
 - **Impact:** Receives `DEFAULT_ADMIN_ROLE` and `OPERATOR_ROLE`.
- `operator`
 - **Control:** Deployer via the proxy initializer.
 - **Constraints:** Must be nonzero.
 - **Impact:** Receives `OPERATOR_ROLE` and `BLACKLISTER_ROLE`.
- `vault`
 - **Control:** Deployer via the proxy initializer.
 - **Constraints:** Must be nonzero.
 - **Impact:** Stored as `multiliquidVault` for delegate fee routing.

Branches and code coverage

Intended branches

- Initializes `AccessControl`, `Pausable`, `ReentrancyGuard`, and `UUPS` modules.
 - ☒ Test coverage
- Grants admin, operator, and blacklister roles and sets `BLACKLISTER_ROLE` admin to `OPERATOR_ROLE`.
 - ☒ Test coverage
- Persists `multiliquidVault` for protocol fee collection.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidAdmin` when admin or operator is zero.

- ☒ Negative test
 - Reverts with `InvalidVault` when `vault` is zero.
- ☒ Negative test

Function call analysis

- Internal OpenZeppelin initialization only — no external calls.

Function: `pause()`

The `pause` function halts swap and other `whenNotPaused` entrypoints for operators or external pausers.

Branches and code coverage

Intended branches

- Sets the Pausable flag via `_pause()` when caller has `OPERATOR_ROLE` or `EXTERNAL_PAUSER_ROLE`.
- ☒ Test coverage

Negative behavior

- Reverts with OpenZeppelin access-control error when caller lacks pause authority.
- ☒ Negative test

Function: `quoteSwap(address user, SwapInputs inputs)`

The `quoteSwap` view quotes one swap leg without spending allowances, verifying permits, or mutating yield-bearing state.

Inputs

- `user`
 - **Control:** Quote requester.
 - **Constraints:** Must be nonzero.
 - **Impact:** Used for yield-bearing quote math on applicable routes.
- `inputs`
 - **Control:** Swap submitter via `swap calldata`.
 - **Constraints:** Must satisfy route and slippage validation.

- **Impact:** Single leg route parameters and slippage hints.

Branches and code coverage

Intended branches

- Validates route envelope and configuration, checks price guardrails, and returns `_calculateSwapQuote`.
 - ☒ Test coverage
- Uses stored yield state rather than calling `accrueInterest` during the view quote.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidSwapAddress` when user is zero.
 - ☒ Negative test
- Reverts on unsupported routes, unaccepted assets, missing prices, or guardrail violations.
 - ☒ Negative test

Function call analysis

- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.
- `IYieldBearingDelegate(inputs.yieldDelegate)`
 - `.getYieldAmount(inputs.user, inputAssetWad, continuous)`
 - **What is controllable?** `inputs.user` and the quoted input amount are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The yield-adjusted WAD amount feeds exact-in pricing; an inflated return understates required raw input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap `nonReentrant` guard.

- `IYieldBearingDelegate(inputs.yieldDelegate)`
- `.getAmountForTargetValue(inputs.user, inputAssetWad, continuous)`
- **What is controllable?** `inputs.user` and the target yield-adjusted value are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The raw input WAD is derived for exact-out routes; a deflated return overstates required input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap `nonReentrant` guard.

Function: `setAutoLiquidityStablecoinID(byte[32] stablecoinID, byte[32] autoLiquidityStablecoinID)`

The `setAutoLiquidityStablecoinID` function configures the yield stablecoin used for automatic LP prefund and sweep legs for one delegate domain.

Inputs

- `stablecoinID`
 - **Control:** Stablecoin delegate caller via `onlyStablecoinDelegate`.
 - **Constraints:** Must match `msg.sender` delegate binding.
 - **Impact:** Selects which delegate domain owns the auto-liquidity config.
- `autoLiquidityStablecoinID`
 - **Control:** Issuer delegate caller.
 - **Constraints:** `bytes32(0)` disables auto legs; nonzero must be accepted.
 - **Impact:** Routes prefund and sweep behavior in swap.

Branches and code coverage

Intended branches

- Validates acceptance for nonzero IDs, writes `autoLiquidityStablecoinIDs`, and emits `AutoLiquidityStablecoinIDSet`.

☑ Test coverage

Negative behavior

- Reverts with `NotStablecoinDelegate` when caller is not the registered delegate.

☑ Negative test

- Reverts with `StablecoinNotAccepted` when a nonzero auto-liquidity ID is not accepted.

☒ Negative test

Function: `setBlacklisted(address user, bool _blacklisted)`

The `setBlacklisted` function updates the global user blacklist checked on every swap call.

Inputs

- `user`
 - **Control:** Blacklist via `onlyRole(BLACKLISTER_ROLE)`.
 - **Constraints:** Any address.
 - **Impact:** Address blocked from acting as user, operator, or receiver in swaps.
- `_blacklisted`
 - **Control:** Blacklist via `onlyRole(BLACKLISTER_ROLE)`.
 - **Constraints:** Arbitrary boolean.
 - **Impact:** `true` blocks the address; `false` removes the block.

Branches and code coverage

Intended branches

- Writes `blacklisted[user]` and emits `UserBlacklistUpdated`.

☒ Test coverage

Negative behavior

- No local validation beyond role gating.

☒ Negative test

Function: `setIssuerPaidProtocolFeeRate(byte[32] stablecoinID, uint256 rate)`

The `setIssuerPaidProtocolFeeRate` function sets the issuer-paid explicit protocol fee rate for one stablecoin.

Inputs

- `stablecoinID`

- **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must satisfy function-specific validation.
 - **Impact:** Selects which stablecoin pays the explicit protocol fee.
- `rate`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must be $\leq$ WAD (100%).
 - **Impact:** Stored in `issuerPaidProtocolFeeRates`.

Branches and code coverage

Intended branches

- Persists the new rate and emits `IssuerPaidProtocolFeeRateSet`.
- ☒ Test coverage

Negative behavior

- Reverts with `InvalidRate` when `rate > WAD`.
- ☒ Negative test

Function: `setMaxSwapLegs(uint256 newMaxSwapLegs)`

The `setMaxSwapLegs` function configures the maximum number of legs accepted by one swap call.

Inputs

- `newMaxSwapLegs`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Zero intentionally disables swap until reconfigured.
 - **Impact:** Stored in `_maxSwapLegs`.

Branches and code coverage

Intended branches

- Updates `_maxSwapLegs` without emitting an event.
- ☒ Test coverage

Negative behavior

- No local validation beyond role gating.

☒ Negative test

Function: `setMultiliquidVault(address vault)`

The `setMultiliquidVault` function updates the protocol-fee vault address passed to delegate settlement hooks.

Inputs

- `vault`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must be nonzero.
 - **Impact:** Becomes `multiliquidVault` for all future delegate fee routing.

Branches and code coverage

Intended branches

- Persists the new vault and emits `MultiliquidVaultSet`.

☒ Test coverage

Negative behavior

- Reverts with `InvalidVault` when `vault` is zero.

☒ Negative test

Function: `setProtocolFeeExempt(byte[32] stablecoinID, bool exempt)`

The `setProtocolFeeExempt` function exempts or unexempts a stablecoin from issuer-paid protocol fees and spread protocol take.

Inputs

- `stablecoinID`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must satisfy function-specific validation.
 - **Impact:** Selects which stablecoin's protocol fee configuration is changed.
- `exempt`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.

- **Constraints:** Arbitrary boolean.
- **Impact:** Sets protocolFeeExempt[stablecoinID].

Branches and code coverage

Intended branches

- Writes protocolFeeExempt and emits ProtocolFeeExemptSet.

☒ Test coverage

Negative behavior

- No local validation beyond role gating.

☒ Negative test

Function: setRWAAcceptance(byte[32] rwaID, address _delegate, address _assetAddress, bool _accepted)

The setRWAAcceptance function registers or removes an RWA asset for protocol swaps and stores optional per-RWA risk delegate metadata.

Inputs

- rwaID
 - **Control:** Operator via onlyRole(OPERATOR_ROLE).
 - **Constraints:** Must satisfy function-specific validation.
 - **Impact:** Primary key for the RWA in rwaInfo.
- _delegate
 - **Control:** Operator via onlyRole(OPERATOR_ROLE).
 - **Constraints:** Zero disables additionalRiskControls; nonzero enables delegate checks.
 - **Impact:** Optional RWA risk delegate.
- _assetAddress
 - **Control:** Operator via onlyRole(OPERATOR_ROLE).
 - **Constraints:** Must be nonzero when accepting.
 - **Impact:** ERC-20 address of the RWA token.
- _accepted
 - **Control:** Operator via onlyRole(OPERATOR_ROLE).
 - **Constraints:** false deletes registry state for the ID.

- **Impact:** Acceptance flag.

Branches and code coverage

Intended branches

- Deletes `rwaInfo[rwaID]` and emits a cleared event when `_accepted` is false.
 - ☑ Test coverage
- Reads token decimals, stores `RWAInfo`, and emits `RWAAcceptanceSet` when accepting.
 - ☑ Test coverage

Negative behavior

- Reverts with `InvalidAssetAddress` when accepting with a zero asset address.
 - ☑ Negative test
- Reverts with `AssetIDAlreadyRegistered` when the ID is already a stablecoin.
 - ☑ Negative test
- Reverts with `UnsupportedDecimals` when decimals exceed 18.
 - ☑ Negative test

Function call analysis

- `IERC20Metadata(_assetAddress).decimals()`
 - **What is controllable?** Operator supplies `_assetAddress` when accepting an asset.
 - **If the return value is controllable, how is it used and how can it go wrong?** Decimals are stored in `rwaInfo / stablecoinInfo` and bound all WAD conversions for the asset.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts acceptance; nonstandard metadata tokens may return unexpected values.

Function: `setRWADiscountRate(byte[32] stablecoinID, byte[32] rwaID, uint256 rate)`

The `setRWADiscountRate` function stores an issuer-configured RWA discount rate for a stablecoin/RWA pair.

Inputs

- `stablecoinID`
 - **Control:** Stablecoin delegate caller via `onlyStablecoinDelegate`.
 - **Constraints:** Must match `msg.sender` delegate binding.
 - **Impact:** Scopes the discount to one issuer stablecoin domain.
- `rwaID`
 - **Control:** Issuer delegate caller.
 - **Constraints:** Arbitrary accepted RWA key.
 - **Impact:** Selects which inbound RWA receives the discount.
- `rate`
 - **Control:** Issuer delegate caller.
 - **Constraints:** Must be $< \text{WAD}$.
 - **Impact:** Adjusts RWA-to-stable and RWA-to-RWA pricing.

Branches and code coverage

Intended branches

- Validates WAD bounds, writes `discountRates`, and emits `DiscountRateSet`.
- ☒ Test coverage

Negative behavior

- Reverts with `NotStablecoinDelegate` or `InvalidRate`.
- ☒ Negative test

Function: `setRWARedemptionFee(byte[32] stablecoinID, byte[32] rwaID, uint256 rate)`

The `setRWARedemptionFee` function stores an issuer-configured RWA redemption fee for a stablecoin/RWA pair.

Inputs

- `stablecoinID`
 - **Control:** Stablecoin delegate caller via `onlyStablecoinDelegate`.
 - **Constraints:** Must match `msg.sender` delegate binding.
 - **Impact:** Scopes the fee to one issuer stablecoin domain.
- `rwaID`

- **Control:** Issuer delegate caller.
 - **Constraints:** Arbitrary accepted RWA key.
 - **Impact:** Selects which outbound RWA pays the redemption fee.
- rate
 - **Control:** Issuer delegate caller.
 - **Constraints:** Must be < WAD.
 - **Impact:** Adjusts stable-to-RWA and RWA-to-RWA pricing.

Branches and code coverage

Intended branches

- Validates WAD bounds, writes redemptionFees, and emits RedemptionFeeSet.
- ☒ Test coverage

Negative behavior

- Reverts with NotStablecoinDelegate or InvalidRate.
- ☒ Negative test

Function: setRwaPriceAdapter(byte[32] rwaID, address priceAdapter)

The setRwaPriceAdapter function binds an RWA asset ID to a price adapter used for quoting and acceptance checks.

Inputs

- rwaID
 - **Control:** Operator via onlyRole(OPERATOR_ROLE).
 - **Constraints:** Must satisfy function-specific validation.
 - **Impact:** Selects the RWA whose adapter mapping is updated.
- priceAdapter
 - **Control:** Operator via onlyRole(OPERATOR_ROLE).
 - **Constraints:** May be zero to clear pricing for the asset.
 - **Impact:** Stored in rwaPriceAdapters[rwaID].

Branches and code coverage

Intended branches

- Writes `rwaPriceAdapters` and emits `RwaPriceAdapterSet`.

☒ Test coverage

Negative behavior

- No local validation beyond role gating.

☒ Negative test

Function: `setRwaWhitelistAdapter(byte[32] rwaID, address adapter)`

The `setRwaWhitelistAdapter` function configures an optional whitelist adapter for RWA recipient checks on outbound RWA legs.

Inputs

- `rwaID`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must satisfy function-specific validation.
 - **Impact:** Selects the RWA whose whitelist adapter is updated.
- `adapter`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must have code when nonzero; zero disables whitelist enforcement.
 - **Impact:** Stored in `rwaWhitelistAdapters[rwaID]`.

Branches and code coverage

Intended branches

- Writes the adapter mapping and emits `RwaWhitelistAdapterSet`.

☒ Test coverage

Negative behavior

- Reverts with `InvalidDelegate` when `adapter` is nonzero but has no code.

☒ Negative test

Function: `setSpreadProtocolTakeRate(uint256 rate)`

The `setSpreadProtocolTakeRate` function sets the protocol's take rate on issuer spread between configured leg fees.

Inputs

- `rate`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must be $\leq$ WAD (100%).
 - **Impact:** Stored in `spreadProtocolTakeRate`.

Branches and code coverage

Intended branches

- Persists the new global spread take rate and emits `SpreadProtocolTakeRateSet`.
- ☒ Test coverage

Negative behavior

- Reverts with `InvalidRate` when `rate > WAD`.
- ☒ Negative test

Function: `setStablecoinAcceptance(byte[32] stablecoinID, address _delegate, address _assetAddress, bool _accepted, bool _yieldBearing)`

The `setStablecoinAcceptance` function registers or removes a stablecoin for protocol swaps, including optional delegate and yield-bearing metadata.

Inputs

- `stablecoinID`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must satisfy function-specific validation.
 - **Impact:** Primary key for the stablecoin in `stablecoinInfo`.
- `_delegate`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Zero allowed for swap-only stablecoins; required when `_yieldBearing` is true.

- **Impact:** Stablecoin delegate for RWA-capable routes.
- `_assetAddress`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must be nonzero when accepting.
 - **Impact:** ERC-20 address of the stablecoin.
- `_accepted`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** `false` deletes registry state for the ID.
 - **Impact:** Acceptance flag.
- `_yieldBearing`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Cannot be true with a zero delegate.
 - **Impact:** Marks yield-bearing delegate pricing behavior.

Branches and code coverage

Intended branches

- Deletes `stablecoinInfo[stablecoinID]` and emits a cleared event when `_accepted` is `false`.
 - ☒ Test coverage
- Reads token decimals, stores `StablecoinInfo`, and emits `StablecoinAcceptanceSet` when accepting.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidAssetAddress` when accepting with a zero asset address.
 - ☒ Negative test
- Reverts with `InvalidDelegate` when `_yieldBearing` is `true` but `_delegate` is zero.
 - ☒ Negative test
- Reverts with `AssetIDAlreadyRegistered` when the ID is already an RWA.
 - ☒ Negative test
- Reverts with `UnsupportedDecimals` when decimals exceed 18.
 - ☒ Negative test

Function call analysis

- `IERC20Metadata(_assetAddress).decimals()`
 - **What is controllable?** Operator supplies `_assetAddress` when accepting an asset.
 - **If the return value is controllable, how is it used and how can it go wrong?** Decimals are stored in `rwaInfo / stablecoinInfo` and bound all WAD conversions for the asset.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts acceptance; nonstandard metadata tokens may return unexpected values.

Function: `setStablecoinAcceptanceFee(byte[32] stablecoinOutID, address stablecoin, uint256 rate)`

The `setStablecoinAcceptanceFee` function stores an issuer-configured acceptance fee for inbound stablecoin backing assets.

Inputs

- `stablecoinOutID`
 - **Control:** Stablecoin delegate caller via `onlyStablecoinDelegate`.
 - **Constraints:** Must match `msg.sender` delegate binding.
 - **Impact:** Selects the delegate-owned fee schedule.
- `stablecoin`
 - **Control:** Issuer delegate caller.
 - **Constraints:** Counterparty stablecoin ERC-20 address.
 - **Impact:** Selects which inbound stablecoin pays the acceptance fee.
- `rate`
 - **Control:** Issuer delegate caller.
 - **Constraints:** Must be $< \text{WAD}$.
 - **Impact:** Adjusts stablecoin-to-stablecoin pricing.

Branches and code coverage

Intended branches

- Validates WAD bounds, writes `stablecoinAcceptanceFees`, and emits `StablecoinAcceptanceFeeSet`.

☒ Test coverage

Negative behavior

- Reverts with `NotStablecoinDelegate` or `InvalidRate`.

☒ Negative test

Function: `setStablecoinRedemptionFee(byte[32] stablecoinOutID, address stablecoin, uint256 rate)`

The `setStablecoinRedemptionFee` function stores an issuer-configured redemption fee for outbound stablecoin backing assets.

Inputs

- `stablecoinOutID`
 - **Control:** Stablecoin delegate caller via `onlyStablecoinDelegate`.
 - **Constraints:** Must match `msg.sender` delegate binding.
 - **Impact:** Selects the delegate-owned fee schedule.
- `stablecoin`
 - **Control:** Issuer delegate caller.
 - **Constraints:** Counterparty stablecoin ERC-20 address.
 - **Impact:** Selects which outbound stablecoin pays the redemption fee.
- `rate`
 - **Control:** Issuer delegate caller.
 - **Constraints:** Must be $< WAD$.
 - **Impact:** Adjusts stablecoin-to-stablecoin pricing.

Branches and code coverage

Intended branches

- Validates `WAD` bounds, writes `stablecoinRedemptionFees`, and emits `StablecoinRedemptionFeeSet`.

☒ Test coverage

Negative behavior

- Reverts with `NotStablecoinDelegate` or `InvalidRate`.

☒ Negative test

Function: `setStablecoinUSDValue(byte[32] stablecoinID, uint256 value)`

The `setStablecoinUSDValue` function sets the admin USD peg used for stablecoin quoting and guardrail comparison.

Inputs

- `stablecoinID`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** Must satisfy function-specific validation.
 - **Impact:** Selects the stablecoin whose admin USD value is updated.
- `value`
 - **Control:** Operator via `onlyRole(OPERATOR_ROLE)`.
 - **Constraints:** WAD-scaled USD value.
 - **Impact:** Stored in `stablecoinUSDValue[stablecoinID]`.

Branches and code coverage

Intended branches

- Writes `stablecoinUSDValue` and emits `StablecoinUSDValueSet`.
- ☒ Test coverage

Negative behavior

- No local validation beyond role gating.
- ☒ Negative test

Function: `swap(address user, address receiver, SwapInputs[] inputs, ApprovalPermit permit)`

The `swap` function is the multileg atomic swap entry point; it validates identities, optionally verifies an EIP-712 batch permit, and executes each requested leg plus any derived autoliquidity prefund and sweep legs.

Inputs

- `user`
 - **Control:** Swap submitter via `swap calldata`.
 - **Constraints:** Must be nonzero and not globally blacklisted.

- **Impact:** Economic account whose assets and standing approvals fund every leg.
- receiver
 - **Control:** Swap submitter via swap calldata.
 - **Constraints:** Must be nonzero and not globally blacklisted.
 - **Impact:** Account that receives output assets for every leg.
- inputs
 - **Control:** Swap submitter via swap calldata.
 - **Constraints:** Length must be between 1 and `_maxSwapLegs`.
 - **Impact:** Ordered swap legs with route IDs, asset IDs, delegate ID, and slippage bounds.
- permit
 - **Control:** Swap submitter via swap calldata.
 - **Constraints:** Nonempty signature requires `user != operator` and a valid signed digest.
 - **Impact:** Optional EIP-712 authorization for delegated batch settlement.

Branches and code coverage

Intended branches

- Rejects zero user or receiver and enforces global blacklist on user, operator, and receiver.
 - ☒ Test coverage
- Verifies an optional batch permit before executing any leg.
 - ☒ Test coverage
- Iterates each submitted leg through `_executeRequestedLegWithAutoLiquidity`.
 - ☒ Test coverage

Negative behavior

- Reverts with `InvalidSwapAddress` for zero addresses.
 - ☒ Negative test
- Reverts with `InvalidLegCount` when no legs are submitted or the batch exceeds `_maxSwapLegs`.
 - ☒ Negative test
- Reverts with `UserBlacklisted` when any batch identity is blacklisted.
 - ☒ Negative test
- Reverts with `InvalidSignature` when a permit is supplied but `user == operator`.

- ☒ Negative test
- Reverts when permit verification, route validation, pricing guardrails, or delegate settlement fails.
- ☐ Negative test

Function call analysis

- `IPriceAdapter(priceAdapter).getPrice()`
 - **What is controllable?** The configured price adapter controls the returned WAD price.
 - **If the return value is controllable, how is it used and how can it go wrong?** The oracle price is compared to the admin USD peg for guardrails or used in route-unit conversion for quotes.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or stale/malformed price aborts the swap or quote before settlement.
- `IStablecoinDelegate(stablecoinInfo[stablecoinDelegateID].delegate)`
`.stablecoinCustodyAddress()`
 - **What is controllable?** The stablecoin delegate address is operator-configured at acceptance time.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned LP vault address gates prefund, sweep, and autoliquidity legs; zero or blacklisted vaults revert.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert or invalid custody address aborts autoliquidity; delegate view calls cannot reenter.
- `IYieldBearingDelegate(stablecoin.delegate).accrueInterest(user)`
 - **What is controllable?** user is the swap account whose yield state is synchronized before pricing.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed; accrual updates delegate state used by subsequent yield math.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the entire swap leg; malicious delegates could grief settlement.
- `IYieldBearingDelegate(inputs.yieldDelegate)`
`.getYieldAmount(inputs.user, inputAssetWad, continuous)`
 - **What is controllable?** inputs.user and the quoted input amount are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The yield-adjusted WAD amount feeds exact-in pricing; an inflated return

understates required raw input.

- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap nonReentrant guard.
- `IYieldBearingDelegate(inputs.yieldDelegate)`
`.getAmountForTargetValue(inputs.user, inputAssetWad, continuous)`
- **What is controllable?** `inputs.user` and the target yield-adjusted value are caller-controlled; the yield delegate is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The raw input WAD is derived for exact-out routes; a deflated return overstates required input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts quoting; delegate reentrancy is blocked by the swap nonReentrant guard.
- `IStablecoinDelegate(stablecoinDelegate).balanceSheetCustodyAddress()`
 - **What is controllable?** The stablecoin delegate is route-configured via `stablecoinDelegateID`.
 - **If the return value is controllable, how is it used and how can it go wrong?** The custody address is passed to the RWA whitelist adapter as the transfer origin.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the swap leg before delegate settlement.
- `IWhitelistAdapter(adapter).isWhitelisted(rwaCustodyAddress, user, amount)`
 - **What is controllable?** The adapter address is operator-configured; user and amount come from the swap leg.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `UserNotWhitelisted`; a permissive adapter allows any receiver.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; adapter hooks could reenter, but nonReentrant guards the swap entry point.
- `IRWADelegate(rwa.delegate).checkRWAIn(user, amount)`
 - **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValityCheckFailed` and blocks inbound RWA risk.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but nonReentrant guards the swap entry point.
- `IRWADelegate(rwa.delegate).checkRWAOut(user, amount)`

- **What is controllable?** user and amount are derived from the resolved swap leg when `additionalRiskControls` is true.
- **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `RWAValityCheckFailed` and blocks outbound RWA risk.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts settlement; delegate hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(stablecoinDelegate)`
`.receiveStablecoin(DelegateParams(...))`
- **What is controllable?** Swap caller supplies user, receiver, and amounts; delegate and asset addresses are protocol-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployRWA`; the delegate performs ERC-20 transfers and RWA delivery.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(stablecoinDelegate)`
`.deployStablecoin(DelegateParams(...))`
- **What is controllable?** Swap caller supplies user, receiver, and amounts; delegate and asset addresses are protocol-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployStablecoin`; the delegate pulls RWA and mints/transfers stablecoin.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(stablecoinDelegate)`
`.exchangeRWAs(DelegateParams(...))`
- **What is controllable?** Swap caller supplies user, receiver, and amounts; RWA addresses come from accepted asset registry.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployRWA`; the delegate atomically exchanges both RWA legs.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IStablecoinDelegate(delegateStablecoin)`
`.exchangeStablecoins(DelegateParams(...))`

- **What is controllable?** Swap caller supplies user, receiver, and amounts; stablecoin addresses come from accepted asset registry.
 - **If the return value is controllable, how is it used and how can it go wrong?** A false return reverts with `FailedToDeployStablecoin`; the delegate atomically exchanges both stablecoin legs.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the leg; delegate token hooks could reenter, but `nonReentrant` guards the swap entry point.
- `IERC20Metadata(stablecoinInfo[assetInID].assetAddress)`
`.balanceOf(lpVault)'`
- **What is controllable?** The LP vault address comes from the stablecoin delegate; the token address is operator-configured.
 - **If the return value is controllable, how is it used and how can it go wrong?** The balance delta sizes optional autosweep legs; a manipulated balance could skew sweep input.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts sweep derivation; fee-on-transfer tokens may report unexpected balances.

Function: `unpause()`

The `unpause` function restores `whenNotPaused` entry points for operators.

Branches and code coverage

Intended branches

- Clears the Pausable flag via `_unpause()` when the caller has `OPERATOR_ROLE`.

☒ Test coverage

Negative behavior

- Reverts with an OpenZeppelin access-control error when the caller lacks `OPERATOR_ROLE`.

☒ Negative test

4.13. Module: `RWADelegate.sol`

Function: `checkRWAIn(address, uint256)`

The `checkRWAIn` function is the inbound RWA risk hook called by `MultiliquidSwap` before stablecoin is deployed on RWA-in swap legs. The base implementation is a no-op that always returns `true`.

Inputs

- `<unnamed@0>` (to)
 - **Control:** Set by `MultiliquidSwap` as the swap receiver on RWA-in paths.
 - **Constraints:** Ignored in the base implementation; overrides may enforce per-recipient policy.
 - **Impact:** In concrete delegates, identifies which user bucket is charged for inbound volume.
- `<unnamed@1>` (amount)
 - **Control:** Set by `MultiliquidSwap` from the computed RWA output amount.
 - **Constraints:** Ignored in the base implementation; overrides may cap or reject amounts.
 - **Impact:** Magnitude checked by protocol-specific risk controls when overridden.

Branches and code coverage

Intended branches

- Returns `true` when the caller holds `MULTILIQUID_SWAP_CONTRACT`, the contract is unpaused, and the reentrancy guard passes.

☒ Test coverage

Negative behavior

- Reverts if the caller lacks `MULTILIQUID_SWAP_CONTRACT`.
- Reverts when the delegate is paused via `whenNotPaused`.

☒ Negative test

- Reverts on reentrant entry via `nonReentrant`.

☐ Negative test

Function: checkRWAOut(address, uint256)

The checkRWAOut function is the outbound RWA risk hook called by MultiliquidSwap before RWA is released on RWA-out swap legs. The base implementation is a no-op that always returns true.

Inputs

- <unnamed@0> (from)
 - **Control:** Set by MultiliquidSwap as the swap payer on RWA-out paths.
 - **Constraints:** Ignored in the base implementation; overrides may enforce per-sender policy.
 - **Impact:** In concrete delegates, identifies which user bucket is charged for outbound volume.
- <unnamed@1> (amount)
 - **Control:** Set by MultiliquidSwap from the computed RWA input amount.
 - **Constraints:** Ignored in the base implementation; overrides may cap or reject amounts.
 - **Impact:** Magnitude checked by protocol-specific risk controls when overridden.

Branches and code coverage

Intended branches

- Returns true when the caller holds MULTILQUID_SWAP_CONTRACT, the contract is unpaused, and the reentrancy guard passes.

☒ Test coverage

Negative behavior

- Reverts if the caller lacks MULTILQUID_SWAP_CONTRACT.
- Reverts when the delegate is paused via whenNotPaused.
- Reverts on reentrant entry via nonReentrant.

☒ Negative test

☒ Negative test

☐ Negative test

Function: constructor()

The constructor function disables initializers on the RWADelegate implementation so the logic contract cannot be initialized or used outside the UUPS proxy.

Branches and code coverage

Intended branches

- Calls `_disableInitializers()` to permanently lock direct initialization of the implementation.

☒ Test coverage

Negative behavior

- N/A for runtime callers — the implementation cannot be initialized through this path after deployment.

☐ Negative test

Function: `initialize(address multiliquidAdmin, address initialIssuerAdmin, address _multiliquidSwap, byte[32] _rwaID, address _rwaAddress)`

The `initialize` function performs one-time proxy setup: validates admin and router addresses; initializes access control, pause, reentrancy, and UUPS modules; records RWA metadata; seeds issuer admin; and grants MultiliquidSwap the sole risk-hook caller role.

Inputs

- `multiliquidAdmin`
 - **Control:** Set by the deployer during proxy initialization.
 - **Constraints:** Must be nonzero and distinct from `initialIssuerAdmin` and `_multiliquidSwap`.
 - **Impact:** Receives `DEFAULT_ADMIN_ROLE` and `PAUSE_ROLE` for upgrades and Multiliquid pause control.
- `initialIssuerAdmin`
 - **Control:** Set by the deployer during proxy initialization.
 - **Constraints:** Must be nonzero and distinct from `multiliquidAdmin`.
 - **Impact:** Becomes the first issuer admin who can pause and manage issuer-side policy.
- `_multiliquidSwap`
 - **Control:** Set by the deployer during proxy initialization.
 - **Constraints:** Must be nonzero and distinct from `multiliquidAdmin`.
 - **Impact:** Stored as `multiliquidSwap` and granted `MULTILIQUID_SWAP_CONTRACT` for `checkRWAIn` / `checkRWAOut`.
- `_rwaID`
 - **Control:** Set by the deployer during proxy initialization.

- **Constraints:** Must not be bytes32(0).
- **Impact:** Immutable identifier linking this delegate to one RWA in Multiliquid.
- `_rwaAddress`
 - **Control:** Set by the deployer during proxy initialization.
 - **Constraints:** Must be nonzero.
 - **Impact:** Stored reference to the underlying RWA token contract.

Branches and code coverage

Intended branches

- Initializes OZ modules, issuer admin, and immutable RWA fields and grants admin, pause, and swap roles on valid distinct inputs.

☒ Test coverage

Negative behavior

- Reverts InvalidInitializationParams if any address is zero or `_rwaID` is zero.
 - ☒ Negative test
- Reverts InvalidIssuerAdmin if `multiliquidAdmin` equals `initialIssuerAdmin`.
 - ☒ Negative test
- Reverts InvalidMultiliquidSwap if `multiliquidAdmin` equals `_multiliquidSwap`.
 - ☒ Negative test
- Reverts if called outside the initializer context or when already initialized.
 - ☒ Negative test

Function call analysis

- `this.__ReentrancyGuard_init() -> this.__ReentrancyGuard_init_unchained()`
`-> this._getReentrancyGuardStorage()`
 - **What is controllable?** Nothing at runtime; internal OZ initializer only.
 - **If the return value is controllable, how is it used and how can it go wrong?**
 No return value is consumed.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization before roles or RWA state are committed.
- `this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> this._getAccessControlEnumerableStorage()`
 - **What is controllable?** `multiliquidAdmin` is chosen at init by the deployer.
 - **If the return value is controllable, how is it used and how can it go wrong?**

No return value is consumed. A malicious admin address gains upgrade and role-admin powers.

- **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the entire initialization transaction.
- `this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> super._grantRole(role, account) -> this._getAccessControlStorage()`
 - **What is controllable?** Role and account are fixed at initialization.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> super._grantRole(role, account) -> this.hasRole(role, account) -> this._getAccessControlStorage()`
 - **What is controllable?** Internal membership check only — account is deployer-supplied.
 - **If the return value is controllable, how is it used and how can it go wrong?** Boolean result gates duplicate grants — no user-facing impact.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> EnumerableSet.add($_roleMembers[role], account)`
 - **What is controllable?** multiliquidAdmin address at init.
 - **If the return value is controllable, how is it used and how can it go wrong?** Set-add success is ignored when membership already exists.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization; no external callbacks occur.

Function: pause ()

The pause function halts delegate risk hooks and other whenNotPaused functionality. Only issuer admins may call it.

Branches and code coverage

Intended branches

- Sets the paused flag via `_pause ( )` when the caller is an issuer admin.

☒ Test coverage

Negative behavior

- Reverts NotIssuerAdmin if the caller is not in issuerAdmins.

☒ Negative test

Function call analysis

- `this._pause()` -> `this._getPausableStorage()`
 - **What is controllable?** Only issuer admins reach this call.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Pausing blocks `checkRWAIn` / `checkRWAOut` and any other `whenNotPaused` entry points.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert leaves the contract unpaused. No external callbacks occur.

Function: `pauseMultiliquid()`

The `pauseMultiliquid` function lets the Multiliquid admin pause this delegate without issuer-admin rights.

Branches and code coverage

Intended branches

- Sets the paused flag via `_pause()` when the caller holds `PAUSE_ROLE`.

☒ Test coverage

Negative behavior

- Reverts if the caller lacks `PAUSE_ROLE`.

☒ Negative test

Function call analysis

- `this._pause()` -> `this._getPausableStorage()`
 - **What is controllable?** Only `PAUSE_ROLE` holders (typically `multiliquidAdmin`) reach this call.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Pausing blocks router risk-hook calls for this RWA.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert leaves the contract unpaused. No external callbacks occur.

Function: `unpause()`

The `unpause` function restores delegate operation after an issuer-initiated pause.

Branches and code coverage

Intended branches

- Clears the paused flag via `_unpause()` when the caller is an issuer admin.

☒ Test coverage

Negative behavior

- Reverts `NotIssuerAdmin` if the caller is not in `issuerAdmins`.

☒ Negative test

Function call analysis

- `this._unpause()` -> `this._getPausableStorage()`
 - **What is controllable?** Only issuer admins reach this call.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Unpausing reenables risk hooks.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert leaves the contract paused. No external callbacks occur.

Function: `unpauseMultiliquid()`

The `unpauseMultiliquid` function lets the Multiliquid admin restore delegate operation after a Multiliquid-initiated pause.

Branches and code coverage

Intended branches

- Clears the paused flag via `_unpause()` when the caller holds `PAUSE_ROLE`.

☒ Test coverage

Negative behavior

- Reverts if the caller lacks `PAUSE_ROLE`.

☒ Negative test

Function call analysis

- `this._unpause()` -> `this._getPausableStorage()`
 - **What is controllable?** Only PAUSE_ROLE holders reach this call.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Unpausing reenables router risk hooks.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert leaves the contract paused. No external callbacks occur.

4.14. Module: StablecoinDelegateBase.sol

Function: `__stablecoinDelegateBase_init(address multiliquidAdmin, address initialIssuerAdmin, address _multiliquidSwap, byte[32] _stablecoinID, address _stablecoinAddress)`

The `__stablecoinDelegateBase_init` function wires core delegate state, separates Multiliquid upgrade/pause authority from issuer-admin custody control, and grants MULTILIQID_SWAP_CONTRACT to the swap router.

Inputs

- `multiliquidAdmin`
 - **Control:** Deployer via the proxy initializer.
 - **Constraints:** Must be nonzero, distinct from `initialIssuerAdmin` and `_multiliquidSwap`.
 - **Impact:** Receives DEFAULT_ADMIN_ROLE and PAUSE_ROLE for UUPS upgrades and Multiliquid-scoped pause only.
- `initialIssuerAdmin`
 - **Control:** Deployer via the proxy initializer.
 - **Constraints:** Must be nonzero, distinct from `multiliquidAdmin` and `_multiliquidSwap`.
 - **Impact:** Becomes the first issuer admin who controls mint/burn, custody, whitelist, fees, and issuer pause.
- `_multiliquidSwap`
 - **Control:** Deployer via the proxy initializer.
 - **Constraints:** Must be nonzero and distinct from both admin addresses.
 - **Impact:** Stored as `multiliquidSwap` and granted MULTILIQID_SWAP_CONTRACT for swap callbacks.
- `_stablecoinID`
 - **Control:** Deployer via the proxy initializer.

- **Constraints:** Must be nonzero bytes32.
 - **Impact:** Keys this delegate in MultiliquidSwap fee and routing configuration.
- `_stablecoinAddress`
 - **Control:** Deployer via the proxy initializer.
 - **Constraints:** Must be nonzero.
 - **Impact:** Binds the delegate to one issuer stablecoin ERC-20 for withdrawals and exchange validation.

Branches and code coverage

Intended branches

- Initializes AccessControl, Pausable, ReentrancyGuard, UUPS, and issuer-admin modules.
 - ☒ Test coverage
- Persists multiliquidSwap, stablecoinID, and stablecoinAddress.
 - ☒ Test coverage
- Grants DEFAULT_ADMIN_ROLE, PAUSE_ROLE, and MULTILIQUID_SWAP_CONTRACT.
 - ☒ Test coverage
- Runs only during the initializer path via onlyInitializing.
 - ☒ Test coverage

Negative behavior

- Reverts with InvalidInitializationParams when any required address or `_stablecoinID` is zero.
 - ☒ Negative test
- Reverts with InvalidIssuerAdmin when multiliquidAdmin equals initialIssuerAdmin.
 - ☒ Negative test
- Reverts with InvalidMultiliquidSwap when `_multiliquidSwap` equals either admin address.
 - ☒ Negative test

Function: `_checkBlacklist(address account)`

The `_checkBlacklist` internal guard blocks swap participants flagged in this delegate's issuer-managed blacklist.

Inputs

- `account`
 - **Control:** Swap caller via `child delegate params.user` or `params.receiver`.
 - **Constraints:** Must not be true in `blacklist`.
 - **Impact:** Prevents blacklisted users or receivers from executing issuer-gated swaps.

Branches and code coverage

Intended branches

- Allows execution when `blacklist[account]` is false.
 - ☒ Test coverage
- Complements but does not replace MultiliquidSwap's global blacklist.
 - ☒ Test coverage

Negative behavior

- Reverts with `AddressBlacklisted(account)` when the account is blacklisted.
 - ☒ Negative test

Function: `_checkIssuerStablecoinInExchange(address assetIn, address assetOut)`

The `_checkIssuerStablecoinInExchange` internal guard ensures at least one swap leg involves this delegate's bound `stablecoinAddress`.

Inputs

- `assetIn`
 - **Control:** Swap caller via `child delegate params.assetInAddress`.
 - **Constraints:** Must equal `stablecoinAddress` unless `assetOut` does.
 - **Impact:** Validates the inbound leg for stablecoin exchange routes.
- `assetOut`
 - **Control:** Swap caller via `child delegate params.assetOutAddress`.
 - **Constraints:** Must equal `stablecoinAddress` unless `assetIn` does.
 - **Impact:** Validates the outbound leg for stablecoin exchange routes.

Branches and code coverage

Intended branches

- Allows execution when `assetIn == stablecoinAddress` or `assetOut == stablecoinAddress`.

☒ Test coverage

Negative behavior

- Reverts with `InvalidStablecoinExchange(assetIn, assetOut, stablecoinAddress)` when neither leg is the issuer stablecoin.

☒ Negative test

Function: `_checkRate(uint256 rate)`

The `_checkRate` internal guard enforces that fee and discount rates stay below 100% (1e18 fixed-point).

Inputs

- `rate`
 - **Control:** Issuer admin via fee-setting externals that call this helper.
 - **Constraints:** Must be strictly less than `MAX_RATE_EXCLUSIVE` (1e18).
 - **Impact:** Bounds discount and fee parameters forwarded to `MultiliquidSwap`.

Branches and code coverage

Intended branches

- Allows execution when `rate < MAX_RATE_EXCLUSIVE`.

☒ Test coverage

- Accepts zero as a valid rate.

☒ Test coverage

Negative behavior

- Reverts with `InvalidRate` when `rate >= MAX_RATE_EXCLUSIVE`.

☒ Negative test

Function: `_custodyAddressesSet()`

The `_custodyAddressesSet` internal guard requires both hot-wallet custody addresses to be configured before swaps run.

Branches and code coverage

Intended branches

- Allows execution when `balanceSheetCustodyAddress` and `stablecoinCustodyAddress` are both nonzero.
 - ☒ Test coverage
- Used by the `custodyAddressesSet` modifier on child swap entry points.
 - ☒ Test coverage

Negative behavior

- Reverts with `SetRWACustodyAddress` when `balanceSheetCustodyAddress` is zero.
 - ☒ Negative test
- Reverts with `SetStablecoinCustodyAddress` when `stablecoinCustodyAddress` is zero.
 - ☒ Negative test

Function: `_onlyWhitelistedRWA(address rwa)`

The `_onlyWhitelistedRWA` internal guard requires the RWA token address to be marked accepted in `rwaWhitelist`.

Inputs

- `rwa`
 - **Control:** Swap caller via child delegate params (asset in/out address).
 - **Constraints:** Must be `true` in `rwaWhitelist`.
 - **Impact:** Determines whether RWA legs in mint, burn, or RWA-to-RWA swaps proceed.

Branches and code coverage

Intended branches

- Allows execution when `rwaWhitelist[rwa]` is `true`.

☒ Test coverage

Negative behavior

- Reverts with `RWANotWhitelisted` when the RWA is not whitelisted.

☒ Negative test

Function: `_onlyWhitelistedStablecoin(address stablecoin)`

The `_onlyWhitelistedStablecoin` internal guard requires the stablecoin address to be marked accepted in `stablecoinWhitelist`.

Inputs

- `stablecoin`
 - **Control:** Swap caller via child delegate params (asset in/out address).
 - **Constraints:** Must be true in `stablecoinWhitelist`.
 - **Impact:** Determines whether cross-stablecoin exchange legs proceed.

Branches and code coverage

Intended branches

- Allows execution when `stablecoinWhitelist[stablecoin]` is true.

☒ Test coverage

Negative behavior

- Reverts with `StablecoinNotWhitelisted(stablecoin)` when the stablecoin is not whitelisted.

☒ Negative test

Function: `addRWAColdStorageAddress(address coldStorageAddress)`

The `addRWAColdStorageAddress` function records an RWA cold-storage address for transparency; it is not used in swap execution.

Inputs

- `coldStorageAddress`
 - **Control:** Issuer admin via `onlyIssuerAdmin`.

- **Constraints:** Must be nonzero and must not be `address(this)`.
- **Impact:** Appended to `_rwaColdStorageAddresses` and exposed via `getRWAColdStorageAddresses`.

Branches and code coverage

Intended branches

- Adds a new address to the set and emits `RWAColdStorageAddressAdded` when insertion succeeds.
 - ☒ Test coverage
- Returns without event when the address is already present.
 - ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts with `AddressNotAllowed` when the address is zero or the delegate itself.
 - ☒ Negative test

Function call analysis

- `EnumerableSet.add(this._rwaColdStorageAddresses, coldStorageAddress)`
 - **What is controllable?** `coldStorageAddress` is issuer-admin controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** The boolean return gates event emission only. Duplicate adds are harmless no-ops.
 - **What happens if it reverts, reenters or does other unusual control flow?** `EnumerableSet` is internal library code with no external calls; it cannot reenter.

Function: `addStablecoinColdStorageAddress(address coldStorageAddress)`

The `addStablecoinColdStorageAddress` function records a stablecoin reserve cold-storage address for transparency; it is not used in swap execution.

Inputs

- `coldStorageAddress`

- **Control:** Issuer admin via `onlyIssuerAdmin`.
- **Constraints:** Must be nonzero and must not be `address(this)`.
- **Impact:** Appended to `_stablecoinColdStorageAddresses` and exposed via `getStablecoinColdStorageAddresses`.

Branches and code coverage

Intended branches

- Adds a new address to the set and emits `StablecoinColdStorageAddressAdded` when insertion succeeds.
 - ☒ Test coverage
- Returns without event when the address is already present.
 - ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts with `AddressNotAllowed` when the address is zero or the delegate itself.
 - ☒ Negative test

Function call analysis

- `EnumerableSet.add(this._stablecoinColdStorageAddresses, coldStorageAddress)`
 - **What is controllable?** `coldStorageAddress` is issuer-admin controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** The boolean return gates event emission only. Duplicate adds are harmless no-ops.
 - **What happens if it reverts, reenters or does other unusual control flow?** `EnumerableSet` is internal library code with no external calls; it cannot reenter.

Function: constructor ()

The constructor function disables initializers on the implementation contract so the logic contract cannot be initialized outside the proxy deployment path.

Branches and code coverage

Intended branches

- Calls `_disableInitializers()` so only the proxy initializer can run `__stablecoinDelegateBase_init`.

☒ Test coverage

Negative behavior

- No revert paths — the constructor always succeeds.

☐ Negative test

Function: `getRWAColdStorageAddresses()`

The `getRWAColdStorageAddresses` view returns every RWA cold-storage address recorded for transparency.

Branches and code coverage

Intended branches

- Returns `_rwaColdStorageAddresses.values()` as a memory array.

☒ Test coverage

- Callable by any address — no access control on the view.

☒ Test coverage

Negative behavior

- No revert paths — returns an empty array when none are registered.

☐ Negative test

Function: `getStablecoinColdStorageAddresses()`

The `getStablecoinColdStorageAddresses` view returns every stablecoin cold-storage address recorded for transparency.

Branches and code coverage

Intended branches

- Returns `_stablecoinColdStorageAddresses.values()` as a memory array.

- ☒ Test coverage
- Callable by any address — no access control on the view.

☒ Test coverage

Negative behavior

- No revert paths — returns an empty array when none are registered.

☐ Negative test

Function: pause ()

The pause function lets an issuer admin halt delegate swap entry points guarded by whenNotPaused.

Branches and code coverage

Intended branches

- Sets the Pausable flag via _pause (), blocking child swap functions that use whenNotPaused.

☒ Test coverage

- Separate from pauseMultiliquid, which is limited to PAUSE_ROLE holders.

☒ Test coverage

Negative behavior

- Reverts with NotIssuerAdmin when the caller is not an issuer admin.

☒ Negative test

- Reverts with EnforcedPause when the contract is already paused.

☐ Negative test

Function call analysis

- this._pause() -> this._getPausableStorage()
 - **What is controllable?** Only issuer admins may invoke this path.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. The paused flag is a single storage slot shared with pauseMultiliquid.
 - **What happens if it reverts, reenters or does other unusual control flow?** OpenZeppelin _pause only flips local storage and emits Paused; it makes no external calls and cannot reenter.

Function: pauseMultiliquid()

The pauseMultiliquid function lets Multiliquid's PAUSE_ROLE holder pause the delegate without issuer-admin authority.

Branches and code coverage

Intended branches

- Sets the shared Pausable flag via _pause() when the caller holds PAUSE_ROLE.
 - ☒ Test coverage
- The PAUSE_ROLE is granted to multiliquidAdmin at initialization, not to issuer admins.
 - ☒ Test coverage

Negative behavior

- Reverts with OpenZeppelin access-control error when the caller lacks PAUSE_ROLE.
 - ☒ Negative test
- Reverts with EnforcedPause when the contract is already paused.
 - ☐ Negative test

Function call analysis

- this._pause() -> this._getPausableStorage()
 - **What is controllable?** Only PAUSE_ROLE holders, typically multiliquidAdmin, may invoke this path.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Issuer admins cannot unpause via unpauseMultiliquid unless also granted PAUSE_ROLE.
 - **What happens if it reverts, reenters or does other unusual control flow?** OpenZeppelin _pause only flips local storage and emits Paused; it makes no external calls and cannot reenter.

Function: removeRWAColdStorageAddress(address coldStorageAddress)

The removeRWAColdStorageAddress function removes a previously recorded RWA cold-storage address from the transparency set.

Inputs

- coldStorageAddress

- **Control:** Issuer admin via `onlyIssuerAdmin`.
- **Constraints:** Must be nonzero and must not be `address(this)`.
- **Impact:** Removed from `_rwaColdStorageAddresses` when present.

Branches and code coverage

Intended branches

- Removes an existing address and emits `RWAColdStorageAddressRemoved` when removal succeeds.
 - ☒ Test coverage
- Returns without event when the address is absent.
 - ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts with `AddressNotAllowed` when the address is zero or the delegate itself.
 - ☒ Negative test

Function call analysis

- `EnumerableSet.remove(this._rwaColdStorageAddresses, coldStorageAddress)`
 - **What is controllable?** `coldStorageAddress` is issuer-admin controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** The boolean return gates event emission only. Removing a missing address is a harmless no-op.
 - **What happens if it reverts, reenters or does other unusual control flow?** `EnumerableSet` is internal library code with no external calls; it cannot reenter.

Function: `removeStablecoinColdStorageAddress(address coldStorageAddress)`

The `removeStablecoinColdStorageAddress` function removes a previously recorded stablecoin cold-storage address from the transparency set.

Inputs

- coldStorageAddress
 - **Control:** Issuer admin via onlyIssuerAdmin.
 - **Constraints:** Must be nonzero and must not be address(this).
 - **Impact:** Removed from _stablecoinColdStorageAddresses when present.

Branches and code coverage

Intended branches

- Removes an existing address and emits StablecoinColdStorageAddressRemoved when removal succeeds.
 - ☒ Test coverage
- Returns without event when the address is absent.
 - ☒ Test coverage

Negative behavior

- Reverts with NotIssuerAdmin when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts with AddressNotAllowed when the address is zero or the delegate itself.
 - ☒ Negative test

Function call analysis

- EnumerableSet.remove(this._stablecoinColdStorageAddresses, coldStorageAddress)
 - **What is controllable?** coldStorageAddress is issuer-admin controlled.
 - **If the return value is controllable, how is it used and how can it go wrong?** The boolean return gates event emission only. Removing a missing address is a harmless no-op.
 - **What happens if it reverts, reenters or does other unusual control flow?** EnumerableSet is internal library code with no external calls; it cannot reenter.

Function: setAutoLiquidityStablecoinID(byte[32] autoLiquidityStablecoinID)

The setAutoLiquidityStablecoinID function selects which yield stablecoin MultiliquidSwap uses for autoliquidity prefund and sweep legs on routes through this delegate.

Inputs

- `autoLiquidityStablecoinID`
 - **Control:** Issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Arbitrary `bytes32`; `bytes32(0)` disables autoliquidity legs per `NatSpec`.
 - **Impact:** Routes autoliquidity behavior for this delegate's `stablecoinID` in `MultiliquidSwap`.

Branches and code coverage

Intended branches

- Forwards the ID to `MultiliquidSwap` and emits `AutoLiquidityStablecoinIDSet`.
 - ☒ Test coverage
- Accepts `bytes32(0)` to disable automatic prefund and sweep legs.
 - ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts if `MultiliquidSwap` rejects the caller or an unknown `stablecoinID`.
 - ☐ Negative test

Function call analysis

- `IMultiliquidSwap(this.multiliquidSwap)`
 - `.setAutoLiquidityStablecoinID(this.stablecoinID, autoLiquidityStablecoinID)`
- **What is controllable?** `autoLiquidityStablecoinID` is issuer-admin controlled; `stablecoinID` and `multiliquidSwap` are fixed at init.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. An incorrect ID can misroute autoliquidity or disable it when zero is passed.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the local emit. `MultiliquidSwap` callbacks could reenter, but this function mutates no local state.

Function: `setBlacklist(address account, bool isBlacklisted)`

The `setBlacklist` function lets an issuer admin block or unblock addresses in addition to MultiliquidSwap's global blacklist.

Inputs

- `account`
 - **Control:** Issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Must be nonzero.
 - **Impact:** Address checked by `checkBlacklist` on swap user and receiver in child delegates.
- `isBlacklisted`
 - **Control:** Issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Arbitrary boolean.
 - **Impact:** `true` blocks the account from issuer-gated swaps; `false` removes the block.

Branches and code coverage

Intended branches

- Writes `blacklist[account]` and emits `BlacklistUpdated`.
 - ☒ Test coverage
- Child swap entry points use `checkBlacklist` on user and receiver.
 - ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts with `AddressNotAllowed` when `account` is zero.
 - ☒ Negative test

Function: `setRWACustodyAddress(address _balanceSheetCustodyAddress)`

The `setRWACustodyAddress` function sets the hot wallet that receives and sends RWA collateral during atomic swaps.

Inputs

- `_balanceSheetCustodyAddress`
 - **Control:** Issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Must be nonzero and must not be `address(this)`.
 - **Impact:** Becomes `balanceSheetCustodyAddress` used by child delegates for RWA custody transfers.

Branches and code coverage

Intended branches

- Persists `balanceSheetCustodyAddress` and emits `RWACustodyAddressSet`.
 - ☒ Test coverage
- Satisfies `_custodyAddressesSet` once both custody addresses are configured.
 - ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
 - ☒ Negative test
- Reverts with `AddressNotAllowed` when the address is zero or the delegate itself.
 - ☒ Negative test

Function: `setRWADiscountRate(byte[32] rwaID, uint256 rate)`

The `setRWADiscountRate` function forwards an issuer-chosen RWA discount rate to `MultiliquidSwap` for this delegate's `stablecoinID`.

Inputs

- `rwaID`
 - **Control:** Issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Arbitrary `bytes32` key understood by `MultiliquidSwap`.
 - **Impact:** Selects which RWA leg receives the discount in swap pricing.
- `rate`
 - **Control:** Issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Must be strictly less than `MAX_RATE_EXCLUSIVE` (`1e18`).
 - **Impact:** Adjusts effective RWA pricing for routes involving this stablecoin.

Branches and code coverage

Intended branches

- Validates the rate locally, calls MultiliquidSwap, and emits RWADiscountRateSet.
- ☒ Test coverage

Negative behavior

- Reverts with NotIssuerAdmin when the caller is not an issuer admin.
- ☒ Negative test
- Reverts with InvalidRate when rate >= MAX_RATE_EXCLUSIVE.
- ☒ Negative test
- Reverts if MultiliquidSwap rejects the caller or parameters.
- ☐ Negative test

Function call analysis

- IMultiliquidSwap(this.multiliquidSwap)
- .setRWADiscountRate(this.stablecoinID, rwaID, rate)
- What is controllable?** rwaID and rate are issuer-admin controlled; stablecoinID and multiliquidSwap are fixed at init.
 - If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Misconfiguration only affects future swap pricing in MultiliquidSwap.
 - What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the local emit. MultiliquidSwap callbacks could reenter, but this function mutates no local state before the external call beyond rate validation.

Function: setRWARedemptionFee(byte[32] rwaID, uint256 rate)

The setRWARedemptionFee function forwards an issuer-chosen RWA redemption fee to MultiliquidSwap for this delegate's stablecoinID.

Inputs

- rwaID
 - Control:** Issuer admin via onlyIssuerAdmin.
 - Constraints:** Arbitrary bytes32 key understood by MultiliquidSwap.

- **Impact:** Selects which RWA leg pays the redemption fee on exit routes.
- `rate`
 - **Control:** Issuer admin via `onlyIssuerAdmin`.
 - **Constraints:** Must be strictly less than `MAX_RATE_EXCLUSIVE` (1e18).
 - **Impact:** Sets the fee charged when redeeming this stablecoin into the named RWA.

Branches and code coverage

Intended branches

- Validates the rate locally, calls `MultiliquidSwap`, and emits `RedemptionFeeSet`.
- ☒ Test coverage

Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
- ☒ Negative test
- Reverts with `InvalidRate` when `rate >= MAX_RATE_EXCLUSIVE`.
- ☒ Negative test
- Reverts if `MultiliquidSwap` rejects the caller or parameters.
- ☐ Negative test

Function call analysis

- ```
IMultiliquidSwap(this.multiliquidSwap).setRWARedemptionFee(this.stablecoinID, rwaID, rate)
```

  - **What is controllable?** `rwaID` and `rate` are issuer-admin controlled; `stablecoinID` and `multiliquidSwap` are fixed at init.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Misconfiguration only affects future redemption pricing in `MultiliquidSwap`.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the local emit. `MultiliquidSwap` callbacks could reenter, but this function mutates no local state before the external call beyond rate validation.

## Function: `setStablecoinAcceptanceFee(address stablecoin, uint256 rate)`

The `setStablecoinAcceptanceFee` function forwards an issuer-chosen acceptance fee for inbound stablecoin backing assets to `MultiliquidSwap`.

### Inputs

- `stablecoin`
  - **Control:** Issuer admin via `onlyIssuerAdmin`.
  - **Constraints:** Arbitrary ERC-20 address key understood by `MultiliquidSwap`.
  - **Impact:** Selects which inbound stablecoin pays the acceptance fee.
- `rate`
  - **Control:** Issuer admin via `onlyIssuerAdmin`.
  - **Constraints:** Must be strictly less than `MAX_RATE_EXCLUSIVE` (1e18).
  - **Impact:** Sets the fee charged when accepting the named stablecoin as backing.

### Branches and code coverage

#### Intended branches

- Validates the rate locally, calls `MultiliquidSwap`, and emits `StablecoinAcceptanceFeeSet`.

☒ Test coverage

#### Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.

☒ Negative test

- Reverts with `InvalidRate` when `rate >= MAX_RATE_EXCLUSIVE`.

☒ Negative test

- Reverts if `MultiliquidSwap` rejects the caller or parameters.

☐ Negative test

### Function call analysis

- `IMultiliquidSwap(this.multiliquidSwap).setStablecoinAcceptanceFee(this.stablecoin, rate)`

- **What is controllable?** stablecoin and rate are issuer-admin controlled; stablecoinID and multiliquidSwap are fixed at init.
- **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Misconfiguration only affects future acceptance pricing in MultiliquidSwap.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the local emit. MultiliquidSwap callbacks could reenter, but this function mutates no local state before the external call beyond rate validation.

### Function: `setStablecoinCustodyAddress(address _stablecoinCustodyAddress)`

The `setStablecoinCustodyAddress` function sets the hot wallet that holds stablecoin reserves used in atomic swap custody flows.

#### Inputs

- `_stablecoinCustodyAddress`
  - **Control:** Issuer admin via `onlyIssuerAdmin`.
  - **Constraints:** Must be nonzero and must not be `address(this)`.
  - **Impact:** Becomes `stablecoinCustodyAddress` exposed as a public view and required by `custodyAddressesSet`.

#### Branches and code coverage

##### Intended branches

- Persists `stablecoinCustodyAddress` and emits `StablecoinCustodyAddressSet`.
  - ☒ Test coverage
- Satisfies `_custodyAddressesSet` once both custody addresses are configured.
  - ☒ Test coverage

##### Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
  - ☒ Negative test
- Reverts with `AddressNotAllowed` when the address is zero or the delegate itself.
  - ☒ Negative test

## Function: `setStablecoinRedemptionFee(address stablecoin, uint256 rate)`

The `setStablecoinRedemptionFee` function forwards an issuer-chosen redemption fee for outbound stablecoin backing assets to `MultiliquidSwap`.

### Inputs

- `stablecoin`
  - **Control:** Issuer admin via `onlyIssuerAdmin`.
  - **Constraints:** Arbitrary ERC-20 address key understood by `MultiliquidSwap`.
  - **Impact:** Selects which outbound stablecoin pays the redemption fee.
- `rate`
  - **Control:** Issuer admin via `onlyIssuerAdmin`.
  - **Constraints:** Must be strictly less than `MAX_RATE_EXCLUSIVE` (1e18).
  - **Impact:** Sets the fee charged when redeeming into the named stablecoin.

### Branches and code coverage

#### Intended branches

- Validates the rate locally, calls `MultiliquidSwap`, and emits `StablecoinRedemptionFeeSet`.

☒ Test coverage

#### Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.

☒ Negative test

- Reverts with `InvalidRate` when `rate >= MAX_RATE_EXCLUSIVE`.

☒ Negative test

- Reverts if `MultiliquidSwap` rejects the caller or parameters.

☐ Negative test

### Function call analysis

- - `IMultiliquidSwap(this.multiliquidSwap).setStablecoinRedemptionFee(this.stablecoin, rate)`
  - **What is controllable?** `stablecoin` and `rate` are issuer-admin controlled; `stablecoinID` and `multiliquidSwap` are fixed at init.

- **If the return value is controllable, how is it used and how can it go wrong?**  
No return value is consumed. Misconfiguration only affects future redemption pricing in MultiliquidSwap.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the local emit. MultiliquidSwap callbacks could reenter, but this function mutates no local state before the external call beyond rate validation.

## Function: `unpause()`

The `unpause` function lets an issuer admin restore delegate swap entry points after a pause.

## Branches and code coverage

### Intended branches

- Clears the Pausable flag via `_unpause()`, reenabling child swap functions that use `whenNotPaused`.  
☒ Test coverage
- Separate from `unpauseMultiliquid`, which is limited to `PAUSE_ROLE` holders.  
☒ Test coverage

### Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.  
☒ Negative test
- Reverts with `ExpectedPause` when the contract is not paused.  
☐ Negative test

## Function call analysis

- `this._unpause()` -> `this._getPausableStorage()`
  - **What is controllable?** Only issuer admins may invoke this path.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value is consumed. The paused flag is a single storage slot shared with `unpauseMultiliquid`.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
`OpenZeppelin _unpause` only flips local storage and emits `Unpaused`; it makes no external calls and cannot reenter.

### Function: `unpauseMultiliquid()`

The `unpauseMultiliquid` function lets Multiliquid's `PAUSE_ROLE` holder unpause the delegate without issuer-admin authority.

### Branches and code coverage

#### Intended branches

- Clears the shared Pausable flag via `_unpause()` when the caller holds `PAUSE_ROLE`.
  - ☒ Test coverage
- Issuer admins use `unpause()` instead; role separation prevents Multiliquid from controlling issuer-only admin actions.
  - ☒ Test coverage

#### Negative behavior

- Reverts with OpenZeppelin access-control error when the caller lacks `PAUSE_ROLE`.
  - ☒ Negative test
- Reverts with `ExpectedPause` when the contract is not paused.
  - ☐ Negative test

### Function call analysis

- `this._unpause()` -> `this._getPausableStorage()`
  - **What is controllable?** Only `PAUSE_ROLE` holders, typically `multiliquidAdmin`, may invoke this path.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. An issuer pause can be cleared by any `PAUSE_ROLE` holder via this function.
  - **What happens if it reverts, reenters or does other unusual control flow?** OpenZeppelin `_unpause` only flips local storage and emits `Unpaused`; it makes no external calls and cannot reenter.

### Function: `whitelistRWA(address rwa, bool accepted)`

The `whitelistRWA` function lets an issuer admin allow or deny a specific RWA token address for swap legs guarded by `onlyWhitelistedRWA`.



## Inputs

- `rwa`
  - **Control:** Issuer admin via `onlyIssuerAdmin`.
  - **Constraints:** Must be nonzero.
  - **Impact:** Sets whether that RWA can appear in issuer-gated swap paths.
- `accepted`
  - **Control:** Issuer admin via `onlyIssuerAdmin`.
  - **Constraints:** Arbitrary boolean.
  - **Impact:** `true` whitelists the RWA; `false` removes it and causes future swaps to revert.

## Branches and code coverage

### Intended branches

- Writes `rwaWhitelist[rwa]` and emits `RWAWhitelist`.
  - ☒ Test coverage
- Child swap entry points use `onlyWhitelistedRWA` to enforce the mapping.
  - ☒ Test coverage

### Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.
  - ☒ Negative test
- Reverts with `AddressNotAllowed` when `rwa` is zero.
  - ☒ Negative test

## Function: `whitelistStablecoin(address stablecoin, bool accepted)`

The `whitelistStablecoin` function lets an issuer admin allow or deny another stablecoin as a backing asset in stablecoin-to-stablecoin swap legs.

## Inputs

- `stablecoin`
  - **Control:** Issuer admin via `onlyIssuerAdmin`.
  - **Constraints:** Must be nonzero.
  - **Impact:** Sets whether that stablecoin can be used in issuer-gated stablecoin exchange paths.

- accepted
  - **Control:** Issuer admin via onlyIssuerAdmin.
  - **Constraints:** Arbitrary boolean.
  - **Impact:** true whitelists the stablecoin; false blocks future swaps involving it.

## Branches and code coverage

### Intended branches

- Writes stablecoinWhitelist[stablecoin] and emits StablecoinWhitelist.
  - ☒ Test coverage
- Child swap entry points call \_onlyWhitelistedStablecoin for cross-stablecoin routes.
  - ☒ Test coverage

### Negative behavior

- Reverts with NotIssuerAdmin when the caller is not an issuer admin.
  - ☒ Negative test
- Reverts with AddressNotAllowed when stablecoin is zero.
  - ☒ Negative test

## Function: withdrawStablecoin(address to, uint256 amount)

The withdrawStablecoin function lets an issuer admin transfer stablecoin ERC-20 balance held by the delegate to an external recipient.

### Inputs

- to
  - **Control:** Issuer admin via onlyIssuerAdmin.
  - **Constraints:** Must be nonzero.
  - **Impact:** Receives the withdrawn stablecoin tokens.
- amount
  - **Control:** Issuer admin via onlyIssuerAdmin.
  - **Constraints:** Must not exceed the delegate's token balance; token must honor transfer.
  - **Impact:** Quantity of stablecoinAddress tokens sent out of delegate custody.

## Branches and code coverage

### Intended branches

- Transfers amount of `stablecoinAddress` via `SafeERC20` and emits `StablecoinWithdrawn`.

☒ Test coverage

### Negative behavior

- Reverts with `NotIssuerAdmin` when the caller is not an issuer admin.

☒ Negative test

- Reverts with `ZeroAddressNotAllowed` when `to` is zero.

☒ Negative test

- Reverts when the token transfer fails or balance is insufficient.

☐ Negative test

## Function call analysis

- `SafeERC20.safeTransfer(IERC20(this.stablecoinAddress), to, amount)`
  - **What is controllable?** `to` and `amount` are issuer-admin controlled; `stablecoinAddress` is fixed at init.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. `SafeERC20` enforces a successful transfer; fee-on-transfer or nonstandard tokens may behave unexpectedly.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the withdrawal and emit. Malicious token hooks could reenter before the emit, but issuer admins are trusted, and no further state is updated after the transfer.

### 4.15. Module: `ThirdPartySetterPriceAdapter.sol`

#### Function: `constructor(address admin)`

The constructor function initializes a manually set price adapter that is not upgradable and grants its initial default admin role.

#### Inputs

- `admin`
- **Control:** Set by the deployer

- **Constraints:** Must not be zero.
- **Impact:** The initial DEFAULT\_ADMIN\_ROLE member.

## Branches and code coverage

### Intended branches

- Grants DEFAULT\_ADMIN\_ROLE to a nonzero admin while keeping the price zero.

☒ Test coverage

### Negative behavior

- Reverts if the admin address is zero.

☒ Negative test

## Function: getPrice()

The getPrice function returns the currently stored WAD price after requiring that it is not zero.

## Branches and code coverage

### Intended branches

- Returns the nonzero stored price.

☒ Test coverage

### Negative behavior

- Reverts if the price is unset or zero.

☒ Negative test

## Function: grantRole(bytes32 role, address account)

The grantRole function grants role to account if the caller has the admin role for role.

## Inputs

- `role`
  - **Control:** Set by role admin.
  - **Constraints:** The caller must hold the admin role associated with `role`.
  - **Impact:** The role to grant.

- `account`
  - **Control:** Set by role admin.
  - **Constraints:** Arbitrary.
  - **Impact:** The account to grant the role to.

## Branches and code coverage

### Intended branches

- Grants and enumerates a new member.
- ☒ Test coverage

### Negative behavior

- Reverts if the caller does not have the admin role for `role`.
- ☒ Negative test

## Function: `renounceRole(bytes32 role, address callerConfirmation)`

The `renounceRole` function revokes `role` from `msg.sender` if `callerConfirmation` equals `msg.sender`.

## Inputs

- `role`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The role to renounce.
- `callerConfirmation`
  - **Control:** Arbitrary.
  - **Constraints:** Must equal `msg.sender`.
  - **Impact:** The address that must match `msg.sender`.

## Branches and code coverage

### Intended branches

- Removes `role` and its enumeration entry after matching confirmation.
- ☐ Test coverage

### Negative behavior

- Reverts if callerConfirmation does not equal msg.sender.

☐ Negative test

### Function: revokeRole(bytes32 role, address account)

The revokeRole function revokes role from account if the caller has the admin role for role.

### Inputs

- role
  - **Control:** Set by the role admin.
  - **Constraints:** The caller must hold the admin role associated with role.
  - **Impact:** The role to revoke.
- account
  - **Control:** Set by role admin.
  - **Constraints:** Arbitrary.
  - **Impact:** The account to revoke the role from.

### Branches and code coverage

#### Intended branches

- Revokes role and removes its enumeration entry.

☒ Test coverage

### Negative behavior

- Reverts if the caller does not have the admin role for role.

☐ Negative test

### Function: setPrice(uint256 newPrice)

The setPrice function stores a new manually supplied WAD price when called by a PRICE\_SETTER\_ROLE member.

### Inputs

- newPrice

- **Control:** Set by price setter.
- **Constraints:** The caller must hold PRICE\_SETTER\_ROLE.
- **Impact:** The new value returned by getPrice.

## Branches and code coverage

### Intended branches

- Stores a positive price.
  - ☒ Test coverage
- Stores zero so getPrice reverts.
  - ☒ Test coverage

### Negative behavior

- Reverts if the caller does not have PRICE\_SETTER\_ROLE.
  - ☒ Negative test

## 4.16. Module: Treasury.sol

### Function: \_update(address from, address to, uint256 amount)

The \_update function overrides ERC-20 balance updates to enforce pause and blacklist rules on every mint, burn, and transfer.

### Inputs

- from
  - **Control:** Set by internal mint/burn/transfer callers.
  - **Constraints:** Must not be blacklisted when nonzero (burns and sends).
  - **Impact:** Debited account — address(0) for mints.
- to
  - **Control:** Set by internal mint/burn/transfer callers.
  - **Constraints:** Must not be blacklisted when nonzero (mints and receives).
  - **Impact:** Credited account — address(0) for burns.
- amount
  - **Control:** Set by calling function.
  - **Constraints:** Standard ERC-20 balance sufficiency enforced by parent.
  - **Impact:** Token amount moved or created/destroyed.

## Branches and code coverage

### Intended branches

- Reverts when a contract is paused via whenNotPaused.  
☒ Test coverage
- Reverts AccountBlacklisted(from) when the sender is blacklisted.  
☒ Test coverage
- Reverts AccountBlacklisted(to) when the receiver is blacklisted.  
☒ Test coverage
- Delegates to super.\_update when pause and blacklist checks pass.  
☒ Test coverage

### Negative behavior

- Reverts on pause, blacklisted from, or blacklisted to before parent balance logic runs.  
☒ Negative test
- Propagates parent ERC-20 insufficient-balance reverts.  
☒ Negative test

## Function: addBridge(address bridge)

The addBridge function grants BRIDGE\_ROLE to a cross-chain adapter, so it may call bridgeMint for secondary issuance.

### Inputs

- bridge
  - **Control:** Set by an operator via OPERATOR\_ROLE.
  - **Constraints:** Must not be address(0).
  - **Impact:** Receives BRIDGE\_ROLE and is recorded in BridgeAdded events for monitoring.

## Branches and code coverage

### Intended branches

- Grants BRIDGE\_ROLE to bridge and emits BridgeAdded.  
☒ Test coverage



## Negative behavior

- Reverts ZeroAddress if bridge is zero.
  - ☑ Negative test
- Reverts if the caller lacks OPERATOR\_ROLE.
  - ☑ Negative test

## Function call analysis

- `this.grantRole(Treasury.BRIDGE_ROLE, bridge) -> this._grantRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** bridge address chosen by operator.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. A malicious bridge can mint via `bridgeMint` without whitelist checks.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert leaves the bridge unregistered. No external callbacks to bridge.
- `this.grantRole(Treasury.BRIDGE_ROLE, bridge) -> this._grantRole(role, account) -> this.hasRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** Internal duplicate-membership check.
  - **If the return value is controllable, how is it used and how can it go wrong?** Idempotent if role is already held.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the grant.
- `this.grantRole(Treasury.BRIDGE_ROLE, bridge) -> this._grantRole(role, account) -> this._getAccessControlEnumerableStorage()`
  - **What is controllable?** Operator-supplied bridge address.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the grant.
- `this.grantRole(Treasury.BRIDGE_ROLE, bridge) -> this._grantRole(role, account) -> EnumerableSet.add($_roleMembers[role], account)`
  - **What is controllable?** bridge address.
  - **If the return value is controllable, how is it used and how can it go wrong?** Set-add result is unused.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts the grant — no external calls.

### Function: `addToBlacklist(address account)`

The `addToBlacklist` function blocks an address from sending or receiving treasury tokens.

#### Inputs

- `account`
  - **Control:** Set by a blacklist via `BLACKLISTER_ROLE`.
  - **Constraints:** Must not be `address(0)`.
  - **Impact:** Sets `blacklisted[account] = true` and emits `AddedToBlacklist` — all `_update` paths involving the address revert.

#### Branches and code coverage

##### Intended branches

- Sets blacklist flag and emits event for a nonzero account.
  - ☒ Test coverage
- Idempotent if the account is already blacklisted.
  - ☒ Test coverage

##### Negative behavior

- Reverts `ZeroAddress` if account is zero.
  - ☒ Negative test
- Reverts if the caller lacks `BLACKLISTER_ROLE`.
  - ☒ Negative test

### Function: `addToBlacklistBatch(address[] accounts)`

The `addToBlacklistBatch` function blacklists multiple addresses in one transaction.

#### Inputs

- `accounts`
  - **Control:** Set by a blacklist via `BLACKLISTER_ROLE`.
  - **Constraints:** Each element must not be `address(0)`; empty array is a no-op.
  - **Impact:** Sets `blacklisted[account] = true` and emits `AddedToBlacklist` for each entry.

## Branches and code coverage

### Intended branches

- Iterates all entries, blacklists each, and emits one event per address.
  - ☒ Test coverage
- Succeeds with zero-length input without state changes.
  - ☒ Test coverage

### Negative behavior

- Reverts ZeroAddress on the first zero entry in the array.
  - ☒ Negative test
- Reverts if the caller lacks BLACKLISTER\_ROLE.
  - ☒ Negative test

## Function: addToWhitelist(address account)

The addToWhitelist function marks an address as eligible to receive primary-issuance mint transfers.

### Inputs

- account
  - **Control:** Set by a whitelister via WHITELISTER\_ROLE.
  - **Constraints:** Must not be address(0).
  - **Impact:** Sets whitelisted[account] = true and emits AddedToWhitelist.

## Branches and code coverage

### Intended branches

- Sets whitelist flag and emits event for a nonzero account.
  - ☒ Test coverage
- Idempotent if the account is already whitelisted.
  - ☒ Test coverage

### Negative behavior

- Reverts ZeroAddress if account is zero.
  - ☒ Negative test

- Reverts if the caller lacks WHITELISTER\_ROLE.

☒ Negative test

### Function: bridgeMint(address to, uint256 amount)

The bridgeMint function mints treasury tokens for cross-chain secondary issuance. Only BRIDGE\_ROLE holders may call it, and the whitelist is not required.

#### Inputs

- to
  - **Control:** Set by the bridge contract (msg.sender must hold BRIDGE\_ROLE).
  - **Constraints:** Must not be address(0) — subject to blacklist check in \_update.
  - **Impact:** Recipient of bridged-minted tokens on this chain.
- amount
  - **Control:** Set by the bridge from verified cross-chain message data.
  - **Constraints:** Must be greater than zero.
  - **Impact:** Increases total supply — emitted in BridgeMint for off-chain reconciliation.

### Branches and code coverage

#### Intended branches

- Emits BridgeMint and mints amount to to when the caller holds BRIDGE\_ROLE and contract is unpaused.

☒ Test coverage

#### Negative behavior

- Reverts ZeroAddress if to is zero.
 

☒ Negative test
- Reverts InvalidAmount if amount is zero.
 

☒ Negative test
- Reverts when paused, on reentrancy, or if the caller lacks BRIDGE\_ROLE.
 

☒ Negative test
- Reverts AccountBlacklisted if to is blacklisted.
 

☒ Negative test

## Function call analysis

- `this._mint(to, amount) -> this._update(address(0), account, value) -> this._getERC20Storage()`
  - **What is controllable?** `to` and `amount` are bridge-supplied after LayerZero (or other) verification off-chain/on adapter.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Compromised bridge can inflate supply to arbitrary nonblacklisted recipients.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back mint and event, and `nonReentrant` blocks reentrant bridge mints.

## Function: `burn(uint256 amount)`

The burn function lets the caller destroy their own treasury token balance.

### Inputs

- `amount`
  - **Control:** Set by `msg.sender`.
  - **Constraints:** Must be greater than zero; cannot exceed caller balance.
  - **Impact:** Reduces caller balance and total supply.

## Branches and code coverage

### Intended branches

- Burns amount from `msg.sender` via `super.burn` when unpaused.
  - ☒ Test coverage

### Negative behavior

- Reverts `InvalidAmount` if amount is zero.
  - ☒ Negative test
- Reverts when paused or on insufficient balance.
  - ☒ Negative test
- Reverts `AccountBlacklisted` if caller is blacklisted.
  - ☒ Negative test

## Function call analysis

- `super.burn(amount) -> this._burn(this._msgSender(), value) -> this._update(account, address(0), value) -> this._getERC20Storage()`
  - **What is controllable?** Caller controls amount subject to own balance.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pause or blacklist in `_update` reverts the burn. No external callbacks.

## Function: `burnFrom(address from, uint256 amount)`

The `burnFrom` function burns tokens from `from` using the caller's allowance. Bridge callers additionally emit `BridgeBurn` for monitoring.

## Inputs

- `from`
  - **Control:** Set by the caller — must have approved `msg.sender` (or bridge).
  - **Constraints:** Must have sufficient balance and allowance — subject to blacklist check.
  - **Impact:** Account whose tokens are destroyed — bridge burns reduce cross-chain supply.
- `amount`
  - **Control:** Set by the caller.
  - **Constraints:** Must be greater than zero — cannot exceed balance or allowance.
  - **Impact:** Amount removed from supply and `from` balance.

## Branches and code coverage

### Intended branches

- Burns via `super.burnFrom` after zero-amount check when unpaused.
  - ☒ Test coverage
- Emits `BridgeBurn` when `msg.sender` holds `BRIDGE_ROLE`.
  - ☒ Test coverage

### Negative behavior

- Reverts `InvalidAmount` if amount is zero.

- ☑ Negative test
- Reverts when paused, on insufficient balance/allowance, or if from is blacklisted.
- ☑ Negative test

## Function call analysis

- `this.hasRole(Treasury.BRIDGE_ROLE, msg.sender) -> this._getAccessControlStorage()`
  - **What is controllable?** `msg.sender` determines whether bridge monitoring event fires.
  - **If the return value is controllable, how is it used and how can it go wrong?** Boolean only gates BridgeBurn emission; burn proceeds regardless.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts before burn. No external callbacks.
- `super.burnFrom(from, amount) -> this._spendAllowance(account, this._msgSender(), value) -> this.allowance(owner, spender) -> this._getERC20Storage()`
  - **What is controllable?** Caller controls from, amount, and prior approval amount.
  - **If the return value is controllable, how is it used and how can it go wrong?** Allowance must be sufficient; infinite allowance avoids decrement issues.
  - **What happens if it reverts, reenters or does other unusual control flow?** Insufficient allowance reverts. No external token calls.
- `super.burnFrom(from, amount) -> this._spendAllowance(account, this._msgSender(), value) -> this._approve(owner, spender, currentAllowance - value, False) -> this._getERC20Storage()`
  - **What is controllable?** Spender is `msg.sender` — allowance reduced unless infinite.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back allowance spend and burn.
- `super.burnFrom(from, amount) -> this._burn(account, value) -> this._update(account, address(0), value) -> this._getERC20Storage()`
  - **What is controllable?** from balance must cover amount.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Irreversible supply reduction on success.
  - **What happens if it reverts, reenters or does other unusual control flow?** Blacklist or pause in `_update` reverts the burn.

## Function: constructor ( )

The constructor function disables initializers on the Treasury implementation so the logic contract cannot be initialized outside its UUPS proxy.

## Branches and code coverage

### Intended branches

- Calls `_disableInitializers()` to lock the implementation contract.

☒ Test coverage

### Negative behavior

- N/A for runtime callers — direct initialization of the implementation is permanently disabled.

☐ Negative test

## Function: forceBurn(address from, uint256 amount)

The forceBurn function lets an operator burn tokens from any account without allowance, for emergency or compliance use.

## Inputs

- `from`
  - **Control:** Set by the operator.
  - **Constraints:** Must not be `address(0)` — must have sufficient balance.
  - **Impact:** Account whose tokens are destroyed — emits `ForcedBurn`.
- `amount`
  - **Control:** Set by the operator.
  - **Constraints:** Must be greater than zero.
  - **Impact:** Supply reduction bypassing user consent.

## Branches and code coverage

### Intended branches

- Emits `ForcedBurn` and burns via `super._update(from, address(0), amount)`.

☒ Test coverage



### Negative behavior

- Reverts ZeroAddress if from is zero.
  - ☒ Negative test
- Reverts InvalidAmount if amount is zero.
  - ☒ Negative test
- Reverts if the caller lacks OPERATOR\_ROLE or from has insufficient balance.
  - ☒ Negative test

### Function call analysis

- `super._update(from, address(0), amount) -> this._getERC20Storage()`
  - **What is controllable?** Operator chooses from and amount — no allowance required.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Compromised operator can seize value by burning user balances.
  - **What happens if it reverts, reenters or does other unusual control flow?** Insufficient balance or blacklist on from reverts. Pause via whenNotPaused on \_update also blocks forceBurn while paused.

### Function: forceTransfer(address from, address to, uint256 amount)

The forceTransfer function lets an operator move tokens between accounts without approval, for emergency or compliance use.

### Inputs

- from
  - **Control:** Set by the operator.
  - **Constraints:** Must not be address(0), must have sufficient balance, and must not be blacklisted.
  - **Impact:** Source account debited — emits ForcedTransfer.
- to
  - **Control:** Set by the operator.
  - **Constraints:** Must not be address(0) and not be blacklisted.
  - **Impact:** Destination account credited.
- amount

- **Control:** Set by the operator.
- **Constraints:** Must be greater than zero.
- **Impact:** Value moved without user consent.

## Branches and code coverage

### Intended branches

- Emits ForcedTransfer and moves tokens via `super._update(from, to, amount)`.

☒ Test coverage

### Negative behavior

- Reverts ZeroAddress if from or to is zero.

☒ Negative test

- Reverts InvalidAmount if amount is zero.

☒ Negative test

- Reverts if the caller lacks OPERATOR\_ROLE, balance is insufficient, or either party is blacklisted.

☒ Negative test

## Function call analysis

- `super._update(from, to, amount) -> this._getERC20Storage()`
  - **What is controllable?** Operator chooses all transfer parameters — no allowance required.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Compromised operator can confiscate balances to any non-blacklisted address.
  - **What happens if it reverts, reenters or does other unusual control flow?** Blacklist or insufficient balance reverts. Not gated by pause modifier on `forceTransfer` itself, but `_update` enforces the blacklist.

### Function: `initialize(string name_, string symbol_, address admin_)`

The `initialize` function sets up the upgradable ERC-20 treasury token with permit support, access control, pause, and reentrancy guard and grants all operational roles to `admin_`.

## Inputs

- `name_`
  - **Control:** Set by the deployer during proxy initialization.
  - **Constraints:** Arbitrary string — also used for EIP-712 permit domain.
  - **Impact:** ERC-20 token name and permit domain identifier.
- `symbol_`
  - **Control:** Set by the deployer during proxy initialization.
  - **Constraints:** Arbitrary string.
  - **Impact:** ERC-20 token symbol.
- `admin_`
  - **Control:** Set by the deployer during proxy initialization.
  - **Constraints:** Must not be `address(0)`.
  - **Impact:** Receives `DEFAULT_ADMIN_ROLE`, `OPERATOR_ROLE`, `MINTER_ROLE`, `PAUSER_ROLE`, `WHITELISTER_ROLE`, and `BLACKLISTER_ROLE`.

## Branches and code coverage

### Intended branches

- Initializes ERC-20, burnable, permit, access control, pause, reentrancy, and UUPS modules then grants roles and sets `OPERATOR_ROLE` as admin for `BRIDGE_ROLE` and `MINTER_ROLE`.

☒ Test coverage

### Negative behavior

- Reverts `ZeroAddress` if `admin_` is zero.
- Reverts if called outside the initializer context or when already initialized.

☒ Negative test

## Function call analysis

- `this.__ERC20_init(name_, symbol_) -> this.__ERC20_init_unchained(name_, symbol_) -> this.getERC20Storage()`
  - **What is controllable?** Token name and symbol are deployer-supplied at init.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed.

- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization before any roles are granted.
- `this.__ERC20Permit_init(name_) -> this.__EIP712_init_unchained(name, "1")`  
`-> this._getEIP712Storage()`
  - **What is controllable?** Permit domain name mirrors name\_ from deployer.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value is consumed. Wrong name breaks off-chain permit signatures.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this.__ReentrancyGuard_init() -> this.__ReentrancyGuard_init_unchained()`  
`-> this._getReentrancyGuardStorage()`
  - **What is controllable?** Internal OZ initializer only.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value is consumed.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, admin_) ->`  
`this._getAccessControlEnumerableStorage()`
  - **What is controllable?** admin\_ is deployer chosen.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value is consumed. Compromised admin gains full treasury control.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, admin_) ->`  
`super._grantRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** Role and account fixed at init.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value is consumed.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, admin_) ->`  
`super._grantRole(role, account) -> this.hasRole(role, account) ->`  
`this._getAccessControlStorage()`
  - **What is controllable?** Internal membership check only.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Boolean gates idempotent grant.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, admin_) ->`

```
EnumerableSet.add($_roleMembers[role], account)
```

- **What is controllable?** admin\_address at init.
- **If the return value is controllable, how is it used and how can it go wrong?** Set-add result is unused.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this._setRoleAdmin(Treasury.BRIDGE_ROLE, Treasury.OPERATOR_ROLE) -> this._getAccessControlStorage()`
  - **What is controllable?** Hardcoded role hierarchy at init.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Operators can later grant or revoke bridge and minter roles.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `this._setRoleAdmin(Treasury.BRIDGE_ROLE, Treasury.OPERATOR_ROLE) -> this.getRoleAdmin(role) -> this._getAccessControlStorage()`
  - **What is controllable?** Internal read of role admin mapping.
  - **If the return value is controllable, how is it used and how can it go wrong?** Prior admin value is overwritten — no user input.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.

### Function: mint(address to, uint256 amount)

The mint function performs primary issuance minting to whitelisted recipients only. Callers must hold MINTER\_ROLE.

#### Inputs

- to
  - **Control:** Set by minter (primary issuance pipeline).
  - **Constraints:** Must be in whitelisted mapping.
  - **Impact:** Recipient of newly minted treasury tokens, subject to blacklist checks in \_update.
- amount
  - **Control:** Set by minter.
  - **Constraints:** Must be greater than zero.
  - **Impact:** Increases total supply and to balance.

## Branches and code coverage

### Intended branches

- Mints amount to whitelisted to when unpaused and the caller holds MINTER\_ROLE.

☒ Test coverage

### Negative behavior

- Reverts InvalidAmount if amount is zero.

☒ Negative test

- Reverts AccountNotWhitelisted if to is not whitelisted.

☒ Negative test

- Reverts when paused, on reentrancy, or if the caller lacks MINTER\_ROLE.

☒ Negative test

- Reverts AccountBlacklisted if to is blacklisted (via \_update).

☒ Negative test

## Function call analysis

- `this._mint(to, amount) -> this._update(address(0), account, value) -> this._getERC20Storage()`
  - **What is controllable?** to and amount are minter-supplied subject to whitelist and nonzero checks.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Unchecked mint inflation if minter role is compromised.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the mint, and nonReentrant blocks callback reentry. ERC-20 hooks in \_update run before balance commit.

### Function: pause ( )

The pause function halts all token transfers, mints, and burns that route through \_update or whenNotPaused entry points.

## Branches and code coverage

### Intended branches

- Sets the paused flag via \_pause ( ) when the caller holds PAUSER\_ROLE.

- ☒ Test coverage

### Negative behavior

- Reverts if the caller lacks PAUSER\_ROLE.

- ☒ Negative test

### Function call analysis

- `this._pause()` -> `this._getPausableStorage()`
  - **What is controllable?** Only PAUSER\_ROLE holders reach this call.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value is consumed. Pausing freezes user transfers, mint, bridgeMint, and burn.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert leaves the contract unpaused. No external callbacks.

### Function: removeBridge(address bridge)

The removeBridge function revokes BRIDGE\_ROLE from a bridge adapter, blocking further bridgeMint calls from that contract.

### Inputs

- bridge
  - **Control:** Set by an operator via OPERATOR\_ROLE.
  - **Constraints:** No zero-address check — revoking a nonmember is a no-op in OZ AccessControl.
  - **Impact:** Removes bridge mint authority and emits BridgeRemoved.

### Branches and code coverage

#### Intended branches

- Revokes BRIDGE\_ROLE from bridge and emits BridgeRemoved.

- ☒ Test coverage

#### Negative behavior

- Reverts if the caller lacks OPERATOR\_ROLE.

- ☒ Negative test

## Function call analysis

- `this.revokeRole(Treasury.BRIDGE_ROLE, bridge) -> this._revokeRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** The bridge address chosen by operator.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Revoking the wrong bridge halts its cross-chain mint path.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert leaves the role unchanged. No external callbacks.
- `this.revokeRole(Treasury.BRIDGE_ROLE, bridge) -> this._revokeRole(role, account) -> this.hasRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** Internal membership check — idempotent if role absent.
  - **If the return value is controllable, how is it used and how can it go wrong?** Boolean gates whether removal occurs.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts revocation.
- `this.revokeRole(Treasury.BRIDGE_ROLE, bridge) -> this._revokeRole(role, account) -> this._getAccessControlEnumerableStorage()`
  - **What is controllable?** Operator-supplied bridge address.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts revocation.
- `this.revokeRole(Treasury.BRIDGE_ROLE, bridge) -> this._revokeRole(role, account) -> EnumerableSet.remove($_roleMembers[role], account)`
  - **What is controllable?** The bridge address.
  - **If the return value is controllable, how is it used and how can it go wrong?** Set remove result is unused.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts revocation — no external calls.

## Function: `removeFromBlacklist(address account)`

The `removeFromBlacklist` function restores transfer eligibility for a previously blocked address.

## Inputs

- `account`



- **Control:** Set by a blacklister via BLACKLISTER\_ROLE.
- **Constraints:** Must currently be blacklisted.
- **Impact:** Sets `blacklisted[account] = false` and emits `RemovedFromBlacklist`.

## Branches and code coverage

### Intended branches

- Clears blacklist flag and emits event for a currently blacklisted account.

☒ Test coverage

### Negative behavior

- Reverts `AccountNotBlacklisted` if account is not blacklisted.

☒ Negative test

- Reverts if the caller lacks `BLACKLISTER_ROLE`.

☒ Negative test

## Function: `removeFromBlacklistBatch(address[] accounts)`

The `removeFromBlacklistBatch` function removes multiple addresses from the blacklist in one transaction.

## Inputs

- `accounts`
  - **Control:** Set by a blacklister via `BLACKLISTER_ROLE`.
  - **Constraints:** Each element must currently be blacklisted; empty array is a no-op.
  - **Impact:** Sets `blacklisted[account] = false` and emits `RemovedFromBlacklist` for each entry.

## Branches and code coverage

### Intended branches

- Iterates all entries, clears blacklist flag, and emits one event per address.

☒ Test coverage

- Succeeds with zero-length input without state changes.

☒ Test coverage

#### Negative behavior

- Reverts AccountNotBlacklisted on the first account that is not blacklisted.

☒ Negative test

- Reverts if the caller lacks BLACKLISTER\_ROLE.

☒ Negative test

### Function: removeFromWhitelist(address account)

The removeFromWhitelist function revokes primary-issuance mint eligibility for an address.

#### Inputs

- account
  - **Control:** Set by a whitelister via WHITELISTER\_ROLE.
  - **Constraints:** Must currently be whitelisted.
  - **Impact:** Sets whitelisted[account] = false and emits RemovedFromWhitelist — blocks future mint to that address.

### Branches and code coverage

#### Intended branches

- Clears whitelist flag and emits an event for a currently whitelisted account.

☒ Test coverage

#### Negative behavior

- Reverts AccountNotWhitelisted if account is not whitelisted.

☒ Negative test

- Reverts if the caller lacks WHITELISTER\_ROLE.

☒ Negative test

### Function: unpause ( )

The unpause function restores normal token operation after an emergency pause.

## Branches and code coverage

### Intended branches

- Clears the paused flag via `_unpause()` when the caller holds `PAUSER_ROLE`.
- ☒ Test coverage

### Negative behavior

- Reverts if the caller lacks `PAUSER_ROLE`.
- ☒ Negative test

## Function call analysis

- `this._unpause()` -> `this._getPausableStorage()`
  - **What is controllable?** Only `PAUSER_ROLE` holders reach this call.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
No return value is consumed. Premature unpause restores transfer and mint paths.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert leaves the contract paused. No external callbacks.

## 4.17. Module: TreasuryDelegate.sol

### Function: `_accrueInterest(address user)`

The `_accrueInterest` function wraps `_accrueInterestPaged(user, 0)` for unlimited daily catch-up plus conditional withheld merge.

### Inputs

- `user`
  - **Control:** Internal callers (swap paths) or public accrual functions.
  - **Constraints:** None at this layer.
  - **Impact:** Updates user yield state before credits change.

## Branches and code coverage

### Intended branches

- Delegates to paged accrual with `maxDays = 0` (unlimited daily loop).

☒ Test coverage

#### Negative behavior

- Inherits OOG risk from unbounded merge (L5).

☐ Negative test

#### Function: `_accrueInterestPaged(address user, uint256 maxDays)`

The `_accrueInterestPaged` function compounds main credits day by day through `targetDay = currentDay - 1` then merges mature withheld credits only when main credits are caught up.

#### Inputs

- `user`
  - **Control:** Internal/public accrual entry points.
  - **Constraints:** None.
  - **Impact:** Account whose multiplier, `lastAccrualDay`, and deque merge.
- `maxDays`
  - **Control:** Internal (0) or `accrueInterestPaged` caller.
  - **Constraints:** Limits only the daily compounding for loop.
  - **Impact:** Partial catch-up — merge still unbounded.

#### Branches and code coverage

##### Intended branches

- Updates `yieldMultiplier` and `lastAccrualDay` for processed days.
  - ☒ Test coverage
- Calls `_mergeWithheldCredits` when `mainCreditsCaughtUp` and withheld balance are `> 0`.
  - ☒ Test coverage
- Sets `isComplete` when caught up or next day rate missing.
  - ☒ Test coverage

##### Negative behavior

- Stops daily loop on missing `ratePosted` without merging if main is not caught up.
  - ☒ Negative test
- `_mergeWithheldCredits` OOG on large FIFO queues (L5).

❑ Negative test

**Function:** `_addWithheldCredit(address user, address _tokenAddress, uint256 _amount, uint256 _releaseTimestamp, uint256 _earningStartTimestamp, bool _isRWA)`

The `_addWithheldCredit` function appends one `WithheldCredits` deque record per call (no coalescing), increments `totalWithheldCredits`, and emits `PendingCreditsAdded`.

## Inputs

- `user`
  - **Control:** Internal callers only (swap paths).
  - **Constraints:** None at this layer.
  - **Impact:** Deque owner — shared vault/custody queues accumulate cross-user fees (L5).
- `_tokenAddress`
  - **Control:** Internal caller from `DelegateParams` asset address.
  - **Constraints:** None stored beyond bookkeeping.
  - **Impact:** Recorded backing token metadata per entry.
- `_amount`
  - **Control:** Internal caller (mint/credit amount).
  - **Constraints:** Expected to match paired Treasury mint where applicable.
  - **Impact:** Increments `totalWithheldCredits[user]` by full amount.
- `_releaseTimestamp`
  - **Control:** Internal caller (`block.timestamp + WITHHELD_PERIOD`).
  - **Constraints:** Typically 24h no-fee window.
  - **Impact:** FIFO merge gate in `_mergeWithheldCredits`.
- `_earningStartTimestamp`
  - **Control:** Internal caller (immediate for RWA, T+1 for stablecoin deposits).
  - **Constraints:** Drives `_calculateCreditMultiplier` path.
  - **Impact:** When yield begins accruing for the withheld record.
- `_isRWA`
  - **Control:** Internal caller.
  - **Constraints:** Chooses daily vs continuous yield math at merge.
  - **Impact:** RWA uses `_applyDailyRate` only — stablecoin uses partial first day.

## Branches and code coverage

### Intended branches

- pushBack one deque element and increase totalWithheldCredits.

☒ Test coverage

### Negative behavior

- Unbounded pushes enable gas grief on later merge/accrue (L5) — no per-address cap here.

☐ Negative test

**Function:** `_applyDailyRate(uint256 multiplier, uint256 startDay, uint256 currentDay)`

The `_applyDailyRate` function compounds `multiplier` by each posted day's `dailyCompoundingRate` from `startDay` inclusive to `currentDay` exclusive, stopping at the first gap.

### Inputs

- `multiplier`
  - **Control:** Caller-internal prior multiplier.
  - **Constraints:** WAD-scaled.
  - **Impact:** Starting value for compounding loop.
- `startDay`
  - **Control:** Caller-internal day index.
  - **Constraints:** `<= currentDay` expected.
  - **Impact:** First day applied.
- `currentDay`
  - **Control:** Usually `block.timestamp / 1 days`.
  - **Constraints:** Loop runs while `day < currentDay`.
  - **Impact:** Exclusive end day for compounding.

## Branches and code coverage

### Intended branches

- Multiplies by `WAD + dailyRate` for each posted day with positive rate.

☒ Test coverage

- Breaks on first day without ratePosted.

☑ Test coverage

#### Negative behavior

- Long gaps between posted days stop accrual until rates are backfilled.

☑ Negative test

### Function: `_applyPartialDayRate(uint256 currentMultiplier, uint256 day, uint256 secondsElapsed)`

The `_applyPartialDayRate` function applies pro rata compounding for a specific posted day without stale-rate fallback (used for withheld stablecoin merge math).

#### Inputs

- `currentMultiplier`
  - **Control:** Internal caller.
  - **Constraints:** WAD-scaled.
  - **Impact:** Starting multiplier.
- `day`
  - **Control:** Internal earning-start day.
  - **Constraints:** Requires `ratePosted[day]`.
  - **Impact:** Rate source — no fallback if missing.
- `secondsElapsed`
  - **Control:** Internal partial-day length.
  - **Constraints:** Zero skips accrual.
  - **Impact:** Fraction of daily rate applied.

#### Branches and code coverage

##### Intended branches

- Returns input when seconds zero or day not posted.
 

☑ Test coverage
- Applies  $(\text{dailyRate} * \text{secondsElapsed}) / \text{SECONDS\_PER\_DAY}$  via `mulWad`.
 

☑ Test coverage

##### Negative behavior

- Missing start-day rate blocks stablecoin withheld merge in `_mergeWithheldCredits`.

☒ Negative test

### Function: `_applyPartialDayRateWithFallback(uint256 currentMultiplier, uint256 day, uint256 secondsElapsed)`

The `_applyPartialDayRateWithFallback` function applies a pro rata daily rate for stablecoin redemptions, falling back to `lastRatePostingDay` when `day` is unposted and capping seconds via `maxFallbackRateSeconds`.

#### Inputs

- `currentMultiplier`
  - **Control:** Caller-internal accrued multiplier.
  - **Constraints:** WAD-scaled.
  - **Impact:** Base before partial uplift.
- `day`
  - **Control:** Caller passes `lastRatePostingDay` or current day.
  - **Constraints:** Uses posted day or fallback.
  - **Impact:** Rate lookup key.
- `secondsElapsed`
  - **Control:** Derived from `block.timestamp` in callers.
  - **Constraints:** Capped by `_capFallbackRateSeconds`.
  - **Impact:** Length of partial-day accrual.

#### Branches and code coverage

##### Intended branches

- Returns unchanged multiplier when `secondsElapsed == 0` or rate is zero.
  - ☒ Test coverage
- Uses `lastRatePostingDay` rate when day not posted.
  - ☒ Test coverage

##### Negative behavior

- Uncapped fallback (when `maxFallbackRateSeconds == 0`) extends yield through long stale periods.
  - ☐ Negative test



## Function: `_calculateCreditMultiplier(uint256 earningStartTimestamp, bool isRWA)`

The `_calculateCreditMultiplier` function computes a WAD multiplier from `earningStartTimestamp` to now using RWA daily-only or stablecoin continuous (partial first day plus full days) paths without stale-rate fallback.

### Inputs

- `earningStartTimestamp`
  - **Control:** Stored on withheld record at deposit.
  - **Constraints:** Future timestamp yields WAD.
  - **Impact:** Start of yield window for merge blending.
- `isRWA`
  - **Control:** Stored flag at deposit.
  - **Constraints:** Selects RWA vs stablecoin math branch.
  - **Impact:** Whether partial-day accrual applies and which days compound.

### Branches and code coverage

#### Intended branches

- Returns WAD when earning has not started yet.
  - ☒ Test coverage
- RWA — `_applyDailyRate` from `startDay` to `currentDay`.
  - ☒ Test coverage
- Stablecoin — partial start day then daily compounding when `currentDay > startDay`.
  - ☒ Test coverage

#### Negative behavior

- Stops at first missing `ratePosted` day in `_applyDailyRate` (no fallback here).
  - ☒ Negative test

## Function: `_capFallbackRateSeconds(uint256 secondsElapsed)`

The `_capFallbackRateSeconds` function limits partial-day fallback seconds to `maxFallbackRateSeconds` when that config is nonzero.

## Inputs

- secondsElapsed
  - **Control:** Internal caller from timestamp math.
  - **Constraints:** None.
  - **Impact:** Elapsed seconds before rate application.

## Branches and code coverage

### Intended branches

- Returns secondsElapsed when cap is zero or elapsed is within cap.
  - ☒ Test coverage
- Returns maxFallbackRateSeconds when elapsed exceeds cap.
  - ☒ Test coverage

### Negative behavior

- No revert paths.
  - ☐ Negative test

## Function: `_checkDailyRate(uint256 grossRate)`

The `_checkDailyRate` function enforces hard 100% APY and optional `maxDailyRate` on gross inputs before fee subtraction.

## Inputs

- grossRate
  - **Control:** Rate poster or operator depending on caller.
  - **Constraints:** Must be  $\leq$  WAD; if `maxDailyRate`  $\neq$  0, must be  $\leq$  `maxDailyRate`.
  - **Impact:** Gates whether a rate write proceeds.

## Branches and code coverage

### Intended branches

- Succeeds silently when within bounds.
  - ☒ Test coverage

### Negative behavior

- Reverts RateTooHigh when above WAD or configured cap.

☒ Negative test

### Function: `_convertApyToDailyRate(uint256 apy)`

The `_convertApyToDailyRate` function converts WAD net APY to per-day compounding rate via  $(1 + \text{apy})^{(1/365)} - 1$ .

### Inputs

- `apy`
  - **Control:** From posted/corrected net rates ( $\leq$  WAD).
  - **Constraints:** Zero APY returns zero rate.
  - **Impact:** Stored in `dailyCompoundingRates` and used in all compounding loops.

### Branches and code coverage

#### Intended branches

- Returns 0 for zero APY.
  - ☒ Test coverage
- Uses `FixedPointMathLib.powWad` for positive APY.
  - ☒ Test coverage

#### Negative behavior

- Returns 0 when `dailyFactor`  $\leq$  WAD after pow (degenerate / rounding).
  - ☒ Negative test

### Function: `_exchangeStablecoinsMint(DelegateParams params)`

The `_exchangeStablecoinsMint` function pulls whitelisted stablecoin, adds T+1 withheld credits for the user (`isRWA=false`), mints Treasury to receiver, and records fee withheld credits on vault/custody.

## Inputs

- params
  - **Control:** MultiliquidSwap via exchangeStablecoins only.
  - **Constraints:** onlyWhitelistedStablecoin(assetIn).
  - **Impact:** Stablecoin in, Treasury mint out, per-fee deque growth without coalescing.

## Branches and code coverage

### Intended branches

- Accrues user interest, transfers stablecoin to custody, withholds user credits with T+1 earning.
  - ☒ Test coverage
- Mints Treasury to receiver and optional fee recipients.
  - ☒ Test coverage

### Negative behavior

- Reverts on non-whitelisted stablecoin or failed ERC-20/Treasury calls.
  - ☒ Negative test
- Accrual/merge OOG can revert before mint (L5).
  - ☐ Negative test

## Function: `_exchangeStablecoinsRedeem(DelegateParams params)`

The `_exchangeStablecoinsRedeem` function burns Treasury for whitelisted stablecoin: accrues, reduces credits, applies partial-day fallback multiplier on stablecoin exit, burns, mints fees, and transfers stablecoin out.

## Inputs

- params
  - **Control:** MultiliquidSwap via exchangeStablecoins.
  - **Constraints:** onlyWhitelistedStablecoin(assetOut) — sufficient credits.
  - **Impact:** Credit reduction, yield-bearing burn pricing, custody stablecoin transfer.

## Branches and code coverage

### Intended branches

- Enforces `InsufficientCredits` against `credits + totalWithheldCredits`.
  - ☒ Test coverage
- Applies `_applyPartialDayRateWithFallback` for stablecoin withdrawal yield on main credits.
  - ☒ Test coverage
- Burns Treasury and transfers stablecoin from `stablecoinCustodyAddress`.
  - ☒ Test coverage

### Negative behavior

- Reverts on insufficient credits, failed burn, or transfer.
  - ☒ Negative test
- Accrual/merge OOG before redeem (L5).
  - ☐ Negative test

### Function: `_getAmountForTargetValue(address user, uint256 targetDollars, bool stablecoinWithdrawal)`

The `_getAmountForTargetValue` function solves for Treasury amount needed to reach `targetDollars`, using withheld 1:1 then `divWadUp` on main credits with optional partial-day multiplier.

### Inputs

- `user`
  - **Control:** View caller.
  - **Constraints:** Must have credits to satisfy target or revert.
  - **Impact:** Account debited in the sizing calculation.
- `targetDollars`
  - **Control:** View caller.
  - **Constraints:** Positive effective value desired.
  - **Impact:** Target effective redemption value.
- `stablecoinWithdrawal`
  - **Control:** View caller.
  - **Constraints:** Boolean.

- **Impact:** Applies fallback partial-day rate to multiplier.

## Branches and code coverage

### Intended branches

- Returns withheld-only Treasury when target fits in `totalWithheldCredits`.  
☒ Test coverage
- Computes `neededFromMain` via `divWadUp` and sums withheld plus main Treasury.  
☒ Test coverage

### Negative behavior

- Reverts `InsufficientCredits` when main leg cannot satisfy remainder.  
☒ Negative test

## Function: `_getDailyCompoundingRate(uint256 day)`

The `_getDailyCompoundingRate` function reads cached `dailyCompoundingRates` or derives from stored APY for legacy rows.

### Inputs

- `day`
  - **Control:** Internal day index from accrual loops.
  - **Constraints:** None.
  - **Impact:** Rate used for that day's `mulWad` compounding.

## Branches and code coverage

### Intended branches

- Returns cached compounding rate when nonzero.  
☒ Test coverage
- Falls back to `_convertApyToDailyRate(dailyRates[day])` when cache is empty but APY is posted.  
☒ Test coverage

### Negative behavior

- Returns zero when no rate is stored for the day.

☒ Negative test

**Function: `_getYieldAmount(address user, uint256 redeemValue, bool stablecoinWithdrawal)`**

The `_getYieldAmount` function computes effective redemption value: all withheld at 1:1, then main credits at blended multiplier with optional stablecoin partial-day fallback.

### Inputs

- `user`
  - **Control:** View caller via `getYieldAmount`.
  - **Constraints:** None.
  - **Impact:** Credit/multiplier source.
- `redeemValue`
  - **Control:** View caller.
  - **Constraints:** None enforced here.
  - **Impact:** Treasury units being valued.
- `stablecoinWithdrawal`
  - **Control:** View caller.
  - **Constraints:** Boolean.
  - **Impact:** Toggles partial-day fallback on main leg.

### Branches and code coverage

#### Intended branches

- Early zero return — 1:1 withheld-only path, blended main path.

☒ Test coverage

#### Negative behavior

- Does not revert on insufficient credits — may return partial value semantics via zero consumed edge case.

☐ Negative test

**Function: `_mergeWithheldCredits(address user)`**

The `_mergeWithheldCredits` function FIFO-merges mature withheld deque records into main credits, blending `yieldMultiplier` by weighted average when applied-through days align.

## Inputs

- `user`
  - **Control:** Internal via `_accrueInterestPaged` only.
  - **Constraints:** Requires `mainCreditsCaughtUp` at the caller.
  - **Impact:** Whose deque is drained and credits/multiplier updated.

## Branches and code coverage

### Intended branches

- Pops front records past `releaseTimestamp` and earning-start gates.
  - ☒ Test coverage
- Blends merged value into `yieldMultiplier` and increases credits.
  - ☒ Test coverage
- Stops when applied-through days differ between batchable credits.
  - ☒ Test coverage

### Negative behavior

- Unbounded while over deque can OOG with many one-record fee deposits (L5 grief).
  - ☐ Negative test
- Stablecoin records wait for `ratePosted[earningStartDay]` before merge.
  - ☒ Negative test

## Function: `_reduceWithheldAndCredits(address user, uint256 amount)`

The `_reduceWithheldAndCredits` function consumes redemption amount from withheld deque tail (LIFO, no yield) then main credits, reverting if principal insufficient.

## Inputs

- `user`
  - **Control:** Internal redeem paths.
  - **Constraints:** None.
  - **Impact:** Whose deque and main credits shrink.
- `amount`
  - **Control:** Internal from `params.assetInAmount`.
  - **Constraints:** Must be `<= totalWithheldCredits + credits` at the caller.



- **Impact:** Treasury burn / credit reduction quantity.

## Branches and code coverage

### Intended branches

- Reduces withheld from back with full or partial pop/push.
  - ☒ Test coverage
- Reduces main credits for remainder and emits reduction events.
  - ☒ Test coverage

### Negative behavior

- Reverts `InsufficientCredits` if main credits cannot cover remainder.
  - ☒ Negative test
- Large withheld deque increases gas for LIFO walk (L5).
  - ☐ Negative test

## Function: `_withheldCreditAppliedThroughDay(uint256 earningStartTimestamp, uint256 currentDay)`

The `_withheldCreditAppliedThroughDay` function returns the last day included in a withheld credit's standalone multiplier, capping before predeposit days when start-day rate is missing.

### Inputs

- `earningStartTimestamp`
  - **Control:** From withheld record.
  - **Constraints:** Used to derive `earningStartDay`.
  - **Impact:** Determines conservative applied-through day before merge batching.
- `currentDay`
  - **Control:** `block.timestamp / 1 days` at merge time.
  - **Constraints:** None.
  - **Impact:** Upper bound `currentDay - 1` vs `lastRatePostingDay`.

## Branches and code coverage

### Intended branches

- Returns `earningStartDay - 1` when start day beyond `lastRatePostingDay`.

☒ Test coverage

- Otherwise returns `min(currentDay - 1, lastRatePostingDay)`.

☒ Test coverage

#### Negative behavior

- No revert paths.

☐ Negative test

### Function: `accrueInterest(address user)`

The `accrueInterest` function permissionlessly compounds main credits through posted days and merges mature withheld credits when caught up (`maxDays = 0` unlimited).

#### Inputs

- `user`
  - **Control:** Arbitrary caller.
  - **Constraints:** Contract not paused — `nonReentrant`.
  - **Impact:** Whose `yieldMultiplier`, `lastAccrualDay`, `credits`, and `withheld deque` are updated.

### Branches and code coverage

#### Intended branches

- Delegates to `_accrueInterestPaged(user, 0)` for full catch-up.

☒ Test coverage

- Merges withheld FIFO records when `mainCreditsCaughtUp`.

☒ Test coverage

#### Negative behavior

- Reverts when paused or reentered.

☒ Negative test

- Can OOG when `_mergeWithheldCredits` scans a large deque (L5 grief on vault/custody).

☐ Negative test

## Function call analysis

- `this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) -> this._getDailyCompoundingRate(day) -> this._convertApyToDailyRate(apy) -> FixedPointMathLib.powWad(TreasuryDelegate.WAD_INT + apyInt, TreasuryDelegate.ONE_OVER_DAYS_PER_YEAR_WAD)`
  - **What is controllable?** The APY input comes from posted dailyRates (net of managementFee) or is derived during initialization; it is bounded by `_checkDailyRate` and `WAD`.
  - **If the return value is controllable, how is it used and how can it go wrong?** The per-day compounding rate is stored or multiplied into `yieldMultiplier`. An admin-posted rate affects all users uniformly; there is no per-call caller control.
  - **What happens if it reverts, reenters or does other unusual control flow?** A `powWad` revert aborts the enclosing transaction. This is a pure library call with no external callbacks.
- `this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) -> FixedPointMathLib.mulWad(userMultiplier, TreasuryDelegate.WAD + dailyRate)`
  - **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?** The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- `this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> WithheldCreditsDeque.empty(this.withheldCredits[user])`
  - **What is controllable?** Whether the deque is empty depends on prior `_addWithheldCredit` / `_reduceWithheldAndCredits` history for user.
  - **If the return value is controllable, how is it used and how can it go wrong?** Gates the merge/reduction loop. A nonempty deque triggers further processing and gas cost scaling with record count.
  - **What happens if it reverts, reenters or does other unusual control flow?** View on storage — no external call. False empty skips merge safely.
- `this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> WithheldCreditsDeque.front(this.withheldCredits[user])`
  - **What is controllable?** The front record is the oldest withheld credit — its

fields were set at deposit time by MultiliquidSwap-driven paths.

- **If the return value is controllable, how is it used and how can it go wrong?**  
Determines whether FIFO merge proceeds (release timestamp, earning start, rate availability). Out-of-order maturity stops the loop without corruption.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Memory read from deque storage — no external callback.

```
• this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) ->
 this._mergeWithheldCredits(user) ->
 WithheldCreditsDeque.popFront(this.withheldCredits[user])
```

- **What is controllable?** Only reached when front credit passes release, earning-start, and applied-through-day checks.
- **If the return value is controllable, how is it used and how can it go wrong?**  
Removes one deque record per iteration. Many records make the unbounded while loop gas-heavy (L5 grief on shared vault/custody queues).
- **What happens if it reverts, reenters or does other unusual control flow?**  
Storage update — revert rolls back all pops in the transaction.

```
• this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) ->
 this._mergeWithheldCredits(user) ->
 this._calculateCreditMultiplier(credit.earningStartTimestamp,
 credit.isRWA) -> this._applyDailyRate(multiplier, startDay, currentDay)
 -> FixedPointMathLib.mulWad(multiplier, TreasuryDelegate.WAD + dailyRate)
```

- **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
- **If the return value is controllable, how is it used and how can it go wrong?**  
The product updates yieldMultiplier[user] or intermediate merge math. Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.

```
• this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) ->
 this._mergeWithheldCredits(user) ->
 this._calculateCreditMultiplier(credit.earningStartTimestamp,
 credit.isRWA) -> this._applyPartialDayRate(multiplier, startDay,
 firstDaySeconds) -> FixedPointMathLib.mulWad(currentMultiplier,
 TreasuryDelegate.WAD + partialRate)
```

- **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
- **If the return value is controllable, how is it used and how can it go wrong?**  
The product updates yieldMultiplier[user] or intermediate merge math.

Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.

- `this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> FixedPointMathLib.mulWad(credit.credits, creditMultiplier)`

- **What is controllable?** `credit.credits` and `creditMultiplier` are deque records written by prior swap paths; the caller does not pick them at accrual time.

- **If the return value is controllable, how is it used and how can it go wrong?**

The product contributes to `totalMergedValue` for the weighted-average blend into main credits. Incorrect multipliers would require bad posted rates or corrupt deque state.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math inside merge — failure reverts the entire accrual with no partial deque progress.

- `this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> FixedPointMathLib.mulWad(userCredits, this.yieldMultiplier[user])`

- **What is controllable?** `userCredits` and `yieldMultiplier[user]` reflect prior deposits and accrual — not directly set by the current external caller.

- **If the return value is controllable, how is it used and how can it go wrong?**

Used to compute `oldValue` before blending merged withheld credits. A stale multiplier understates or overstates the blend until accrual catches up.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math — revert aborts merge.

- `this._accrueInterest(user) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> FixedPointMathLib.divWad(oldValue + totalMergedValue, newCredits)`

- **What is controllable?** Numerator sums prior main-credit value and merged withheld value; denominator is new total credits after merge.

- **If the return value is controllable, how is it used and how can it go wrong?**

Becomes the new `yieldMultiplier[user]`. Division truncates — extreme queue sizes increase gas but not caller control over the formula.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math. On OOG, during the surrounding merge loop, the entire transaction reverts with no deque progress (L5 grief).

## Function: accrueInterestPaged(address user, uint256 maxDays)

The accrueInterestPaged function accrues up to maxDays daily compounding steps (0 = unlimited) then may merge withheld credits, and it returns progress flags for iterative catch-up.

### Inputs

- user
  - **Control:** Arbitrary caller.
  - **Constraints:** Not paused — nonReentrant.
  - **Impact:** Account whose multiplier and deque are updated.
- maxDays
  - **Control:** Arbitrary caller.
  - **Constraints:** 0 means no day cap on the compounding loop only.
  - **Impact:** Limits daily loop iterations — does not paginate \_mergeWithheldCredits (L5).

### Branches and code coverage

#### Intended branches

- Returns (daysAccrued, isComplete) from \_accrueInterestPaged.
  - ☒ Test coverage
- Stops daily loop at first missing ratePosted day or maxDays limit.
  - ☒ Test coverage

#### Negative behavior

- Reverts when paused or reentered.
  - ☒ Negative test
- Merge step remains unbounded and can OOG even when maxDays is small (L5).
  - ☐ Negative test

### Function call analysis

- this.\_accrueInterestPaged(user, maxDays) -> this.\_getDailyCompoundingRate(day) -> this.\_convertApyToDailyRate(apy) -> FixedPointMathLib.powWad(TreasuryDelegate.WAD\_INT + apyInt, TreasuryDelegate.ONE\_OVER\_DAYS\_PER\_YEAR\_WAD)

- **What is controllable?** The APY input comes from posted dailyRates (net of managementFee) or is derived during initialization; it is bounded by `_checkDailyRate` and `WAD`.
  - **If the return value is controllable, how is it used and how can it go wrong?** The per-day compounding rate is stored or multiplied into `yieldMultiplier`. An admin-posted rate affects all users uniformly; there is no per-call caller control.
  - **What happens if it reverts, reenters or does other unusual control flow?** A `powWad` revert aborts the enclosing transaction. This is a pure library call with no external callbacks.
- `this._accrueInterestPaged(user, maxDays) ->`  
`FixedPointMathLib.mulWad(userMultiplier, TreasuryDelegate.WAD +`  
`dailyRate)`
    - **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
    - **If the return value is controllable, how is it used and how can it go wrong?** The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.
    - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
  - `this._accrueInterestPaged(user, maxDays) ->`  
`this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.empty(this.withheldCredits[user])`
    - **What is controllable?** Whether the deque is empty depends on prior `_addWithheldCredit` / `_reduceWithheldAndCredits` history for user.
    - **If the return value is controllable, how is it used and how can it go wrong?** Gates the merge/reduction loop. A nonempty deque triggers further processing and gas cost scaling with record count.
    - **What happens if it reverts, reenters or does other unusual control flow?** View on storage — no external call. False empty skips merge safely.
  - `this._accrueInterestPaged(user, maxDays) ->`  
`this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.front(this.withheldCredits[user])`
    - **What is controllable?** The front record is the oldest withheld credit; its fields were set at deposit time by MultiliquidSwap-driven paths.
    - **If the return value is controllable, how is it used and how can it go wrong?** Determines whether FIFO merge proceeds (release timestamp, earning start, rate availability). Out-of-order maturity stops the loop without corruption.
    - **What happens if it reverts, reenters or does other unusual control flow?**

Memory read from deque storage — no external callback.

- `this._accrueInterestPaged(user, maxDays) ->`  
`this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.popFront(this.withheldCredits[user])`
  - **What is controllable?** Only reached when front credit passes release, earning-start, and applied-through-day checks.
  - **If the return value is controllable, how is it used and how can it go wrong?** Removes one deque record per iteration. Many records make the unbounded while loop gas-heavy (L5 grief on shared vault/custody queues).
  - **What happens if it reverts, reenters or does other unusual control flow?** Storage update — revert rolls back all pops in the transaction.
- `this._accrueInterestPaged(user, maxDays) ->`  
`this._mergeWithheldCredits(user) ->`  
`this._calculateCreditMultiplier(credit.earningStartTimestamp,`  
`credit.isRWA) -> this._applyDailyRate(multiplier, startDay, currentDay)`  
`-> FixedPointMathLib.mulWad(multiplier, TreasuryDelegate.WAD + dailyRate)`
  - **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?** The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- `this._accrueInterestPaged(user, maxDays) ->`  
`this._mergeWithheldCredits(user) ->`  
`this._calculateCreditMultiplier(credit.earningStartTimestamp,`  
`credit.isRWA) -> this._applyPartialDayRate(multiplier, startDay,`  
`firstDaySeconds) -> FixedPointMathLib.mulWad(currentMultiplier,`  
`TreasuryDelegate.WAD + partialRate)`
  - **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?** The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- `this._accrueInterestPaged(user, maxDays) ->`



```
this._mergeWithheldCredits(user) ->
FixedPointMathLib.mulWad(credit.credits, creditMultiplier)
```

- **What is controllable?** credit.credits and creditMultiplier are deque records written by prior swap paths; the caller does not pick them at accrual time.
- **If the return value is controllable, how is it used and how can it go wrong?** The product contributes to totalMergedValue for the weighted-average blend into main credits. Incorrect multipliers would require bad posted rates or corrupt deque state.
- **What happens if it reverts, reenters or does other unusual control flow?** Pure math inside merge — failure reverts the entire accrual with no partial deque progress.
- this.\_accrueInterestPaged(user, maxDays) ->
 

```
this._mergeWithheldCredits(user) -> FixedPointMathLib.mulWad(userCredits,
this.yieldMultiplier[user])
```

  - **What is controllable?** userCredits and yieldMultiplier[user] reflect prior deposits and accrual — not directly set by the current external caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** Used to compute oldValue before blending merged withheld credits. A stale multiplier understates or overstates the blend until accrual catches up.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — revert aborts merge.
- this.\_accrueInterestPaged(user, maxDays) ->
 

```
this._mergeWithheldCredits(user) -> FixedPointMathLib.divWad(oldValue +
totalMergedValue, newCredits)
```

  - **What is controllable?** Numerator sums prior main-credit value and merged withheld value — denominator is new total credits after merge.
  - **If the return value is controllable, how is it used and how can it go wrong?** Becomes the new yieldMultiplier[user]. Division truncates — extreme queue sizes increase gas but not caller control over the formula.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math. On OOG, during the surrounding merge loop, the entire transaction reverts with no deque progress (L5 grief).

### Function: batchAccrueInterest(address[] users)

The batchAccrueInterest function loops accrueInterest for each address in one transaction.

### Inputs

- users

- **Control:** Arbitrary caller supplies the array.
- **Constraints:** Not paused — nonReentrant.
- **Impact:** Ordered list of accounts to accrue — total gas is the sum of per-user accrual/merge costs.

## Branches and code coverage

### Intended branches

- Calls `_accrueInterest` for each element until the array ends.
- ☒ Test coverage

### Negative behavior

- Reverts when paused or reentered.
- ☒ Negative test
- Any single user OOG on merge reverts the entire batch.
- ☐ Negative test

## Function call analysis

- `this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) -> this._getDailyCompoundingRate(day) -> this._convertApyToDailyRate(apy) -> FixedPointMathLib.powWad(TreasuryDelegate.WAD_INT + apyInt, TreasuryDelegate.ONE_OVER_DAYS_PER_YEAR_WAD)`
  - **What is controllable?** The APY input comes from posted dailyRates (net of managementFee) or is derived during initialization — it is bounded by `_checkDailyRate` and `WAD`.
  - **If the return value is controllable, how is it used and how can it go wrong?** The per-day compounding rate is stored or multiplied into `yieldMultiplier`. An admin-posted rate affects all users uniformly; there is no per-call caller control.
  - **What happens if it reverts, reenters or does other unusual control flow?** A `powWad` revert aborts the enclosing transaction. This is a pure library call with no external callbacks.
- `this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) -> FixedPointMathLib.mulWad(userMultiplier, TreasuryDelegate.WAD + dailyRate)`
  - **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?**

The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.

- **What happens if it reverts, reenters or does other unusual control flow?**  
Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.

```
• this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) ->
 this._mergeWithheldCredits(user) ->
 WithheldCreditsDeque.empty(this.withheldCredits[user])
```

- **What is controllable?** Whether the deque is empty depends on prior `_addWithheldCredit / _reduceWithheldAndCredits` history for user.
- **If the return value is controllable, how is it used and how can it go wrong?**  
Gates the merge/reduction loop. A nonempty deque triggers further processing and gas cost scaling with record count.
- **What happens if it reverts, reenters or does other unusual control flow?**  
View on storage — no external call. False empty skips merge safely.

```
• this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) ->
 this._mergeWithheldCredits(user) ->
 WithheldCreditsDeque.front(this.withheldCredits[user])
```

- **What is controllable?** The front record is the oldest withheld credit — its fields were set at deposit time by MultiliquidSwap-driven paths.
- **If the return value is controllable, how is it used and how can it go wrong?**  
Determines whether FIFO merge proceeds (release timestamp, earning start, rate availability). Out-of-order maturity stops the loop without corruption.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Memory read from deque storage — no external callback.

```
• this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) ->
 this._mergeWithheldCredits(user) ->
 WithheldCreditsDeque.popFront(this.withheldCredits[user])
```

- **What is controllable?** Only reached when front credit passes release, earning-start, and applied-through-day checks.
- **If the return value is controllable, how is it used and how can it go wrong?**  
Removes one deque record per iteration. Many records make the unbounded while loop gas-heavy (L5 grief on shared vault/custody queues).
- **What happens if it reverts, reenters or does other unusual control flow?**  
Storage update — revert rolls back all pops in the transaction.

```
• this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) ->
 this._mergeWithheldCredits(user) ->
 this._calculateCreditMultiplier(credit.earningStartTimestamp,
 credit.isRWA) -> this._applyDailyRate(multiplier, startDay, currentDay)
 -> FixedPointMathLib.mulWad(multiplier, TreasuryDelegate.WAD + dailyRate)
```

- **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?** The product updates yieldMultiplier[user] or intermediate merge math. Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- `this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> this._calculateCreditMultiplier(credit.earningStartTimestamp, credit.isRWA) -> this._applyPartialDayRate(multiplier, startDay, firstDaySeconds) -> FixedPointMathLib.mulWad(currentMultiplier, TreasuryDelegate.WAD + partialRate)`
    - **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
    - **If the return value is controllable, how is it used and how can it go wrong?** The product updates yieldMultiplier[user] or intermediate merge math. Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.
    - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
  - `this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> FixedPointMathLib.mulWad(credit.credits, creditMultiplier)`
    - **What is controllable?** credit.credits and creditMultiplier are deque records written by prior swap paths; the caller does not pick them at accrual time.
    - **If the return value is controllable, how is it used and how can it go wrong?** The product contributes to totalMergedValue for the weighted-average blend into main credits. Incorrect multipliers would require bad posted rates or corrupt deque state.
    - **What happens if it reverts, reenters or does other unusual control flow?** Pure math inside merge — failure reverts the entire accrual with no partial deque progress.
  - `this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> FixedPointMathLib.mulWad(userCredits, this.yieldMultiplier[user])`
    - **What is controllable?** userCredits and yieldMultiplier[user] reflect

- prior deposits and accrual — not directly set by the current external caller.
  - If the return value is controllable, how is it used and how can it go wrong?**  
Used to compute `oldValue` before blending merged withheld credits. A stale multiplier understates or overstates the blend until accrual catches up.
  - What happens if it reverts, reenters or does other unusual control flow?**  
Pure math — revert aborts merge.
- `this._accrueInterest(users[i]) -> this._accrueInterestPaged(user, 0) -> this._mergeWithheldCredits(user) -> FixedPointMathLib.divWad(oldValue + totalMergedValue, newCredits)`
  - What is controllable?** Numerator sums prior main-credit value and merged withheld value; the denominator is new total credits after merge.
  - If the return value is controllable, how is it used and how can it go wrong?**  
Becomes the new `yieldMultiplier[user]`. Division truncates; extreme queue sizes increase gas but not caller control over the formula.
  - What happens if it reverts, reenters or does other unusual control flow?**  
Pure math. On OOG, during the surrounding merge loop, the entire transaction reverts with no deque progress (L5 grief).

## Function: constructor ( )

The constructor function disables initializers on the TreasuryDelegate implementation so the logic contract cannot be initialized outside the proxy deployment path.

## Branches and code coverage

### Intended branches

- Calls `_disableInitializers()` so only the proxy initialize can wire Treasury yield state.
- ☒ Test coverage

### Negative behavior

- No revert paths — the constructor always succeeds.
- ☐ Negative test

## Function: correctDailyRate(uint256 dayNumber, uint256 grossRate)

The `correctDailyRate` function lets operators rewrite a past day's gross APY within the existing posted series without advancing `lastRatePostingDay`.

## Inputs

- `dayNumber`
  - **Control:** `OPERATOR_ROLE` only (not rate poster).
  - **Constraints:** `firstRateDay`  $\leq$  `dayNumber`  $\leq$  `lastRatePostingDay`.
  - **Impact:** Which historical day's net/compounding rates are overwritten.
- `grossRate`
  - **Control:** Operator.
  - **Constraints:** Same bounds as `postDailyRate` and  $\geq$  `managementFee`.
  - **Impact:** Changes accrual math for users who have not yet accrued through that day — may require `setUserYieldMultiplier` remediation.

## Branches and code coverage

### Intended branches

- Overwrites `dailyRates` / `dailyCompoundingRates` and emits `DailyRateCorrected`.
  - ☒ Test coverage
- Does not move `lastRatePostingDay` forward.
  - ☒ Test coverage

### Negative behavior

- Reverts `CannotCorrectBeforeFirstRate` or `CannotPostFutureRate` outside the series.
  - ☒ Negative test
- Reverts on rate bound / management-fee violations.
  - ☒ Negative test
- Reverts without `OPERATOR_ROLE`.
  - ☒ Negative test

## Function call analysis

- `this._convertApyToDailyRate(netRate) -> FixedPointMathLib.powWad(TreasuryDelegate.WAD_INT + apyInt, TreasuryDelegate.ONE_OVER_DAYS_PER_YEAR_WAD)`
  - **What is controllable?** The APY input comes from posted `dailyRates` (net of `managementFee`) or is derived during initialization; it is bounded by `_checkDailyRate` and `WAD`.

- **If the return value is controllable, how is it used and how can it go wrong?**  
The per-day compounding rate is stored or multiplied into `yieldMultiplier`. An admin-posted rate affects all users uniformly; there is no per-call caller control.
- **What happens if it reverts, reenters or does other unusual control flow?** A `powWad` revert aborts the enclosing transaction. This is a pure library call with no external callbacks.

### Function: `deployStablecoin(DelegateParams params)`

The `deployStablecoin` function handles RWA → Treasury minting: accrues user yield, pulls RWA into balance-sheet custody, records withheld credits (immediate earning, 24h release), and mints Treasury to receiver and fee recipients.

### Inputs

- `params`
  - **Control:** `MultiliquidSwap` constructs `DelegateParams` from the user swap — not directly caller-supplied.
  - **Constraints:** `onlyRole(MULTILIQUID_SWAP_CONTRACT)`, whitelisted RWA, blacklist checks, custody set, not paused, and `nonReentrant`.
  - **Impact:** Controls user/receiver/vault, RWA in/out amounts, protocol fee, and Treasury mint destinations.

### Branches and code coverage

#### Intended branches

- Calls `_accrueInterest(params.user)` before crediting new deposits.
  - ☒ Test coverage
- Transfers `assetInAmount` RWA from the user to `balanceSheetCustodyAddress`.
  - ☒ Test coverage
- Pushes one withheld deque record per user deposit (`isRWA=true`, immediate `earningStartTimestamp`, `WITHHELD_PERIOD` release).
  - ☒ Test coverage
- Mints protocol fee Treasury to vault with matching withheld credit when `protocolFeeAmt > 0`.
  - ☒ Test coverage
- Mints `assetOutAmount` Treasury to receiver.
  - ☒ Test coverage

### Negative behavior

- Reverts when the caller lacks MULTILIQUEID\_SWAP\_CONTRACT, contract is paused, or custody is unset.
  - ☒ Negative test
- Reverts on blacklisted user/receiver or non-whitelisted RWA.
  - ☒ Negative test
- Reverts when RWA safeTransferFrom fails or Treasury mint reverts.
  - ☒ Negative test
- Large withheld queues on shared addresses can make \_accrueInterest OOG before mint (L5 grief).
  - ☐ Negative test

### Function call analysis

- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._getDailyCompoundingRate(day) -> this._convertApyToDailyRate(apy)`  
`-> FixedPointMathLib.powWad(TreasuryDelegate.WAD_INT + apyInt,`  
`TreasuryDelegate.ONE_OVER_DAYS_PER_YEAR_WAD)`
  - **What is controllable?** The APY input comes from posted dailyRates (net of managementFee) or is derived during initialization; it is bounded by \_checkDailyRate and WAD.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
 The per-day compounding rate is stored or multiplied into yieldMultiplier. An admin-posted rate affects all users uniformly; there is no per-call caller control.
  - **What happens if it reverts, reenters or does other unusual control flow?** A powWad revert aborts the enclosing transaction. This is a pure library call with no external callbacks.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> FixedPointMathLib.mulWad(userMultiplier, TreasuryDelegate.WAD +`  
`dailyRate)`
  - **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
 The product updates yieldMultiplier[user] or intermediate merge math. Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
 Pure math — a revert rolls back the whole accrual or swap. No reentrancy



- surface.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.empty(this.withheldCredits[user])`
    - **What is controllable?** Whether the deque is empty depends on prior `_addWithheldCredit / _reduceWithheldAndCredits` history for user.
    - **If the return value is controllable, how is it used and how can it go wrong?** Gates the merge/reduction loop. A nonempty deque triggers further processing and gas cost scaling with record count.
    - **What happens if it reverts, reenters or does other unusual control flow?** View on storage — no external call. False empty skips merge safely.
  - `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.front(this.withheldCredits[user])`
    - **What is controllable?** The front record is the oldest withheld credit; its fields were set at deposit time by MultiliquidSwap-driven paths.
    - **If the return value is controllable, how is it used and how can it go wrong?** Determines whether FIFO merge proceeds (release timestamp, earning start, rate availability). Out-of-order maturity stops the loop without corruption.
    - **What happens if it reverts, reenters or does other unusual control flow?** Memory read from deque storage — no external callback.
  - `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.popFront(this.withheldCredits[user])`
    - **What is controllable?** Only reached when front credit passes release, earning-start, and applied-through-day checks.
    - **If the return value is controllable, how is it used and how can it go wrong?** Removes one deque record per iteration. Many records make the unbounded while loop gas-heavy (L5 grief on shared vault/custody queues).
    - **What happens if it reverts, reenters or does other unusual control flow?** Storage update — revert rolls back all pops in the transaction.
  - `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`this._calculateCreditMultiplier(credit.earningStartTimestamp,`  
`credit.isRWA) -> this._applyDailyRate(multiplier, startDay, currentDay)`  
`-> FixedPointMathLib.mulWad(multiplier, TreasuryDelegate.WAD + dailyRate)`
    - **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
    - **If the return value is controllable, how is it used and how can it go wrong?** The product updates `yieldMultiplier[user]` or intermediate merge math.

Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.

- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`this._calculateCreditMultiplier(credit.earningStartTimestamp,`  
`credit.isRWA) -> this._applyPartialDayRate(multiplier, startDay,`  
`firstDaySeconds) -> FixedPointMathLib.mulWad(currentMultiplier,`  
`TreasuryDelegate.WAD + partialRate)`

- **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.

- **If the return value is controllable, how is it used and how can it go wrong?**

The product updates yieldMultiplier[user] or intermediate merge math. Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.

- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`FixedPointMathLib.mulWad(credit.credits, creditMultiplier)`

- **What is controllable?** credit.credits and creditMultiplier are deque records written by prior swap paths; the caller does not pick them at accrual time.

- **If the return value is controllable, how is it used and how can it go wrong?**

The product contributes to totalMergedValue for the weighted-average blend into main credits. Incorrect multipliers would require bad posted rates or corrupt deque state.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math inside merge — failure reverts the entire accrual with no partial deque progress.

- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`FixedPointMathLib.mulWad(userCredits, this.yieldMultiplier[user])`

- **What is controllable?** userCredits and yieldMultiplier[user] reflect prior deposits and accrual — not directly set by the current external caller.

- **If the return value is controllable, how is it used and how can it go wrong?**

Used to compute oldValue before blending merged withheld credits. A stale multiplier understates or overstates the blend until accrual catches up.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math — revert aborts merge.

- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) -> FixedPointMathLib.divWad(oldValue`  
`+ totalMergedValue, newCredits)`
  - **What is controllable?** Numerator sums prior main-credit value and merged withheld value; denominator is new total credits after merge.
  - **If the return value is controllable, how is it used and how can it go wrong?** Becomes the new `yieldMultiplier[user]`. Division truncates; extreme queue sizes increase gas but not caller control over the formula.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math. On OOG, during the surrounding merge loop, the entire transaction reverts with no deque progress (L5 grief).
- `SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user,`  
`this.balanceSheetCustodyAddress, params.assetInAmount)`
  - **What is controllable?** MultiliquidSwap supplies params after its own pricing; the user must have approved the RWA/stablecoin and hold sufficient balance.
  - **If the return value is controllable, how is it used and how can it go wrong?** Moves backing assets into custody. Fee-on-transfer or rebasing tokens could desync custody vs credited amounts if whitelisted without review.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap before mint/burn. A malicious token could callback; `nonReentrant` on the entry point limits reentrancy into the delegate.
- `this._addWithheldCredit(params.user, params.assetInAddress,`  
`params.assetOutAmount, block.timestamp +`  
`TreasuryDelegate.WITHHELD_PERIOD, block.timestamp, True) ->`  
`WithheldCreditsDeque.pushBack(this.withheldCredits[user],`  
`WithheldCredits(tokenAddress: _tokenAddress, credits: _amount,`  
`releaseTimestamp: _releaseTimestamp, earningStartTimestamp:`  
`_earningStartTimestamp, isRWA: _isRWA))`
  - **What is controllable?** Record fields come from internal callers (`_addWithheldCredit` or partial reduction), ultimately sourced from MultiliquidSwap DelegateParams.
  - **If the return value is controllable, how is it used and how can it go wrong?** Appends one deque entry per deposit/fee without coalescing, growing `totalWithheldCredits` and future merge gas (L5).
  - **What happens if it reverts, reenters or does other unusual control flow?** Storage write — revert drops the push. No external token call in the deque op itself.
- `this.treasury.mint(params.vault, params.protocolFeeAmt)`
  - **What is controllable?** Mint amount and recipient come from MultiliquidSwap DelegateParams (`assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver`).

- **If the return value is controllable, how is it used and how can it go wrong?**  
Increases Treasury supply without reducing user credits when minting to receiver — fee mints pair with `_addWithheldCredit` on vault/custody. Misconfigured swap params could overmint relative to backing.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.
- `this.treasury.mint(params.receiver, params.assetOutAmount)`
  - **What is controllable?** Mint amount and recipient come from MultiliquidSwap DelegateParams (assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver).
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Increases Treasury supply without reducing user credits when minting to receiver — fee mints pair with `_addWithheldCredit` on vault/custody. Misconfigured swap params could overmint relative to backing.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.

### Function: exchangeRWAs(DelegateParams params)

The exchangeRWAs function swaps one RWA for another without minting user credits; it may mint Treasury protocol/LP fees to vault/custody with withheld bookkeeping and moves RWA through custody.

### Inputs

- params
  - **Control:** MultiliquidSwap only.
  - **Constraints:** Both assets whitelisted RWA, swap role, blacklist, custody, and pause.
  - **Impact:** RWA amounts, fee mints, and receiver payout — no user credit/yield state change.

### Branches and code coverage

#### Intended branches

- Pulls assetIn RWA from user to balance-sheet custody.
- ☑ Test coverage
- Optionally adds withheld credits and mints Treasury fees to vault and stablecoinCustodyAddress.

- ☒ Test coverage
- Transfers assetOut RWA from custody to receiver.

☒ Test coverage

#### Negative behavior

- Reverts without swap role, on pause, blacklist, or missing custody.
- ☒ Negative test
- Reverts when either RWA is not whitelisted or ERC-20 transfers fail.
- ☒ Negative test
- Each fee mint pushes another deque record without coalescing (L5 queue growth on shared vault/custody).
- ☐ Negative test

#### Function call analysis

- SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user, this.balanceSheetCustodyAddress, params.assetInAmount)
  - **What is controllable?** MultiliquidSwap supplies params after its own pricing; the user must have approved the RWA/stablecoin and hold sufficient balance.
  - **If the return value is controllable, how is it used and how can it go wrong?** Moves backing assets into custody. Fee-on-transfer or rebasing tokens could desync custody vs credited amounts if whitelisted without review.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap before mint/burn. A malicious token could callback; nonReentrant on the entry point limits reentrancy into the delegate.
- this.\_addWithheldCredit(params.vault, params.assetInAddress, params.protocolFeeAmt, block.timestamp + TreasuryDelegate.WITHHELD\_PERIOD, block.timestamp, True) -> WithheldCreditsDeque.pushBack(this.withheldCredits[user], WithheldCredits(tokenAddress: \_tokenAddress, credits: \_amount, releaseTimestamp: \_releaseTimestamp, earningStartTimestamp: \_earningStartTimestamp, isRWA: \_isRWA))
  - **What is controllable?** Record fields come from internal callers (\_addWithheldCredit or partial reduction), ultimately sourced from MultiliquidSwap DelegateParams.
  - **If the return value is controllable, how is it used and how can it go wrong?** Appends one deque entry per deposit/fee without coalescing, growing totalWithheldCredits and future merge gas (L5).
  - **What happens if it reverts, reenters or does other unusual control flow?**

Storage write — revert drops the push. No external token call in the deque op itself.

- `this.treasury.mint(params.vault, params.protocolFeeAmt)`
  - **What is controllable?** Mint amount and recipient come from MultiliquidSwap DelegateParams (assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver).
  - **If the return value is controllable, how is it used and how can it go wrong?** Increases Treasury supply without reducing user credits when minting to receiver — fee mints pair with `_addWithheldCredit` on vault/custody. Misconfigured swap params could overmint relative to backing.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.
- `this.treasury.mint(this.stablecoinCustodyAddress, params.lpFeeAmt)`
  - **What is controllable?** Mint amount and recipient come from MultiliquidSwap DelegateParams (assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver).
  - **If the return value is controllable, how is it used and how can it go wrong?** Increases Treasury supply without reducing user credits when minting to receiver; fee mints pair with `_addWithheldCredit` on vault/custody. Misconfigured swap params could overmint relative to backing.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.
- `SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.balanceSheetCustodyAddress, params.receiver, params.assetOutAmount)`
  - **What is controllable?** MultiliquidSwap sets assetOutAmount and receiver; custody must hold the tokens from prior deposits.
  - **If the return value is controllable, how is it used and how can it go wrong?** Delivers RWA or stablecoin to the user after credits are reduced and Treasury burned. Insufficient custody balance causes revert.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back credit reduction and burns. Token hooks could reenter but the outer swap path is nonReentrant.

### Function: exchangeStablecoins(DelegateParams params)

The exchangeStablecoins function routes stablecoin <-> Treasury swaps: mint path when assetOut is issuer Treasury, redeem path when assetIn is Treasury, otherwise reverts.

## Inputs

- params
  - **Control:** MultiliquidSwap only.
  - **Constraints:** Swap role, blacklist, custody, and pause, and asset pair must include stablecoinAddress.
  - **Impact:** Selects mint (T+1 earning withheld credits) vs redeem (partial-day yield on exit) internal flows.

## Branches and code coverage

### Intended branches

- Delegates to \_exchangeStablecoinsMint when minting Treasury.
  - ☒ Test coverage
- Delegates to \_exchangeStablecoinsRedeem when burning Treasury for whitelisted stablecoin.
  - ☒ Test coverage

### Negative behavior

- Reverts InvalidStablecoinExchange when neither leg is the issuer stablecoin.
  - ☒ Negative test
- Reverts on missing swap authorization, pause, or custody.
  - ☒ Negative test
- Internal paths inherit accrual/merge OOG and InsufficientCredits behavior.
  - ☐ Negative test

## Function call analysis

- this.\_exchangeStablecoinsMint(params) -> this.\_accrueInterest(params.user) -> this.\_accrueInterestPaged(user, 0) -> this.\_getDailyCompoundingRate(day) -> this.\_convertApyToDailyRate(apy) -> FixedPointMathLib.powWad(TreasuryDelegate.WAD\_INT + apyInt, TreasuryDelegate.ONE\_OVER\_DAYS\_PER\_YEAR\_WAD)
  - **What is controllable?** The APY input comes from posted dailyRates (net of managementFee) or is derived during initialization; it is bounded by \_checkDailyRate and WAD.
  - **If the return value is controllable, how is it used and how can it go wrong?** The per-day compounding rate is stored or multiplied into yieldMultiplier. An admin-posted rate affects all users uniformly; there is no per-call caller

control.

- **What happens if it reverts, reenters or does other unusual control flow?** A powWad revert aborts the enclosing transaction. This is a pure library call with no external callbacks.
- `this._exchangeStablecoinsMint(params) ->`  
`this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> FixedPointMathLib.mulWad(userMultiplier, TreasuryDelegate.WAD +`  
`dailyRate)`
  - **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?** The product updates yieldMultiplier[user] or intermediate merge math. Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- `this._exchangeStablecoinsMint(params) ->`  
`this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.empty(this.withheldCredits[user])`
  - **What is controllable?** Whether the deque is empty depends on prior \_addWithheldCredit / \_reduceWithheldAndCredits history for user.
  - **If the return value is controllable, how is it used and how can it go wrong?** Gates the merge/reduction loop. A nonempty deque triggers further processing and gas cost scaling with record count.
  - **What happens if it reverts, reenters or does other unusual control flow?** View on storage — no external call. False empty skips merge safely.
- `this._exchangeStablecoinsMint(params) ->`  
`this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.front(this.withheldCredits[user])`
  - **What is controllable?** The front record is the oldest withheld credit; its fields were set at deposit time by MultiliquidSwap-driven paths.
  - **If the return value is controllable, how is it used and how can it go wrong?** Determines whether FIFO merge proceeds (release timestamp, earning start, rate availability). Out-of-order maturity stops the loop without corruption.
  - **What happens if it reverts, reenters or does other unusual control flow?** Memory read from deque storage — no external callback.
- `this._exchangeStablecoinsMint(params) ->`  
`this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`



```
-> this._mergeWithheldCredits(user) ->
WithheldCreditsDeque.popFront(this.withheldCredits[user])
```

- **What is controllable?** Only reached when front credit passes release, earning-start, and applied-through-day checks.
  - **If the return value is controllable, how is it used and how can it go wrong?** Removes one deque record per iteration. Many records make the unbounded while loop gas-heavy (L5 grief on shared vault/custody queues).
  - **What happens if it reverts, reenters or does other unusual control flow?** Storage update — revert rolls back all pops in the transaction.
- this.\_exchangeStablecoinsMint(params) ->
 

```
this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)
-> this._mergeWithheldCredits(user) ->
this._calculateCreditMultiplier(credit.earningStartTimestamp,
credit.isRWA) -> this._applyDailyRate(multiplier, startDay, currentDay)
-> FixedPointMathLib.mulWad(multiplier, TreasuryDelegate.WAD + dailyRate)
```

  - **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?** The product updates yieldMultiplier[user] or intermediate merge math. Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- this.\_exchangeStablecoinsMint(params) ->
 

```
this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)
-> this._mergeWithheldCredits(user) ->
this._calculateCreditMultiplier(credit.earningStartTimestamp,
credit.isRWA) -> this._applyPartialDayRate(multiplier, startDay,
firstDaySeconds) -> FixedPointMathLib.mulWad(currentMultiplier,
TreasuryDelegate.WAD + partialRate)
```

  - **What is controllable?** userMultiplier and dailyRate come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?** The product updates yieldMultiplier[user] or intermediate merge math. Rounding is downward via mulWad; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- this.\_exchangeStablecoinsMint(params) ->

```
this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)
-> this._mergeWithheldCredits(user) ->
FixedPointMathLib.mulWad(credit.credits, creditMultiplier)
```

- **What is controllable?** credit.credits and creditMultiplier are deque records written by prior swap paths; the caller does not pick them at accrual time.
- **If the return value is controllable, how is it used and how can it go wrong?** The product contributes to totalMergedValue for the weighted-average blend into main credits. Incorrect multipliers would require bad posted rates or corrupt deque state.
- **What happens if it reverts, reenters or does other unusual control flow?** Pure math inside merge — failure reverts the entire accrual with no partial deque progress.

```
• this._exchangeStablecoinsMint(params) ->
this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)
-> this._mergeWithheldCredits(user) ->
FixedPointMathLib.mulWad(userCredits, this.yieldMultiplier[user])
```

- **What is controllable?** userCredits and yieldMultiplier[user] reflect prior deposits and accrual — not directly set by the current external caller.
- **If the return value is controllable, how is it used and how can it go wrong?** Used to compute oldValue before blending merged withheld credits. A stale multiplier understates or overstates the blend until accrual catches up.
- **What happens if it reverts, reenters or does other unusual control flow?** Pure math — revert aborts merge.

```
• this._exchangeStablecoinsMint(params) ->
this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)
-> this._mergeWithheldCredits(user) -> FixedPointMathLib.divWad(oldValue
+ totalMergedValue, newCredits)
```

- **What is controllable?** Numerator sums prior main-credit value and merged withheld value; the denominator is new total credits after merge.
- **If the return value is controllable, how is it used and how can it go wrong?** Becomes the new yieldMultiplier[user]. Division truncates; extreme queue sizes increase gas but not caller control over the formula.
- **What happens if it reverts, reenters or does other unusual control flow?** Pure math. On OOG, during the surrounding merge loop, the entire transaction reverts with no deque progress (L5 grief).

```
• this._exchangeStablecoinsMint(params) ->
SafeERC20.safeTransferFrom(IERC20(params.assetInAddress), params.user,
this.stablecoinCustodyAddress, params.assetInAmount)
```

- **What is controllable?** MultiliquidSwap supplies params after its own pricing; the user must have approved the RWA/stablecoin and hold sufficient balance.
- **If the return value is controllable, how is it used and how can it go wrong?**

Moves backing assets into custody. Fee-on-transfer or rebasing tokens could desync custody vs credited amounts if whitelisted without review.

- **What happens if it reverts, reenters or does other unusual control flow?**  
Revert aborts the swap before mint/burn. A malicious token could callback; nonReentrant on the entry point limits reentrancy into the delegate.

```
• this._exchangeStablecoinsMint(params) ->
 this._addWithheldCredit(params.vault, params.assetInAddress,
 params.protocolFeeAmt, block.timestamp +
 TreasuryDelegate.WITHHELD_PERIOD, block.timestamp +
 TreasuryDelegate.WITHHELD_PERIOD, False) ->
 WithheldCreditsDeque.pushBack(this.withheldCredits[user],
 WithheldCredits(tokenAddress: _tokenAddress, credits: _amount,
 releaseTimestamp: _releaseTimestamp, earningStartTimestamp:
 _earningStartTimestamp, isRWA: _isRWA))
```

- **What is controllable?** Record fields come from internal callers (\_addWithheldCredit or partial reduction), ultimately sourced from MultiliquidSwap DelegateParams.
- **If the return value is controllable, how is it used and how can it go wrong?**  
Appends one deque entry per deposit/fee without coalescing, growing totalWithheldCredits and future merge gas (L5).
- **What happens if it reverts, reenters or does other unusual control flow?**  
Storage write — revert drops the push. No external token call in the deque op itself.

```
• this._exchangeStablecoinsMint(params) -> this.treasury.mint(params.vault,
 params.protocolFeeAmt)
```

- **What is controllable?** Mint amount and recipient come from MultiliquidSwap DelegateParams (assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver).
- **If the return value is controllable, how is it used and how can it go wrong?**  
Increases Treasury supply without reducing user credits when minting to receiver; fee mints pair with \_addWithheldCredit on vault/custody. Misconfigured swap params could overmint relative to backing.
- **What happens if it reverts, reenters or does other unusual control flow?**  
Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.

```
• this._exchangeStablecoinsMint(params) ->
 this.treasury.mint(this.stablecoinCustodyAddress, params.lpFeeAmt)
```

- **What is controllable?** Mint amount and recipient come from MultiliquidSwap DelegateParams (assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver).
- **If the return value is controllable, how is it used and how can it go wrong?**  
Increases Treasury supply without reducing user credits when minting to receiver; fee mints pair with \_addWithheldCredit on vault/custody.

Misconfigured swap params could overmint relative to backing.

- **What happens if it reverts, reenters or does other unusual control flow?**

Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.

- `this._exchangeStablecoinsMint(params) ->`  
`this.treasury.mint(params.receiver, params.assetOutAmount)`

- **What is controllable?** Mint amount and recipient come from

MultiliquidSwap DelegateParams (assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver).

- **If the return value is controllable, how is it used and how can it go wrong?**

Increases Treasury supply without reducing user credits when minting to receiver; fee mints pair with `_addWithheldCredit` on vault/custody.

Misconfigured swap params could overmint relative to backing.

- **What happens if it reverts, reenters or does other unusual control flow?**

Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.

- `this._exchangeStablecoinsRedeem(params) ->`  
`this._reduceWithheldAndCredits(params.user, params.assetInAmount) ->`  
`WithheldCreditsDeque.empty(this.withheldCredits[user])`

- **What is controllable?** Whether the deque is empty depends on prior `_addWithheldCredit` / `_reduceWithheldAndCredits` history for user.

- **If the return value is controllable, how is it used and how can it go wrong?**

Gates the merge/reduction loop. A nonempty deque triggers further processing and gas cost scaling with record count.

- **What happens if it reverts, reenters or does other unusual control flow?**

View on storage — no external call. False empty skips merge safely.

- `this._exchangeStablecoinsRedeem(params) ->`  
`this._reduceWithheldAndCredits(params.user, params.assetInAmount) ->`  
`WithheldCreditsDeque.back(this.withheldCredits[user])`

- **What is controllable?** The back record is the most recent withheld credit for LIFO reduction during redemption.

- **If the return value is controllable, how is it used and how can it go wrong?**

Determines how much principal is consumed without yield. Large queues increase gas for `_reduceWithheldAndCredits` but not mint/burn authority.

- **What happens if it reverts, reenters or does other unusual control flow?**

Storage read only.

- `this._exchangeStablecoinsRedeem(params) ->`  
`this._reduceWithheldAndCredits(params.user, params.assetInAmount) ->`  
`WithheldCreditsDeque.popBack(this.withheldCredits[user])`

- **What is controllable?** Invoked while reducing redemption amount against withheld credits from the back.

- **If the return value is controllable, how is it used and how can it go wrong?**

Removes or shrinks the newest withheld record. Partial pops rewrite the tail entry via pushBack.

- **What happens if it reverts, reenters or does other unusual control flow?**

Storage mutation — revert restores prior deque state.

- `this._exchangeStablecoinsRedeem(params) ->`  
`this._reduceWithheldAndCredits(params.user, params.assetInAmount) ->`  
`WithheldCreditsDeque.popBack(this.withheldCredits[user])`

- **What is controllable?** Invoked while reducing redemption amount against withheld credits from the back.

- **If the return value is controllable, how is it used and how can it go wrong?**

Removes or shrinks the newest withheld record. Partial pops rewrite the tail entry via pushBack.

- **What happens if it reverts, reenters or does other unusual control flow?**

Storage mutation — revert restores prior deque state.

- `this._exchangeStablecoinsRedeem(params) ->`  
`this._reduceWithheldAndCredits(params.user, params.assetInAmount) ->`  
`WithheldCreditsDeque.pushBack(this.withheldCredits[user],`  
`WithheldCredits(tokenAddress: credit.tokenAddress, credits:`  
`credit.credits - reduceAmount, releaseTimestamp:`  
`credit.releaseTimestamp, earningStartTimestamp:`  
`credit.earningStartTimestamp, isRWA: credit.isRWA))`

- **What is controllable?** Record fields come from internal callers (`_addWithheldCredit` or partial reduction), ultimately sourced from `MultiliquidSwap DelegateParams`.

- **If the return value is controllable, how is it used and how can it go wrong?**

Appends one deque entry per deposit/fee without coalescing, growing `totalWithheldCredits` and future merge gas (L5).

- **What happens if it reverts, reenters or does other unusual control flow?**

Storage write — revert drops the push. No external token call in the deque op itself.

- `this._exchangeStablecoinsRedeem(params) ->`  
`this._applyPartialDayRateWithFallback(userMultiplier,`  
`this.lastRatePostingDay, accruedSeconds) ->`  
`FixedPointMathLib.mulWad(currentMultiplier, TreasuryDelegate.WAD +`  
`partialRate)`

- **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.

- **If the return value is controllable, how is it used and how can it go wrong?**

The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.

- **What happens if it reverts, reenters or does other unusual control flow?**

Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.

- `this._exchangeStablecoinsRedeem(params) -> FixedPointMathLib.mulWad(creditsReduced, userMultiplier)`
  - **What is controllable?** `creditsReduced` follows `_reduceWithheldAndCredits`; `userMultiplier` includes optional partial-day uplift on stablecoin redeem.
  - **If the return value is controllable, how is it used and how can it go wrong?** Drives `YieldClaimed` event amounts and implied yield — does not change burn amount (1:1 Treasury burn vs credits reduced).
  - **What happens if it reverts, reenters or does other unusual control flow?** Pure math after state updates — revert would roll back the whole redeem.
- `this._exchangeStablecoinsRedeem(params) -> this.treasury.burnFrom(params.user, params.assetInAmount)`
  - **What is controllable?** `params.user` and `params.assetInAmount` are set by `MultiliquidSwap` after credit checks; the user must have approved Treasury burn.
  - **If the return value is controllable, how is it used and how can it go wrong?** Burns Treasury after credits are reduced, capping redemption to tracked backing. Burning without credit reduction is prevented by ordering and `InsufficientCredits`.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert restores credits and withheld state. Token hooks could callback before the function completes.
- `this._exchangeStablecoinsRedeem(params) -> this.treasury.mint(params.vault, params.protocolFeeAmt)`
  - **What is controllable?** Mint amount and recipient come from `MultiliquidSwap DelegateParams (assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver)`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Increases Treasury supply without reducing user credits when minting to receiver; fee mints pair with `_addWithheldCredit` on vault/custody. Misconfigured swap params could overmint relative to backing.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.
- `this._exchangeStablecoinsRedeem(params) -> this.treasury.mint(this.stablecoinCustodyAddress, params.lpFeeAmt)`
  - **What is controllable?** Mint amount and recipient come from `MultiliquidSwap DelegateParams (assetOutAmount, protocolFeeAmt, lpFeeAmt, vault, receiver)`.
  - **If the return value is controllable, how is it used and how can it go wrong?**

Increases Treasury supply without reducing user credits when minting to receiver; fee mints pair with `_addWithheldCredit` on vault/custody. Misconfigured swap params could overmint relative to backing.

- **What happens if it reverts, reenters or does other unusual control flow?**

Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.

- `this._exchangeStablecoinsRedeem(params) -> SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.stablecoinCustodyAddress, params.receiver, params.assetOutAmount)`
  - **What is controllable?** MultiliquidSwap sets `assetOutAmount` and `receiver`; custody must hold the tokens from prior deposits.
  - **If the return value is controllable, how is it used and how can it go wrong?** Delivers RWA or stablecoin to the user after credits are reduced and Treasury burned. Insufficient custody balance causes revert.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert rolls back credit reduction and burns. Token hooks could reenter, but the outer swap path is `nonReentrant`.

### Function: `getAmountForTargetValue(address user, uint256 targetValue, bool stablecoinWithdrawal)`

The `getAmountForTargetValue` function inverts yield pricing and returns the Treasury amount needed to achieve `targetValue` effective dollars, consuming withheld at 1:1 then main credits with multiplier (and stablecoin partial-day uplift).

### Inputs

- `user`
  - **Control:** Arbitrary caller (view).
  - **Constraints:** None.
  - **Impact:** Credit and multiplier source account.
- `targetValue`
  - **Control:** Arbitrary caller.
  - **Constraints:** Must be satisfiable with user credits or reverts in internal helper.
  - **Impact:** Desired effective redemption value.
- `stablecoinWithdrawal`
  - **Control:** Arbitrary caller.
  - **Constraints:** Boolean.
  - **Impact:** Enables partial-day fallback on main-credit leg.

## Branches and code coverage

### Intended branches

- Returns 0 for zero target.
  - ☒ Test coverage
- Returns target when covered fully by withheld credits.
  - ☒ Test coverage
- Uses `divWadUp` on main credits after partial-day multiplier.
  - ☒ Test coverage

### Negative behavior

- Reverts `InsufficientCredits` when main credits cannot satisfy remainder.
  - ☒ Negative test

## Function: `getCurrentDay()`

The `getCurrentDay` function returns `block.timestamp / 1 days` for rate posting and accrual alignment.

## Branches and code coverage

### Intended branches

- Returns the current day index derived from block timestamp.
  - ☒ Test coverage

### Negative behavior

- No revert paths.
  - ☐ Negative test

## Function: `getDailyRate(uint256 dayNumber)`

The `getDailyRate` function returns the stored net APY and whether a rate was posted for a calendar day.

## Inputs

- `dayNumber`



- **Control:** Arbitrary caller (view).
- **Constraints:** None.
- **Impact:** Day key into `dailyRates` / `ratePosted`.

## Branches and code coverage

### Intended branches

- Returns `(dailyRates[day], ratePosted[day])`, including zero rate when unposted.

☒ Test coverage

### Negative behavior

- No revert paths for arbitrary day numbers.

☐ Negative test

## Function: `getWithheldCredits(address user)`

The `getWithheldCredits` function returns a memory copy of all deque records for a user (token, credits, release/earning timestamps, `isRWA`).

### Inputs

- `user`
  - **Control:** Arbitrary caller (view).
  - **Constraints:** None.
  - **Impact:** Which withheld deque is materialized — gas scales with queue length.

## Branches and code coverage

### Intended branches

- Returns `withheldCredits[user].toArray()` snapshot.

☒ Test coverage

### Negative behavior

- Very large queues can make the view call OOG off chain (L5 visibility).

☐ Negative test

## Function: getYieldAmount(address user, uint256 redeemValue, bool stablecoinWithdrawal)

The getYieldAmount function view-previews effective dollar value for burning redeemValue Treasury: withheld credits at 1:1, then main credits at yieldMultiplier (plus optional partial-day fallback on stablecoin withdrawal).

### Inputs

- user
  - **Control:** Arbitrary caller (view).
  - **Constraints:** None.
  - **Impact:** Whose credits, totalWithheldCredits, and yieldMultiplier are read.
- redeemValue
  - **Control:** Arbitrary caller.
  - **Constraints:** None on chain – off-chain should bound to user balances.
  - **Impact:** Treasury burn amount being priced.
- stablecoinWithdrawal
  - **Control:** Arbitrary caller.
  - **Constraints:** Boolean flag.
  - **Impact:** When true, applies \_applyPartialDayRateWithFallback to main credits.

### Branches and code coverage

#### Intended branches

- Returns 0 for zero redeemValue.
  - ☒ Test coverage
- Returns 1:1 when redeem fits entirely in withheld credits.
  - ☒ Test coverage
- Blends withheld (1:1) and main (mulWad multiplier) portions.
  - ☒ Test coverage

#### Negative behavior

- Does not accrue interest – stale multiplier if accrual lags.
  - ☐ Negative test
- Does not revert on insufficient credits (view only) – swap path enforces backing

separately.

☑ Negative test

**Function: initialize(address multiliquidAdmin, address initialIssuerAdmin, address \_multiliquidSwap, byte[32] \_stablecoinID, address \_stablecoinAddress, uint256 initialRate, uint256 initialManagementFee, address ratePoster)**

The initialize function bootstraps the yield-bearing Treasury delegate: base Multiliquid wiring, Treasury token binding, optional RATE\_POSTER\_ROLE, and the first net daily rate (gross APY minus managementFee) for the current day.

## Inputs

- multiliquidAdmin
  - **Control:** Proxy deployer (one-shot initializer).
  - **Constraints:** Must pass \_\_stablecoinDelegateBase\_init nonzero/distinct checks.
  - **Impact:** Receives DEFAULT\_ADMIN\_ROLE for upgrades and global caps (maxDailyRate, setUserYieldMultiplier).
- initialIssuerAdmin
  - **Control:** Proxy deployer.
  - **Constraints:** Nonzero and distinct from other admin addresses.
  - **Impact:** Becomes issuer admin for custody, whitelist, and setManagementFee.
- \_multiliquidSwap
  - **Control:** Proxy deployer.
  - **Constraints:** Nonzero — granted MULTILIQUEID\_SWAP\_CONTRACT.
  - **Impact:** Sole caller allowed on swap entry points (deployStablecoin, receiveStablecoin, etc.).
- \_stablecoinID
  - **Control:** Proxy deployer.
  - **Constraints:** Nonzero bytes32.
  - **Impact:** Keys this delegate in MultiliquidSwap routing and fees.
- \_stablecoinAddress
  - **Control:** Proxy deployer.
  - **Constraints:** Nonzero Treasury token address.
  - **Impact:** Stored as treasury for all mint / burnFrom calls.

- `initialRate`
  - **Control:** Proxy deployer.
  - **Constraints:**  $\leq$  WAD (100% APY); must be  $\geq$  `initialManagementFee`.
  - **Impact:** Seeds day-zero gross APY before fee subtraction.
- `initialManagementFee`
  - **Control:** Proxy deployer.
  - **Constraints:**  $\leq$  WAD and not greater than `initialRate`.
  - **Impact:** Persisted `managementFee` subtracted from every posted/corrected gross rate.
- `ratePoster`
  - **Control:** Proxy deployer.
  - **Constraints:** `address(0)` skips grant, otherwise any address.
  - **Impact:** Optional `RATE_POSTER_ROLE` for sequential `postDailyRate`.

## Branches and code coverage

### Intended branches

- Initializes base delegate modules and sets `treasury` to `_stablecoinAddress`.
  - ☒ Test coverage
- Grants `RATE_POSTER_ROLE` when `ratePoster != address(0)`.
  - ☒ Test coverage
- Stores net initial rate, compounding rate, and `firstRateDay / lastRatePostingDay` for the current day.
  - ☒ Test coverage

### Negative behavior

- Reverts `RateTooHigh` when `initialRate > WAD`.
  - ☒ Negative test
- Reverts `ManagementFeeTooHigh` when fee exceeds WAD or gross rate.
  - ☒ Negative test
- Reverts when base init params are invalid (zero address, duplicate admins).
  - ☒ Negative test

## Function call analysis

- `this.**stablecoinDelegateBase_init(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _stablecoinID, _stablecoinAddress) -> this.**ReentrancyGuard_init() -> this.__ReentrancyGuard_init_unchained() -> this._getReentrancyGuardStorage()`
  - **What is controllable?** Addresses are supplied once by the proxy deployer in initialize, not user-controlled afterward.
  - **If the return value is controllable, how is it used and how can it go wrong?** Grants roles and initializes base modules. Compromised deployer keys control initial admin layout.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents initialization. OpenZeppelin internals have no external calls in these traces.
- `this.__stablecoinDelegateBase_init(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _stablecoinID, _stablecoinAddress) -> this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> this._getAccessControlEnumerableStorage()`
  - **What is controllable?** Addresses are supplied once by the proxy deployer in initialize, not user-controlled afterward.
  - **If the return value is controllable, how is it used and how can it go wrong?** Grants roles and initializes base modules. Compromised deployer keys control initial admin layout.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents initialization. OpenZeppelin internals have no external calls in these traces.
- `this.__stablecoinDelegateBase_init(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _stablecoinID, _stablecoinAddress) -> this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> super._grantRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** Addresses are supplied once by the proxy deployer in initialize, not user-controlled afterward.
  - **If the return value is controllable, how is it used and how can it go wrong?** Grants roles and initializes base modules. Compromised deployer keys control initial admin layout.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents initialization. OpenZeppelin internals have no external calls in these traces.
- `this.__stablecoinDelegateBase_init(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _stablecoinID, _stablecoinAddress) -> this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> super._grantRole(role, account) -> this.hasRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** Addresses are supplied once by the proxy deployer in initialize, not user-controlled afterward.
  - **If the return value is controllable, how is it used and how can it go wrong?** Grants roles and initializes base modules. Compromised deployer keys control initial admin layout.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents initialization. OpenZeppelin internals have no external calls in these traces.

- **What is controllable?** Addresses are supplied once by the proxy deployer in initialize, not user-controlled afterward.
  - **If the return value is controllable, how is it used and how can it go wrong?** Grants roles and initializes base modules. Compromised deployer keys control initial admin layout.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents initialization. OpenZeppelin internals have no external calls in these traces.
- ```

• this.__stablecoinDelegateBase_init(multiliquidAdmin, initialIssuerAdmin,
  _multiliquidSwap, _stablecoinID, _stablecoinAddress) ->
  this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE,
    multiliquidAdmin) -> EnumerableSet.add($_roleMembers[role], account)

```
- **What is controllable?** Addresses are supplied once by the proxy deployer in initialize, not user-controlled afterward.
 - **If the return value is controllable, how is it used and how can it go wrong?** Grants roles and initializes base modules. Compromised deployer keys control initial admin layout.
 - **What happens if it reverts, reenters or does other unusual control flow?** Revert prevents initialization. OpenZeppelin internals have no external calls in these traces.
- ```

• this._convertApyToDailyRate(netInitialRate) ->
 FixedPointMathLib.powWad(TreasuryDelegate.WAD_INT + apyInt,
 TreasuryDelegate.ONE_OVER_DAYS_PER_YEAR_WAD)

```
- **What is controllable?** The APY input comes from posted dailyRates (net of managementFee) or is derived during initialization; it is bounded by \_checkDailyRate and WAD.
  - **If the return value is controllable, how is it used and how can it go wrong?** The per-day compounding rate is stored or multiplied into yieldMultiplier. An admin-posted rate affects all users uniformly; there is no per-call caller control.
  - **What happens if it reverts, reenters or does other unusual control flow?** A powWad revert aborts the enclosing transaction. This is a pure library call with no external callbacks.

### Function: postDailyRate(uint256 dayNumber, uint256 grossRate)

The postDailyRate function posts the next sequential day's gross APY: subtracts managementFee, converts to a daily compounding rate, and advances lastRatePostingDay by exactly one day.

### Inputs

- dayNumber

- **Control:** RATE\_POSTER\_ROLE holder.
  - **Constraints:** Must equal lastRatePostingDay + 1 and must be < currentDay (finished day only).
  - **Impact:** Selects which calendar day receives the new net rate.
- grossRate
  - **Control:** Rate poster.
  - **Constraints:** Passes \_checkDailyRate (<= WAD, optional maxDailyRate); must be >= managementFee.
  - **Impact:** Sets dailyRates, dailyCompoundingRates, and ratePosted for the day.

## Branches and code coverage

### Intended branches

- Accepts the next sequential day and stores net rate after management fee subtraction.
  - ☒ Test coverage
- Updates lastRatePostingDay to dayNumber.
  - ☒ Test coverage

### Negative behavior

- Reverts MustPostSequentially when dayNumber != lastRatePostingDay + 1.
  - ☒ Negative test
- Reverts CannotPostRateForUnfinishedDay when posting today or in the future.
  - ☒ Negative test
- Reverts RateTooHigh or ManagementFeeTooHigh on bad rates.
  - ☒ Negative test
- Reverts without RATE\_POSTER\_ROLE.
  - ☒ Negative test

## Function call analysis

- this.\_convertApyToDailyRate(netRate) -> FixedPointMathLib.powWad(TreasuryDelegate.WAD\_INT + apyInt, TreasuryDelegate.ONE\_OVER\_DAYS\_PER\_YEAR\_WAD)
  - **What is controllable?** The APY input comes from posted dailyRates (net of managementFee) or is derived during initialization; it is bounded by \_checkDailyRate and WAD.

- **If the return value is controllable, how is it used and how can it go wrong?**  
The per-day compounding rate is stored or multiplied into `yieldMultiplier`. An admin-posted rate affects all users uniformly; there is no per-call caller control.
- **What happens if it reverts, reenters or does other unusual control flow?** A `powWad` revert aborts the enclosing transaction. This is a pure library call with no external callbacks.

### Function: `receiveStablecoin(DelegateParams params)`

The `receiveStablecoin` function redeems Treasury for RWA: accrues yield, enforces credit backing, reduces withheld (LIFO) then main credits, burns Treasury, mints fee Treasury, and transfers RWA from custody.

### Inputs

- `params`
  - **Control:** MultiliquidSwap supplies `params` after pricing; the user must hold Treasury and credits.
  - **Constraints:** Swap role, whitelisted RWA out, blacklist, custody, pause, and reentrancy guard.
  - **Impact:** Burn amount, credit reduction, fee mints, and RWA delivered to receiver.

### Branches and code coverage

#### Intended branches

- Reverts `InsufficientCredits` when `assetInAmount` exceeds `credits + totalWithheldCredits`.  
☒ Test coverage
- Reduces withheld credits from the deque back (LIFO) then main credits; withheld redeemed at 1:1 without yield.  
☒ Test coverage
- Emits `YieldClaimed` when main credits with `yieldMultiplier > WAD` are consumed.  
☒ Test coverage
- Burns Treasury from `params.user` and transfers RWA out of balance-sheet custody.  
☒ Test coverage
- Resets `yieldMultiplier` and `lastAccrualDay` when user fully exits.  
☒ Test coverage



- Mints protocol/LP fee Treasury with withheld credits on vault/custody when applicable.

☒ Test coverage

### Negative behavior

- Reverts on authorization, pause, blacklist, or custody failures.

☒ Negative test

- Reverts when burn or custody transfer fails.

☒ Negative test

- `_accrueInterest / _mergeWithheldCredits` can OOG on large withheld dequeues (L5).

☐ Negative test

### Function call analysis

- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._getDailyCompoundingRate(day) -> this._convertApyToDailyRate(apy)`  
`-> FixedPointMathLib.powWad(TreasuryDelegate.WAD_INT + apyInt,`  
`TreasuryDelegate.ONE_OVER_DAYS_PER_YEAR_WAD)`

- **What is controllable?** The APY input comes from posted dailyRates (net of managementFee) or is derived during initialization; it is bounded by `_checkDailyRate` and WAD.
- **If the return value is controllable, how is it used and how can it go wrong?** The per-day compounding rate is stored or multiplied into `yieldMultiplier`. An admin-posted rate affects all users uniformly; there is no per-call caller control.
- **What happens if it reverts, reenters or does other unusual control flow?** A `powWad` revert aborts the enclosing transaction. This is a pure library call with no external callbacks.

- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> FixedPointMathLib.mulWad(userMultiplier, TreasuryDelegate.WAD +`  
`dailyRate)`

- **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
- **If the return value is controllable, how is it used and how can it go wrong?** The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.
- **What happens if it reverts, reenters or does other unusual control flow?** Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.

- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.empty(this.withheldCredits[user])`
  - **What is controllable?** Whether the deque is empty depends on prior `_addWithheldCredit` / `_reduceWithheldAndCredits` history for user.
  - **If the return value is controllable, how is it used and how can it go wrong?** Gates the merge/reduction loop. A nonempty deque triggers further processing and gas cost scaling with record count.
  - **What happens if it reverts, reenters or does other unusual control flow?** View on storage — no external call. False empty skips merge safely.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.front(this.withheldCredits[user])`
  - **What is controllable?** The front record is the oldest withheld credit; its fields were set at deposit time by MultiliquidSwap-driven paths.
  - **If the return value is controllable, how is it used and how can it go wrong?** Determines whether FIFO merge proceeds (release timestamp, earning start, rate availability). Out-of-order maturity stops the loop without corruption.
  - **What happens if it reverts, reenters or does other unusual control flow?** Memory read from deque storage — no external callback.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`WithheldCreditsDeque.popFront(this.withheldCredits[user])`
  - **What is controllable?** Only reached when front credit passes release, earning-start, and applied-through-day checks.
  - **If the return value is controllable, how is it used and how can it go wrong?** Removes one deque record per iteration. Many records make the unbounded while loop gas-heavy (L5 grief on shared vault/custody queues).
  - **What happens if it reverts, reenters or does other unusual control flow?** Storage update — revert rolls back all pops in the transaction.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`this._calculateCreditMultiplier(credit.earningStartTimestamp,`  
`credit.isRWA) -> this._applyDailyRate(multiplier, startDay, currentDay)`  
`-> FixedPointMathLib.mulWad(multiplier, TreasuryDelegate.WAD + dailyRate)`
  - **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?** The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.

- **What happens if it reverts, reenters or does other unusual control flow?**  
Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`this._calculateCreditMultiplier(credit.earningStartTimestamp,`  
`credit.isRWA) -> this._applyPartialDayRate(multiplier, startDay,`  
`firstDaySeconds) -> FixedPointMathLib.mulWad(currentMultiplier,`  
`TreasuryDelegate.WAD + partialRate)`
  - **What is controllable?** `userMultiplier` and `dailyRate` come from on-chain user state and admin-posted rates, not from arbitrary caller input in external entry points.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
The product updates `yieldMultiplier[user]` or intermediate merge math. Rounding is downward via `mulWad`; a manipulated rate would require compromising rate posters or operators.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure math — a revert rolls back the whole accrual or swap. No reentrancy surface.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`FixedPointMathLib.mulWad(credit.credits, creditMultiplier)`
  - **What is controllable?** `credit.credits` and `creditMultiplier` are deque records written by prior swap paths; the caller does not pick them at accrual time.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
The product contributes to `totalMergedValue` for the weighted-average blend into main credits. Incorrect multipliers would require bad posted rates or corrupt deque state.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure math inside merge — failure reverts the entire accrual with no partial deque progress.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`  
`-> this._mergeWithheldCredits(user) ->`  
`FixedPointMathLib.mulWad(userCredits, this.yieldMultiplier[user])`
  - **What is controllable?** `userCredits` and `yieldMultiplier[user]` reflect prior deposits and accrual — not directly set by the current external caller.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Used to compute `oldValue` before blending merged withheld credits. A stale multiplier understates or overstates the blend until accrual catches up.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure math — revert aborts merge.
- `this._accrueInterest(params.user) -> this._accrueInterestPaged(user, 0)`

```
-> this._mergeWithheldCredits(user) -> FixedPointMathLib.divWad(oldValue
+ totalMergedValue, newCredits)
```

- **What is controllable?** Numerator sums prior main-credit value and merged withheld value; denominator is new total credits after merge.
- **If the return value is controllable, how is it used and how can it go wrong?** Becomes the new yieldMultiplier[user]. Division truncates; extreme queue sizes increase gas but not caller control over the formula.
- **What happens if it reverts, reenters or does other unusual control flow?** Pure math. On OOG, during the surrounding merge loop, the entire transaction reverts with no deque progress (L5 grief).
- this.\_reduceWithheldAndCredits(params.user, params.assetInAmount) -> WithheldCreditsDeque.empty(this.withheldCredits[user])
  - **What is controllable?** Whether the deque is empty depends on prior \_addWithheldCredit / \_reduceWithheldAndCredits history for user.
  - **If the return value is controllable, how is it used and how can it go wrong?** Gates the merge/reduction loop. A nonempty deque triggers further processing and gas cost scaling with record count.
  - **What happens if it reverts, reenters or does other unusual control flow?** View on storage — no external call. False empty skips merge safely.
- this.\_reduceWithheldAndCredits(params.user, params.assetInAmount) -> WithheldCreditsDeque.back(this.withheldCredits[user])
  - **What is controllable?** The back record is the most recent withheld credit for LIFO reduction during redemption.
  - **If the return value is controllable, how is it used and how can it go wrong?** Determines how much principal is consumed without yield. Large queues increase gas for \_reduceWithheldAndCredits but not mint/burn authority.
  - **What happens if it reverts, reenters or does other unusual control flow?** Storage read only.
- this.\_reduceWithheldAndCredits(params.user, params.assetInAmount) -> WithheldCreditsDeque.popBack(this.withheldCredits[user])
  - **What is controllable?** Invoked while reducing redemption amount against withheld credits from the back.
  - **If the return value is controllable, how is it used and how can it go wrong?** Removes or shrinks the newest withheld record. Partial pops rewrite the tail entry via pushBack.
  - **What happens if it reverts, reenters or does other unusual control flow?** Storage mutation — revert restores prior deque state.
- this.\_reduceWithheldAndCredits(params.user, params.assetInAmount) -> WithheldCreditsDeque.popBack(this.withheldCredits[user])
  - **What is controllable?** Invoked while reducing redemption amount against withheld credits from the back.

- **If the return value is controllable, how is it used and how can it go wrong?**  
Removes or shrinks the newest withheld record. Partial pops rewrite the tail entry via pushBack.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Storage mutation — revert restores prior deque state.
- `this._reduceWithheldAndCredits(params.user, params.assetInAmount) -> WithheldCreditsDeque.pushBack(this.withheldCredits[user], WithheldCredits(tokenAddress: credit.tokenAddress, credits: credit.credits - reduceAmount, releaseTimestamp: credit.releaseTimestamp, earningStartTimestamp: credit.earningStartTimestamp, isRWA: credit.isRWA))`
  - **What is controllable?** Record fields come from internal callers (`_addWithheldCredit` or partial reduction), ultimately sourced from `MultiliquidSwap DelegateParams`.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Appends one deque entry per deposit/fee without coalescing, growing `totalWithheldCredits` and future merge gas (L5).
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Storage write — revert drops the push. No external token call in the deque op itself.
- `FixedPointMathLib.mulWad(creditsReduced, userMultiplier)`
  - **What is controllable?** `creditsReduced` follows `_reduceWithheldAndCredits`; `userMultiplier` includes optional partial-day uplift on stablecoin redeem.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Drives `YieldClaimed` event amounts and implied yield; does not change burn amount (1:1 Treasury burn vs credits reduced).
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Pure math after state updates; revert would roll back the whole redeem.
- `treasuryToken.burnFrom(params.user, params.assetInAmount)`
  - **What is controllable?** `params.user` and `params.assetInAmount` are set by `MultiliquidSwap` after credit checks; the user must have approved Treasury burn.
  - **If the return value is controllable, how is it used and how can it go wrong?**  
Burns Treasury after credits are reduced, capping redemption to tracked backing. Burning without credit reduction is prevented by ordering and `InsufficientCredits`.
  - **What happens if it reverts, reenters or does other unusual control flow?**  
Revert restores credits and withheld state. Token hooks could callback before the function completes.
- `this._addWithheldCredit(params.vault, params.assetOutAddress, params.protocolFeeAmt, block.timestamp + TreasuryDelegate.WITHHELD_PERIOD, block.timestamp, True) ->`

```
WithheldCreditsDeque.pushBack(this.withheldCredits[user],
WithheldCredits(tokenAddress: _tokenAddress, credits: _amount,
releaseTimestamp: _releaseTimestamp, earningStartTimestamp:
_earningStartTimestamp, isRWA: _isRWA))
```

- **What is controllable?** Record fields come from internal callers (`_addWithheldCredit` or partial reduction), ultimately sourced from `MultiliquidSwap DelegateParams`.
- **If the return value is controllable, how is it used and how can it go wrong?** Appends one deque entry per deposit/fee without coalescing, growing `totalWithheldCredits` and future merge gas (L5).
- **What happens if it reverts, reenters or does other unusual control flow?** Storage write — revert drops the push. No external token call in the deque op itself.
- `treasuryToken.mint(params.vault, params.protocolFeeAmt)`
  - **What is controllable?** Mint amount and recipient come from `MultiliquidSwap DelegateParams` (`assetOutAmount`, `protocolFeeAmt`, `lpFeeAmt`, `vault`, `receiver`).
  - **If the return value is controllable, how is it used and how can it go wrong?** Increases Treasury supply without reducing user credits when minting to `receiver`; fee mints pair with `_addWithheldCredit` on `vault/custody`. Misconfigured swap params could overmint relative to backing.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.
- `treasuryToken.mint(this.stablecoinCustodyAddress, params.lpFeeAmt)`
  - **What is controllable?** Mint amount and recipient come from `MultiliquidSwap DelegateParams` (`assetOutAmount`, `protocolFeeAmt`, `lpFeeAmt`, `vault`, `receiver`).
  - **If the return value is controllable, how is it used and how can it go wrong?** Increases Treasury supply without reducing user credits when minting to `receiver`; fee mints pair with `_addWithheldCredit` on `vault/custody`. Misconfigured swap params could overmint relative to backing.
  - **What happens if it reverts, reenters or does other unusual control flow?** Revert aborts the swap. Treasury mint may invoke hooks on the token; state changes before the call are rolled back.
- `SafeERC20.safeTransferFrom(IERC20(params.assetOutAddress), this.balanceSheetCustodyAddress, params.receiver, params.assetOutAmount)`
  - **What is controllable?** `MultiliquidSwap` sets `assetOutAmount` and `receiver`; custody must hold the tokens from prior deposits.
  - **If the return value is controllable, how is it used and how can it go wrong?** Delivers RWA or stablecoin to the user after credits are reduced and Treasury burned. Insufficient custody balance causes revert.

- **What happens if it reverts, reenters or does other unusual control flow?**  
Revert rolls back credit reduction and burns. Token hooks could reenter but the outer swap path is nonReentrant.

### Function: `setManagementFee(uint256 fee)`

The `setManagementFee` function updates the WAD-scaled APY subtracted from every gross rate before storage.

#### Inputs

- `fee`
  - **Control:** Issuer admin only.
  - **Constraints:** `<= WAD`, and if `maxDailyRate != 0`, `fee <= maxDailyRate`.
  - **Impact:** Changes net yields on all future `postDailyRate / correctDailyRate` calls.

### Branches and code coverage

#### Intended branches

- Persists new `managementFee` and emits `ManagementFeeUpdated`.

☒ Test coverage

#### Negative behavior

- Reverts `ManagementFeeTooHigh` when `fee > WAD`.
- Reverts `RateTooHigh` when fee exceeds configured `maxDailyRate`.

☒ Negative test

☒ Negative test

- Reverts for non-issuer-admin callers.

☒ Negative test

### Function: `setMaxDailyRate(uint256 maxRate)`

The `setMaxDailyRate` function sets an optional gross APY ceiling for rate posting and correction (0 disables the cap).

## Inputs

- maxRate
  - **Control:** DEFAULT\_ADMIN\_ROLE.
  - **Constraints:**  $\leq$  WAD, and if nonzero, must be  $\geq$  managementFee.
  - **Impact:** Bounds postDailyRate and correctDailyRate via \_checkDailyRate.

## Branches and code coverage

### Intended branches

- Stores cap and emits MaxDailyRateUpdated — zero clears the soft cap.
- ☒ Test coverage

### Negative behavior

- Reverts when maxRate > WAD or maxRate < managementFee (when nonzero).
- ☒ Negative test
- Reverts without DEFAULT\_ADMIN\_ROLE.
- ☒ Negative test

## Function: setMaxFallbackRateSeconds(uint256 maxSeconds)

The setMaxFallbackRateSeconds function caps how many seconds of partial-day stale-rate fallback apply on stablecoin withdrawals (0 = unlimited).

## Inputs

- maxSeconds
  - **Control:** OPERATOR\_ROLE.
  - **Constraints:** Any uint256 — zero disables the cap.
  - **Impact:** Limits \_applyPartialDayRateWithFallback during stablecoin redeem pricing.

## Branches and code coverage

### Intended branches

- Updates maxFallbackRateSeconds and emits event.



☒ Test coverage

### Negative behavior

- Reverts without OPERATOR\_ROLE.

☒ Negative test

## Function: setUserYieldMultiplier(address user, uint256 newMultiplier)

The setUserYieldMultiplier function manually sets a user's blended WAD multiplier after historical rate corrections; it can increase redeemable value beyond current backing.

### Inputs

- user
  - **Control:** DEFAULT\_ADMIN\_ROLE.
  - **Constraints:** Nonzero address.
  - **Impact:** Whose yieldMultiplier is overwritten.
- newMultiplier
  - **Control:** Admin.
  - **Constraints:** Must be  $\geq$  WAD.
  - **Impact:** Directly changes effective redemption value for main credits — does not update lastAccrualDay.

## Branches and code coverage

### Intended branches

- Sets yieldMultiplier[user] and emits YieldMultiplierUpdated.

☒ Test coverage

### Negative behavior

- Reverts ZeroAddressNotAllowed or InvalidYieldMultiplier when below WAD.

☒ Negative test

- Reverts without DEFAULT\_ADMIN\_ROLE.

☒ Negative test

## 4.18. Module: TreasuryOFTAdapter.sol

## Function: constructor(address \_treasury, address \_lzEndpoint, address \_delegate)

The constructor function initializes the Treasury burn/mint adapter, LayerZero endpoint/delegate configuration, decimal conversion, and owner.

### Inputs

- `_treasury`
  - **Control:** Set by the deployer.
  - **Constraints:** The address must implement decimals and return at least six local decimals.
  - **Impact:** The burn and mint token and decimalConversionRate source.
- `_lzEndpoint`
  - **Control:** Set by the deployer.
  - **Constraints:** Must be a functioning LayerZero V2 endpoint contract.
  - **Impact:** The endpoint used for send, receive, and configuration.
- `_delegate`
  - **Control:** Set by the deployer.
  - **Constraints:** Must not be zero and acceptable to the endpoint.
  - **Impact:** The initial owner and endpoint delegate.

### Branches and code coverage

#### Intended branches

- Initializes immutable state, configuration, and ownership with valid inputs.
  - ☒ Test coverage

#### Negative behavior

- Reverts if `_treasury` is zero or has no deployed code when its decimal count is read.
  - ☒ Negative test
- Reverts if `_lzEndpoint` is zero or has no deployed code when its delegate is set.
  - ☒ Negative test
- Reverts if `_delegate` is zero during ownership initialization.
  - ☒ Negative test
- Reverts if the local decimal count is below `sharedDecimals` or the conversion exponent overflows.

- ☐ Negative test
- Reverts if either external dependency reverts or returns malformed data.
- ☐ Negative test

## Function call analysis

- `IERC20Metadata(_treasury).decimals()`
  - **What is controllable?** `_treasury` is controlled by the deployer, and the target controls its returned `uint8`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Computes `decimalConversionRate` from the returned value. Values below the shared decimal count are rejected. Very large exponents can overflow.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert or malformed response aborts deployment. Runtime code reentrancy cannot use the adapter while construction is incomplete.
- `endpoint.setDelegate(_delegate)`
  - **What is controllable?** Both `endpoint` and `delegate` are controlled by the deployer.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Success registers the LayerZero configuration delegate.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts deployment. Any callback occurs while the adapter's runtime code is not yet deployed.

## Function: `lzReceive(Origin _origin, bytes32 _guid, bytes _message, address _executor, bytes _extraData)`

The `lzReceive` function authenticates the packet, decodes the recipient and amount, and calls `bridgeMint` to mint treasury tokens to the recipient.

## Inputs

- `_origin`
  - **Control:** Set by `endpoint`.
  - **Constraints:** `msg.sender` must be `endpoint`, and `origin.sender` must equal the configured peer for `origin.srcEid`.
  - **Impact:** The remote path and packet metadata used for authentication.
- `_guid`

- **Control:** Set by endpoint.
  - **Constraints:** Arbitrary.
  - **Impact:** The packet and optional compose payload key.
- `_message`
  - **Control:** Set by endpoint.
  - **Constraints:** The input must be a valid OFT message.
  - **Impact:** The encoded recipient, mint amount, and optional compose message.
- `_executor`
  - **Control:** Set by endpoint.
  - **Constraints:** Arbitrary.
  - **Impact:** The executor metadata passed only to the internal receive function.
- `_extraData`
  - **Control:** Set by endpoint.
  - **Constraints:** Arbitrary.
  - **Impact:** The extra data passed only to the internal receive function.

## Branches and code coverage

### Intended branches

- Mints the decoded amount for an authenticated packet without compose data.
  - ☐ Test coverage
- Redirects a zero recipient to 0xdead.
  - ☐ Test coverage
- Calls `sendCompose` with the compose payload.
  - ☐ Test coverage

### Negative behavior

- Reverts if the caller is not the endpoint.
  - ☐ Negative test
- Reverts if the peer is missing or does not match.
  - ☐ Negative test
- Reverts if the message bytes are malformed.
  - ☐ Negative test
- Reverts if Treasury rejects the mint because of bridge role, pause, blacklist, or zero-amount checks.

- ☐ Negative test
- Reverts if `sendCompose` fails. The revert rolls back the inbound credit transaction.
- ☐ Negative test

## Function call analysis

- `TREASURY.bridgeMint(toAddress, amountLD)`
  - **What is controllable?** The authenticated remote message controls the recipient and amount denominated as a `uint64`. `TREASURY` is set at deployment.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Successful execution increases Treasury supply on this chain.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and prevents message completion. Treasury is trusted not to perform harmful callbacks.
- `endpoint.sendCompose(toAddress, _guid, 0, composeMsg)`
  - **What is controllable?** Recipient, guid, and compose payload derive from the authenticated packet.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. The endpoint stores a later compose action.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the entire receive, including Treasury minting.

## Function: `quoteOFT(SendParam _sendParam)`

The `quoteOFT` function returns default OFT limits, no fee details, and the debit receipt after dust adjustment.

## Inputs

- `_sendParam`
  - **Control:** Arbitrary.
  - **Constraints:** The adjusted amount must meet `minAmountLD`.
  - **Impact:** The quoted amount, slippage floor, destination metadata, and unused message fields.

## Branches and code coverage

### Intended branches

- Returns equal sent and received amounts after dust adjustment.
  - ☒ Test coverage
- Returns a zero receipt for a dust-only amount with zero minimum.
  - ☒ Test coverage
- Accepts amounts above the shared-decimal uint64 range.
  - ☒ Test coverage

#### Negative behavior

- Reverts if the adjusted amount is below `minAmountLD`.
  - ☒ Negative test

### Function: `quoteSend(SendParam _sendParam, bool _payInLzToken)`

The `quoteSend` function builds the outbound message and options, calls the endpoint `quote` function, and returns the messaging fee.

#### Inputs

- `_sendParam`
  - **Control:** Arbitrary.
  - **Constraints:** The amount after dust adjustment must meet `minAmountLD`. The destination endpoint ID must have a peer. Composed and enforced options must be valid.
  - **Impact:** The destination, recipient, amount, slippage, options, and compose bytes.
- `_payInLzToken`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The fee token choice passed to the endpoint `quote` call.

### Branches and code coverage

#### Intended branches

- Returns the endpoint messaging fee for valid inputs.
  - ☐ Test coverage
- Runs configured message inspection before quoting.
  - ☐ Test coverage

### Negative behavior

- Reverts if slippage validation fails, the peer is missing, options are invalid, the inspector fails, or the endpoint fails. The quote can be unsafe if the shared amount truncates.
- ☐ Negative test

### Function call analysis

- `endpoint.quote(MessagingParams(...), address(this))`
  - **What is controllable?** The caller controls the destination and message options within peer and format checks. The endpoint is set at deployment.
  - **If the return value is controllable, how is it used and how can it go wrong?** The MessagingFee controlled by the endpoint is returned directly to the caller.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert or malformed response propagates.

### Function: `renounceOwnership()`

The `renounceOwnership` function sets ownership to the zero address if called by the owner, disabling owner-only configuration.

### Branches and code coverage

#### Intended branches

- Clears ownership.
- ☐ Test coverage

#### Negative behavior

- Reverts if the caller is not the owner.
- ☐ Negative test

### Function: `send(SendParam _sendParam, MessagingFee _fee, address _refundAddress)`

The `send` function burns the Treasury amount after dust adjustment, encodes and sends the OFT packet, and returns LayerZero and OFT receipts.

## Inputs

- `_sendParam`
  - **Control:** Arbitrary.
  - **Constraints:** The amount must survive dust removal, meet `minAmountLD`, and be backed by balance and allowance. The destination and options must be configured.
  - **Impact:** The destination, recipient, burn amount, slippage, compose data, and execution options.
- `_fee`
  - **Control:** Arbitrary.
  - **Constraints:** `msg.value` must equal `nativeFee`. A LayerZero token fee also requires endpoint token configuration and caller allowance.
  - **Impact:** The native and LayerZero token fees passed to the endpoint.
- `_refundAddress`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The endpoint fee refund recipient.

## Branches and code coverage

### Intended branches

- Sends a valid packet after burning Treasury and paying fees, then returns both receipts.
  - ☐ Test coverage
- Removes dust before burning and writes adjusted amounts to the OFT receipt.
  - ☒ Test coverage

### Negative behavior

- Reverts if the amount contains only dust, slippage exceeds the limit, allowance or balance is insufficient, or Treasury is paused.
  - ☒ Negative test
- Reverts if the peer is missing, options are invalid, the inspector rejects the message, fee payment fails, or the endpoint fails.
  - ☐ Negative test
- Can burn the full adjusted amount and encode a truncated destination amount if the amount exceeds `uint64` shared decimal capacity.
  - ☐ Negative test



## Function call analysis

- `TREASURY.burnFrom(msg.sender, amountSentLD)`
  - **What is controllable?** The caller controls the requested amount subject to its Treasury balance and allowance. TREASURY is set at deployment.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return is consumed. Success irreversibly reduces source-chain supply once the source transaction completes.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert rolls back the send. Treasury callbacks occur before message construction but send is not protected by a local reentrancy guard.
- `IERC20(lzToken).safeTransferFrom(msg.sender, address(endpoint), _fee.lzTokenFee)`
  - **What is controllable?** The endpoint returns lzToken. The caller controls its allowance and the requested quoted fee.
  - **If the return value is controllable, how is it used and how can it go wrong?** No application return value is used. SafeERC20 validates token success.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert, false return, or malformed return propagates. A malicious token may callback before `endpoint.send`.
- `endpoint.send{value: _fee.nativeFee}(MessagingParams(...), _refundAddress)`
  - **What is controllable?** The caller controls the send parameters and refund address within configured peer and options checks. The endpoint is set at deployment.
  - **If the return value is controllable, how is it used and how can it go wrong?** The `MessagingReceipt` controlled by the endpoint is returned.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert or malformed response rolls back burn and fee transfers. Endpoint callbacks can reenter the unguarded adapter.

## Function: `setDelegate(address _delegate)`

The `setDelegate` function calls the endpoint `setDelegate` function when invoked by the owner.

## Inputs

- `_delegate`
  - **Control:** Set by the owner.
  - **Constraints:** The caller must be the owner, and the endpoint must accept `_delegate`.

- **Impact:** The delegate passed to the endpoint.

## Branches and code coverage

### Intended branches

- Calls `setDelegate` on the endpoint.
- ☐ Test coverage

### Negative behavior

- Reverts if the caller is not the owner.
- ☐ Negative test
- Reverts if endpoint validation fails or the endpoint call reverts.
- ☐ Negative test

## Function call analysis

- `endpoint.setDelegate(_delegate)`
  - **What is controllable?** `_delegate` is controlled by the owner, and endpoint is set at deployment.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. The endpoint stores the resulting delegate.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates. Endpoint callbacks occur before this function returns.

## Function: `setEnforcedOptions(EnforcedOptionParam[] _enforcedOptions)`

The `setEnforcedOptions` function writes Type 3 LayerZero options for endpoint and message type pairs.

### Inputs

- `_enforcedOptions`
  - **Control:** Set by owner.
  - **Constraints:** The caller must be the owner, and every options byte string must encode Type 3.
  - **Impact:** The enforced options mapping entries to update.

## Branches and code coverage

### Intended branches

- Stores every valid option.
  - ☐ Test coverage

### Negative behavior

- Reverts if the caller is not the owner.
  - ☐ Negative test
- Reverts if any option is not Type 3. The revert rolls back the whole batch.
  - ☐ Negative test

## Function: `setMsgInspector(address _msgInspector)`

The `setMsgInspector` function stores the outbound message inspector. The zero address clears it.

### Inputs

- `_msgInspector`
  - **Control:** Set by the owner.
  - **Constraints:** The caller must be the owner.
  - **Impact:** The contract that can veto `quoteSend` and `send message` construction.

## Branches and code coverage

### Intended branches

- Stores or clears the inspector.
  - ☐ Test coverage

### Negative behavior

- Reverts if the caller is not the owner.
  - ☐ Negative test

## Function: setPeer(uint32 \_eid, bytes32 \_peer)

The setPeer function writes or deletes the trusted remote OApp peer for an endpoint ID.

### Inputs

- `_eid`
  - **Control:** Set by owner.
  - **Constraints:** Caller must be owner.
  - **Impact:** The remote endpoint ID used as the peer storage key.
- `_peer`
  - **Control:** Set by owner.
  - **Constraints:** Arbitrary.
  - **Impact:** The authenticated remote sender.

## Branches and code coverage

### Intended branches

- Stores a nonzero peer.
  - ☒ Test coverage
- Clears a peer by storing a zero bytes32 value.
  - ☐ Test coverage

### Negative behavior

- Reverts if the caller is not the owner.
  - ☐ Negative test

## Function: transferOwnership(address newOwner)

The transferOwnership function transfers adapter ownership and all LayerZero configuration authority restricted to the owner to a new address.

### Inputs

- `newOwner`
  - **Control:** Set by the owner.
  - **Constraints:** Caller must be current owner, and newOwner must not be zero.
  - **Impact:** The new owner address.

## Branches and code coverage

### Intended branches

- Updates ownership.
  - ☐ Test coverage

### Negative behavior

- Reverts if the caller is not the owner.
  - ☐ Negative test
- Reverts if newOwner is zero.
  - ☐ Negative test

## 4.19. Module: ULTRAAdapter.sol

### Function: constructor(address \_ultraManager)

The constructor function stores the immutable ULTRA manager used for price calls.

### Inputs

- `_ultraManager`
  - **Control:** Set by the deployer.
  - **Constraints:** Must not be zero.
  - **Impact:** The immutable external price source.

## Branches and code coverage

### Intended branches

- Stores a nonzero manager address.
  - ☒ Test coverage

### Negative behavior

- Reverts if the manager address is zero.
  - ☒ Negative test

## Function: getPrice()

The `getPrice` function converts the manager mint exchange rate denominated in basis points into WAD price units with 18 decimals.

## Branches and code coverage

### Intended branches

- Returns the rate multiplied by 1e18 and divided by a nonzero denominator.
  - ☒ Test coverage
- Returns zero if the rate is zero or rounds down.
  - ☒ Test coverage

### Negative behavior

- Reverts if the denominator is zero.
  - ☒ Negative test
- Reverts if multiplication by 1e18 overflows.
  - ☒ Negative test
- Reverts if either manager call reverts.
  - ☐ Negative test

## Function call analysis

- `ultraManager.lastSetMintExchangeRate()`
  - **What is controllable?** The configured manager controls the returned rate.
  - **If the return value is controllable, how is it used and how can it go wrong?** The rate is multiplied by 1e18 before division. Excessive values overflow.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

## 4.20. Module: ULTRADelegate.sol

## Function: \_checkAndUpdateVolume(address user, uint256 amount)

The `_checkAndUpdateVolume` function resets stale day buckets, rejects swaps that would exceed `dailyVolumeLimit`, and persists the new cumulative volume for user.

## Inputs

- `user`
  - **Control:** Passed from `checkRWAIn (to)` or `checkRWAOut (from)` by `MultiliquidSwap`.
  - **Constraints:** Must not exceed `dailyVolumeLimit` after adding amount in the current day period.
  - **Impact:** Updates `userVolumes[user].volume` and `lastResetDay`.
- `amount`
  - **Control:** Swap amount from `MultiliquidSwap`.
  - **Constraints:** `userData.volume + amount` must be  $\leq$  `dailyVolumeLimit`.
  - **Impact:** Increment applied to the user's daily accumulator; zero is a no-op on the limit check.

## Branches and code coverage

### Intended branches

- Resets volume to zero and updates `lastResetDay` when the stored day differs from `_getCurrentDay()`.
  - ☒ Test coverage
- Persists `userData.volume = newVolume` when `newVolume <= dailyVolumeLimit`.
  - ☒ Test coverage

### Negative behavior

- Reverts `DailyVolumeLimitExceeded` when `userData.volume + amount > dailyVolumeLimit`.
  - ☒ Negative test

### Function: `_getCurrentDay()`

The `_getCurrentDay` function derives a day identifier from `block.timestamp` with boundaries at 06:00 UTC (14:00 Singapore time).

## Branches and code coverage

### Intended branches

- Subtracts one calendar day from the timestamp when the UTC hour is before 06:00, then returns `uint32(timestamp / 1 days)`.

- ☒ Test coverage
- Uses the unadjusted day when the UTC hour is at or after 06:00.

☒ Test coverage

#### Negative behavior

- N/A — pure timestamp arithmetic with no external calls or access control.

☐ Negative test

### Function: `_getNextResetTime()`

The `_getNextResetTime` function computes the Unix timestamp of the next 06:00 UTC volume reset after the current block time.

### Branches and code coverage

#### Intended branches

- Returns today's 06:00 UTC if that time is still in the future.
- ☒ Test coverage
- Adds one day when today's 06:00 UTC has already passed.
- ☒ Test coverage

#### Negative behavior

- N/A — view-only timestamp arithmetic with no external calls.

☐ Negative test

### Function: `checkRWAIIn(address to, uint256 amount)`

The `checkRWAIIn` function enforces ULTRA daily volume limits for the swap receiver before MultiliquidSwap deploys stablecoin on RWA-in legs.

### Inputs

- `to`
  - **Control:** Set by MultiliquidSwap as the swap receiver (not necessarily the transaction user).
  - **Constraints:** Must stay within `dailyVolumeLimit` after `_checkAndUpdateVolume` for the current Singapore-day window.



- **Impact:** Selects which address accumulates inbound swap volume; third parties can consume a victim's daily quota.
- amount
  - **Control:** Set by MultiliquidSwap from computed swap output.
  - **Constraints:** Added to userVolumes[to] — reverts if the new total exceeds dailyVolumeLimit. Zero amounts are accepted.
  - **Impact:** Size of the inbound volume increment — dailyVolumeLimit == 0 blocks all positive swaps.

## Branches and code coverage

### Intended branches

- Calls \_checkAndUpdateVolume(to, amount) and returns true when within the daily cap.
  - ☒ Test coverage
- Resets stale day buckets before applying the new amount.
  - ☒ Test coverage

### Negative behavior

- Reverts DailyVolumeLimitExceeded when to's volume plus amount exceeds dailyVolumeLimit.
  - ☒ Negative test
- Reverts if the caller lacks MULTILIQUID\_SWAP\_CONTRACT, contract is paused, or reentrancy guard fails.
  - ☒ Negative test

## Function: checkRWAOut(address from, uint256 amount)

The checkRWAOut function enforces ULTRA daily volume limits for the swap payer before MultiliquidSwap releases RWA on RWA-out legs.

### Inputs

- from
  - **Control:** Set by MultiliquidSwap as the swap payer (user on outbound paths).
  - **Constraints:** Must stay within dailyVolumeLimit after \_checkAndUpdateVolume for the current day period.

- **Impact:** Selects which address accumulates outbound swap volume in the shared per-user bucket.
- `amount`
  - **Control:** Set by `MultiliquidSwap` from computed RWA input.
  - **Constraints:** Added to `userVolumes[from]` — reverts if the new total exceeds `dailyVolumeLimit`.
  - **Impact:** Size of the outbound volume increment against the same daily cap as inbound.

## Branches and code coverage

### Intended branches

- Calls `_checkAndUpdateVolume(from, amount)` and returns `true` when within the daily cap.
  - ☒ Test coverage
- Shares the same per-address volume bucket as inbound checks for a given user.
  - ☒ Test coverage

### Negative behavior

- Reverts `DailyVolumeLimitExceeded` when `from`'s volume plus `amount` exceeds `dailyVolumeLimit`.
  - ☒ Negative test
- Reverts if the caller lacks `MULTILQUID_SWAP_CONTRACT`, contract is paused, or reentrancy guard fails.
  - ☒ Negative test

## Function: `constructor()`

The `constructor` function disables initializers on the `ULTRADelegate` implementation so the logic contract cannot be initialized outside its UUPS proxy.

## Branches and code coverage

### Intended branches

- Calls `_disableInitializers()` to lock the implementation contract.
  - ☒ Test coverage

### Negative behavior

- N/A for runtime callers — direct initialization of the implementation is permanently disabled.

☐ Negative test

### Function: `getUserVolume(address user)`

The `getUserVolume` function returns a user's consumed volume for the current Singapore-day period, the active limit, and the next reset timestamp.

#### Inputs

- `user`
  - **Control:** Arbitrary caller-supplied address for a view read.
  - **Constraints:** None — unprivileged view function.
  - **Impact:** Selects which `userVolumes` entry is reported — no state change.

### Branches and code coverage

#### Intended branches

- Returns stored volume when `lastResetDay` matches the current day identifier.
  - ☒ Test coverage
- Returns zero volume when the user's bucket is from a prior day period.
  - ☒ Test coverage
- Returns `dailyVolumeLimit` and `_getNextResetTime()` alongside volume.
  - ☒ Test coverage

#### Negative behavior

- N/A — view function with no access control reverts only on invalid internal state (none expected).

☐ Negative test

### Function: `initialize(address multiliquidAdmin, address initialIssuerAdmin, address _multiliquidSwap, byte[32] _rwaID, address _rwaAddress)`

The `initialize` function delegates to `RWADelegate.initialize` to wire Multiliquid admin, issuer admin, router role, and RWA metadata for the ULTRA delegate proxy.

## Inputs

- `multiliquidAdmin`
  - **Control:** Set by the deployer during proxy initialization.
  - **Constraints:** Must pass parent validation (nonzero, distinct from issuer admin and swap).
  - **Impact:** Receives Multiliquid admin and pause roles on the delegate.
- `initialIssuerAdmin`
  - **Control:** Set by the deployer during proxy initialization.
  - **Constraints:** Must be nonzero and distinct from `multiliquidAdmin`.
  - **Impact:** Becomes the first issuer admin for ULTRA policy such as `setDailyVolumeLimit`.
- `_multiliquidSwap`
  - **Control:** Set by the deployer during proxy initialization.
  - **Constraints:** Must be nonzero and distinct from `multiliquidAdmin`.
  - **Impact:** Sole authorized caller for `checkRWAIn` / `checkRWAOut`.
- `_rwaID`
  - **Control:** Set by the deployer during proxy initialization.
  - **Constraints:** Must not be `bytes32(0)`.
  - **Impact:** Links this delegate instance to the ULTRA RWA identifier.
- `_rwaAddress`
  - **Control:** Set by the deployer during proxy initialization.
  - **Constraints:** Must be nonzero.
  - **Impact:** Stored ULTRA RWA token address for reference.

## Branches and code coverage

### Intended branches

- Forwards to `super.initialize` and completes ULTRA delegate setup on valid inputs.

☒ Test coverage

### Negative behavior

- Reverts with parent errors on invalid or duplicate admin or swap addresses.

☒ Negative test

- Reverts if called outside the initializer context or when already initialized.

☒ Negative test

## Function call analysis

- `super.initialize(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _rwaID, _rwaAddress) -> this.__ReentrancyGuard_init() -> this.__ReentrancyGuard_init_unchained() -> this._getReentrancyGuardStorage()`
  - **What is controllable?** All init parameters are deployer-supplied; parent performs validation.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed.
  - **What happens if it reverts, reenters or does other unusual control flow?** A parent revert rolls back the entire initialization.
- `super.initialize(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _rwaID, _rwaAddress) -> this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> this._getAccessControlEnumerableStorage()`
  - **What is controllable?** `multiliquidAdmin` at init.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. Grants upgrade authority to the chosen admin.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `super.initialize(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _rwaID, _rwaAddress) -> this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> super._grantRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** Role membership is set from deployer inputs.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `super.initialize(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _rwaID, _rwaAddress) -> this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE, multiliquidAdmin) -> super._grantRole(role, account) -> this.hasRole(role, account) -> this._getAccessControlStorage()`
  - **What is controllable?** Internal duplicate-membership check only.
  - **If the return value is controllable, how is it used and how can it go wrong?** Boolean gates idempotent grant — no external impact.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization.
- `super.initialize(multiliquidAdmin, initialIssuerAdmin, _multiliquidSwap, _rwaID, _rwaAddress) ->`

```
this._grantRole(AccessControlUpgradeable.DEFAULT_ADMIN_ROLE,
multiliquidAdmin) -> EnumerableSet.add($_roleMembers[role], account)
```

- **What is controllable?** multiliquidAdmin address at init.
- **If the return value is controllable, how is it used and how can it go wrong?** Set-add result is unused.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert aborts initialization — no external calls.

### Function: setDailyVolumeLimit(uint256 newLimit)

The setDailyVolumeLimit function updates the global per-user daily swap cap for ULTRA. Setting the limit to zero disables all positive-volume swaps.

#### Inputs

- newLimit
  - **Control:** Set by an issuer admin.
  - **Constraints:** Any uint256 value is accepted, including zero.
  - **Impact:** Becomes dailyVolumeLimit for all future \_checkAndUpdateVolume calls and emits DailyVolumeLimitUpdated.

### Branches and code coverage

#### Intended branches

- Stores newLimit, emits the update event, and applies immediately to subsequent swaps.
  - ☒ Test coverage
- A limit of zero causes any positive amount to revert DailyVolumeLimitExceeded.
  - ☒ Test coverage

#### Negative behavior

- Reverts NotIssuerAdmin if the caller is not an issuer admin.
  - ☒ Negative test

### 4.21. Module: ULTRAWhitelistAdapter.sol

**Function: constructor(address \_whitelist)**

The constructor function stores the immutable ULTRA KYC contract.

**Inputs**

- `_whitelist`
  - **Control:** Set by the deployer.
  - **Constraints:** Must not be zero.
  - **Impact:** The immutable ULTRA KYC contract.

**Branches and code coverage****Intended branches**

- Deploys with nonzero dependency addresses.
  - ☒ Test coverage

**Negative behavior**

- Reverts if any required dependency address is zero.
  - ☒ Negative test

**Function: isWhitelisted(address from, address to, uint256 amount)**

The `isWhitelisted` function requires both the proposed sender and recipient to pass ULTRA KYC. The amount is ignored.

**Inputs**

- `from`
  - **Control:** Arbitrary.
  - **Constraints:** Must return true from `isKYC` for the transfer to pass.
  - **Impact:** The sender passed to the first KYC call.
- `to`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The recipient passed to the second KYC call.
- `amount`
  - **Control:** Arbitrary.

- **Constraints:** Arbitrary.
- **Impact:** Ignored.

## Branches and code coverage

### Intended branches

- Returns true if the sender and recipient pass KYC.  
☒ Test coverage
- Returns false without checking the recipient if the sender fails KYC.  
☒ Test coverage

### Negative behavior

- Returns false if the sender or recipient does not pass KYC.  
☒ Negative test
- Reverts if either KYC call reverts.  
☐ Negative test

## Function call analysis

- `WHITELIST.isKYC(from)`
  - **What is controllable?** `from` is controlled by the caller, and the whitelist controls the boolean.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns false immediately if the sender check returns false.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and prevents the recipient check.
- `WHITELIST.isKYC(to)`
  - **What is controllable?** `to` is controlled by the caller, and the whitelist controls the boolean.
  - **If the return value is controllable, how is it used and how can it go wrong?** Calls the recipient check only after the sender passes. Returns the recipient status as the final result.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

### 4.22. Module: USTBAdapter.sol



**Function: constructor(address \_priceFeed)**

The constructor function stores the immutable feed used for USTB price calls.

**Inputs**

- `_priceFeed`
  - **Control:** Set by the deployer.
  - **Constraints:** Must not be zero.
  - **Impact:** The immutable external price source.

**Branches and code coverage****Intended branches**

- Stores a nonzero feed address.
  - ☒ Test coverage

**Negative behavior**

- Reverts if the feed address is zero.
  - ☒ Negative test

**Function: getPrice()**

The `getPrice` function reads the feed answer, requires it to be positive, and scales the answer, assumed to have six decimals, to WAD.

**Branches and code coverage****Intended branches**

- Returns a positive answer converted to `uint256` and scaled by `1e12`.
  - ☒ Test coverage

**Negative behavior**

- Reverts if the answer is zero or negative.
  - ☒ Negative test
- Reverts if scaling overflows.
  - ☒ Negative test

- Reverts if the latestRoundData call reverts.

☐ Negative test

## Function call analysis

- `PRICE_FEED.latestRoundData()`
  - **What is controllable?** The configured feed controls the answer and unused round metadata.
  - **If the return value is controllable, how is it used and how can it go wrong?** Only the answer is used. A positive answer is multiplied by 1e12. Excessive values overflow.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

### 4.23. Module: USTBWhitelistAdapter.sol

#### Function: constructor(address \_whitelist)

The constructor function stores the immutable USTB allowlist contract.

#### Inputs

- `_whitelist`
  - **Control:** Set by the deployer.
  - **Constraints:** Must not be zero.
  - **Impact:** The immutable USTB allowlist contract.

## Branches and code coverage

#### Intended branches

- Deploys with nonzero dependency addresses.

☒ Test coverage

#### Negative behavior

- Reverts if any required dependency address is zero.

☒ Negative test

## Function: `isWhitelisted(address from, address to, uint256 amount)`

The `isWhitelisted` function checks only whether the proposed recipient is allowed by USTB.

### Inputs

- `from`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** Ignored.
- `to`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The recipient passed to `isAllowed`.
- `amount`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** Ignored.

### Branches and code coverage

#### Intended branches

- Returns the external `isAllowed` result unchanged.
- ☒ Test coverage

#### Negative behavior

- Returns false if the recipient is not allowed.
- ☒ Negative test
- Reverts if the whitelist dependency reverts.
- ☐ Negative test

### Function call analysis

- `WHITELIST.isAllowed(to)`
  - **What is controllable?** `to` is controlled by the caller, and the whitelist controls the result.

- **If the return value is controllable, how is it used and how can it go wrong?**  
The boolean is returned directly.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

#### 4.24. Module: UniformLabsDelegate.sol

##### Function: constructor ( )

The `constructor` function disables initializers on the implementation contract so the logic contract cannot be initialized directly, following the UUPS upgradable pattern.

##### Branches and code coverage

###### Intended branches

- Calls `_disableInitializers()` to permanently lock initialization on the implementation.

☒ Test coverage

###### Negative behavior

- Cannot initialize the implementation directly after deployment.

☒ Negative test

##### Function: `initialize(address multiliquidAdmin, address initialIssuerAdmin, address _multiliquidSwap, byte[32] _stablecoinID, address _stablecoinAddress)`

The `initialize` function performs one-time proxy setup by delegating to `__stablecoinDelegateBase_init`, which validates addresses, initializes upgradable modules, and grants protocol roles.

##### Inputs

- `multiliquidAdmin`
  - **Control:** Chosen by the proxy deployer at initialization time.
  - **Constraints:** Must be nonzero and distinct from `initialIssuerAdmin` and `_multiliquidSwap`.
  - **Impact:** Receives `DEFAULT_ADMIN_ROLE` and `PAUSE_ROLE` over this delegate.
- `initialIssuerAdmin`

- **Control:** Chosen by the proxy deployer at initialization time.
  - **Constraints:** Must be nonzero and distinct from multiliquidAdmin and \_multiliquidSwap.
  - **Impact:** Becomes the issuer admin who can manage RWA/stablecoin whitelists, fees, and custody addresses.
- \_multiliquidSwap
  - **Control:** Chosen by the proxy deployer at initialization time.
  - **Constraints:** Must be nonzero and distinct from both admin addresses.
  - **Impact:** Stored as multiliquidSwap and granted MULTILIQID\_SWAP\_CONTRACT, making it the sole caller of swap hooks.
- \_stablecoinID
  - **Control:** Chosen by the proxy deployer at initialization time.
  - **Constraints:** Must be nonzero (bytes32(0) reverts).
  - **Impact:** Stored as the protocol identifier for this stablecoin integration.
- \_stablecoinAddress
  - **Control:** Chosen by the proxy deployer at initialization time.
  - **Constraints:** Must be nonzero.
  - **Impact:** Stored as the ERC-20 used for mint, burn, and transfer operations in swap hooks.

## Branches and code coverage

### Intended branches

- Initializes access control, pausable, reentrancy guard, UUPS, and issuer-admin modules via \_\_stablecoinDelegateBase\_init.
  - ☒ Test coverage
- Stores multiliquidSwap, stablecoinID, and stablecoinAddress.
  - ☒ Test coverage
- Grants DEFAULT\_ADMIN\_ROLE and PAUSE\_ROLE to multiliquidAdmin and MULTILIQID\_SWAP\_CONTRACT to \_multiliquidSwap.
  - ☒ Test coverage

### Negative behavior

- Reverts if called when already initialized (initializer modifier).
  - ☒ Negative test
- Reverts on zero or colliding admin/swap addresses (InvalidInitializationParams, InvalidIssuerAdmin, InvalidMultiliquidSwap).

☒ Negative test

### Function call analysis

- No external calls — internal `__stablecoinDelegateBase_init` only (OpenZeppelin initializer chain).

#### 4.25. Module: VBILLWhitelistAdapter.sol

### Function: `constructor(address _whitelist)`

The constructor function stores the immutable VBILL transfer check contract.

### Inputs

- `_whitelist`
  - **Control:** Set by the deployer.
  - **Constraints:** Must not be zero.
  - **Impact:** The immutable VBILL transfer check contract.

### Branches and code coverage

#### Intended branches

- Deploys with nonzero dependency addresses.

☒ Test coverage

#### Negative behavior

- Reverts if any required dependency address is zero.

☒ Negative test

### Function: `isWhitelisted(address from, address to, uint256 amount)`

The `isWhitelisted` function calls `preTransferCheck` with the transfer context and returns true only for status code zero.

### Inputs

- `from`

- **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The sender passed to `preTransferCheck`.
- `to`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The recipient passed to `preTransferCheck`.
- `amount`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The amount passed to `preTransferCheck`.

## Branches and code coverage

### Intended branches

- Returns true if `preTransferCheck` returns zero.

☒ Test coverage

### Negative behavior

- Returns false if the transfer check returns a nonzero code.

☒ Negative test

- Reverts if the whitelist dependency reverts.

☐ Negative test

## Function call analysis

- `WHITELIST.preTransferCheck(from, to, amount)`
  - **What is controllable?** All arguments are controlled by the caller, and the whitelist controls code and reason.
  - **If the return value is controllable, how is it used and how can it go wrong?** Only the code is used. The reason is ignored. Code zero maps to true, and every other value maps to false.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

### 4.26. Module: `WTGXXWhitelistAdapter.sol`

## Function: constructor(address \_whitelist)

The constructor function stores the immutable WTGXX whitelist contract.

### Inputs

- `_whitelist`
  - **Control:** Set by the deployer.
  - **Constraints:** Must not be zero.
  - **Impact:** The immutable WTGXX whitelist contract.

## Branches and code coverage

### Intended branches

- Deploys with nonzero dependency addresses.

☒ Test coverage

### Negative behavior

- Reverts if any required dependency address is zero.

☒ Negative test

## Function: isWhitelisted(address from, address to, uint256 amount)

The `isWhitelisted` function calls the WTGXX whitelist with the transfer context and returns its boolean result.

### Inputs

- `from`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The sender passed to the whitelist call.
- `to`
  - **Control:** Arbitrary.
  - **Constraints:** Arbitrary.
  - **Impact:** The recipient passed to the whitelist call.
- `amount`
  - **Control:** Arbitrary.



- **Constraints:** Arbitrary.
- **Impact:** The amount passed to the whitelist call.

## Branches and code coverage

### Intended branches

- Returns the external whitelist result.

☒ Test coverage

### Negative behavior

- Returns false if the whitelist rejects the transfer context.

☒ Negative test

## Function call analysis

- `WHITELIST.isAddressWhitelisted(from, to, amount)`
  - **What is controllable?** All arguments are controlled by the caller, and the whitelist controls the result.
  - **If the return value is controllable, how is it used and how can it go wrong?** The boolean is returned directly.
  - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates.

## 4.27. Module: WithheldCreditsDeque.sol

### Function: `at(Deque deque, uint256 index)`

The `at` function returns the zero-based element at `index` from front to back.

### Inputs

- `deque`
  - **Control:** View callers including `toArray`.
  - **Constraints:** Must contain at least `index + 1` elements.
  - **Impact:** Read-only access to arbitrary queue position.
- `index`
  - **Control:** Caller-supplied offset.
  - **Constraints:** Must be less than `length(deque)`.

- **Impact:** Selects which withheld record is returned.

## Branches and code coverage

### Intended branches

- Returns `deque._data[deque._begin + index]` for valid indexes.

☒ Test coverage

### Negative behavior

- Reverts with `OutOfBounds` when `index >= length(deque)`.

☒ Negative test

## Function: `back(Deque deque)`

The `back` function returns the tail `WithheldCredits` record without mutating the deque.

### Inputs

- `deque`
  - **Control:** Internal or external view callers.
  - **Constraints:** Must be nonempty.
  - **Impact:** Read-only peek at the newest withheld entry.

## Branches and code coverage

### Intended branches

- Returns `deque._data[deque._end - 1]` when the length is greater than zero.

☒ Test coverage

### Negative behavior

- Reverts with `OutOfBounds` when the deque is empty.

☒ Negative test

## Function: `clear(Deque deque)`

The `clear` function resets begin and end indexes to zero without deleting underlying mapping slots.

## Inputs

- deque
  - **Control:** Internal caller only — not used on production merge paths today.
  - **Constraints:** None.
  - **Impact:** Marks deque empty while stale `_data` entries may remain in storage.

## Branches and code coverage

### Intended branches

- Sets `_begin` and `_end` to zero regardless of prior length.
- ☒ Test coverage

### Negative behavior

- Reverts on any input because the function always succeeds.
- ☐ Negative test

## Function: `empty(Deque deque)`

The `empty` function reports whether the deque currently holds zero records.

## Inputs

- deque
  - **Control:** Any caller.
  - **Constraints:** None.
  - **Impact:** Gates `front`, `back`, and merge loops.

## Branches and code coverage

### Intended branches

- Returns true when `_begin == _end` and false otherwise.
- ☒ Test coverage

### Negative behavior

- Reverts on any input because the function always succeeds.
- ☐ Negative test

## Function: front(Deque deque)

The front function returns the head WithheldCredits record without mutating the deque.

### Inputs

- deque
  - **Control:** Internal or external view callers.
  - **Constraints:** Must be nonempty.
  - **Impact:** Read-only peek at the oldest withheld entry.

## Branches and code coverage

### Intended branches

- Returns deque.\_data[deque.\_begin] when the length is greater than zero.

☒ Test coverage

### Negative behavior

- Reverts with OutOfBounds when the deque is empty.

☒ Negative test

## Function: length(Deque deque)

The length function returns the number of queued WithheldCredits records.

### Inputs

- deque
  - **Control:** Any internal or view caller.
  - **Constraints:** None.
  - **Impact:** Exposes queue size used for merge-loop bounds and griefing analysis.

## Branches and code coverage

### Intended branches

- Returns \_end - \_begin as an unsigned integer.

☒ Test coverage

#### Negative behavior

- Reverts on any input because the function always succeeds.

☐ Negative test

#### Function: popBack (Deque deque)

The popBack function removes and returns the tail record, clearing its storage slot.

#### Inputs

- deque
  - **Control:** TreasuryDelegate internal merge or admin paths.
  - **Constraints:** Must be nonempty.
  - **Impact:** Shrinks queue length by one from the back.

#### Branches and code coverage

##### Intended branches

- Returns the last record and decrements \_end after deleting storage.

☒ Test coverage

#### Negative behavior

- Reverts with QueueEmpty when \_begin == \_end.

☒ Negative test

#### Function: popFront (Deque deque)

The popFront function removes and returns the head record, clearing its storage slot.

#### Inputs

- deque
  - **Control:** TreasuryDelegate \_mergeWithheldCredits and related paths.
  - **Constraints:** Must be nonempty.

- **Impact:** Shrinks queue length by one from the front; repeated calls drive merge gas cost.

## Branches and code coverage

### Intended branches

- Returns the first record and increments `_begin` after deleting storage.

☒ Test coverage

### Negative behavior

- Reverts with `QueueEmpty` when `_begin == _end`.

☒ Negative test

## Function: `pushBack(Deque deque, WithheldCredits value)`

The `pushBack` function appends one `WithheldCredits` record to the tail of the deque in  $O(1)$  time.

### Inputs

- `deque`
  - **Control:** `TreasuryDelegate` internal storage reference.
  - **Constraints:** Must not be full (`_end + 1 != _begin`).
  - **Impact:** Grows queue length and stores the new record at the back index.
- `value`
  - **Control:** `TreasuryDelegate` caller of `_addWithheldCredit`.
  - **Constraints:** Struct fields are trusted internal accounting data.
  - **Impact:** Defines token, credits, release time, yield start, and RWA flag for one withheld entry.

## Branches and code coverage

### Intended branches

- Stores value at `_end` and increments `_end` when capacity remains.

☒ Test coverage

### Negative behavior

- Reverts with `QueueFull` when the next back index would collide with `_begin`.

☒ Negative test

### Function: `pushFront(Deque deque, WithheldCredits value)`

The `pushFront` function prepends one `WithheldCredits` record to the head of the deque.

#### Inputs

- `deque`
  - **Control:** Internal library caller only.
  - **Constraints:** Must not be full.
  - **Impact:** Grows queue length from the front.
- `value`
  - **Control:** Internal caller.
  - **Constraints:** Trusted struct payload.
  - **Impact:** New head record for withheld-credit accounting.

#### Branches and code coverage

##### Intended branches

- Stores `value` at `_begin - 1` and moves `_begin` backward when capacity remains.

☒ Test coverage

##### Negative behavior

- Reverts with `QueueFull` when the next front index would collide with `_end`.

☒ Negative test

### Function: `toArray(Deque deque)`

The `toArray` function materializes the full deque into memory for external inspection.

#### Inputs

- `deque`
  - **Control:** External view wrappers on `TreasuryDelegate`.
  - **Constraints:** Memory use scales linearly with `length(deque)`.
  - **Impact:** Can become expensive or revert for very large queues.

## Branches and code coverage

### Intended branches

- Allocates a memory array of length (deque) and copies each element in order.
  - ☒ Test coverage
- Returns an empty array when the deque is empty.
  - ☒ Test coverage

### Negative behavior

- Reverts when memory allocation fails for extremely large queues.
  - ☐ Negative test



## 5. Assessment Results

During our assessment on the scoped Multiliquid EVM contracts, we discovered seven findings. No critical issues were found. Two findings were of low impact and the remaining findings were informational in nature.

---

### 5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.